

Spreadsheets

Data Silos

Warehouse

Business
IntelligenceAdvanced
Analytics

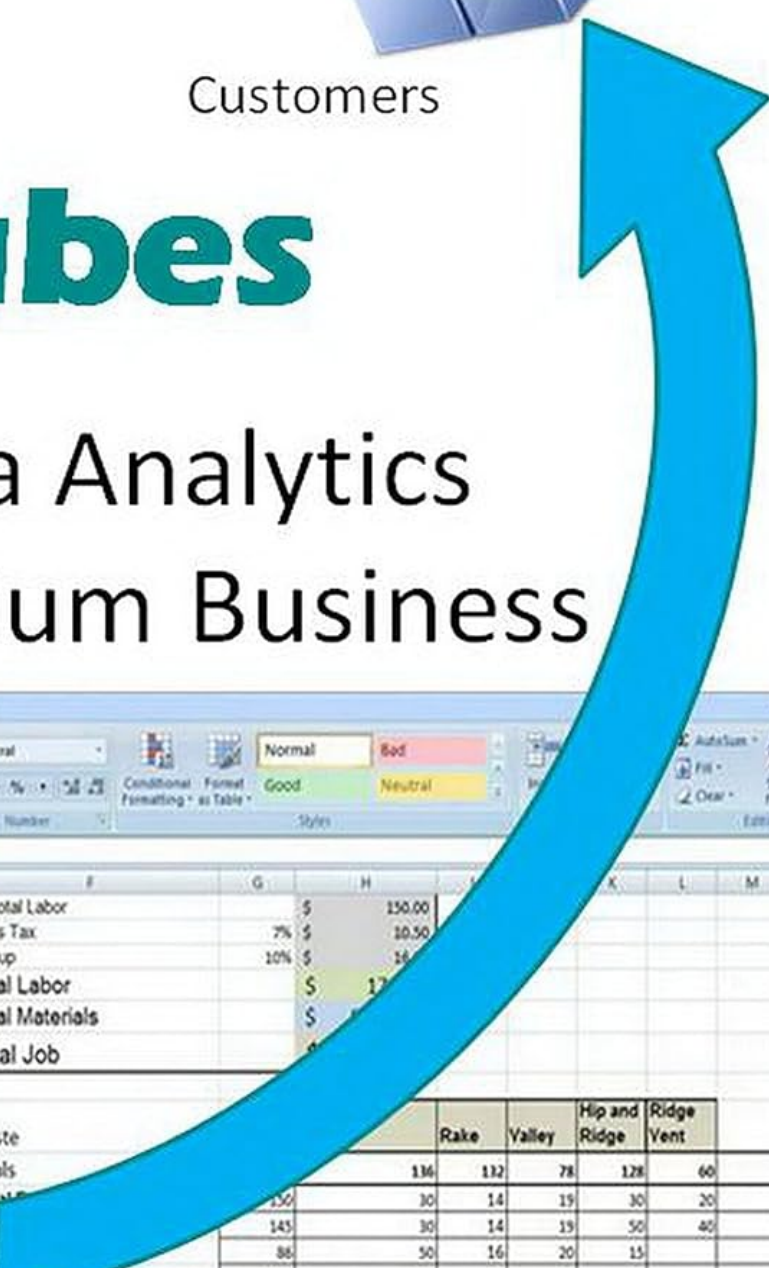
Prices

Products

Spreadsheets To B.I.-Cubes

Customers

Advanced Data Analytics
for Small Medium Business



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
4	Subtotal Materials			\$ 5,264.39		Subtotal Labor		\$ 150.00						
5	Contingency	20%		\$ 1,052.88		Sales Tax	7%	\$ 10.50						
6	Sales Tax	7%		\$ 442.21		Markup	10%	\$ 15.00						
7	Markup	20%		\$ 1,351.90		Total Labor		\$ 175.50						
8	Total Materials			\$ 8,111.37		Total Materials		\$ 1,351.90						
9						Total Job		\$ 1,351.90						
10		Eave	Rake	SF										
11	Area #1	30	14	420		Waste								
12	Area #2	30	14	420		Totals		136	132	78	128	60		
13	Area #3	50	16	800		Actual		30	14	19	30	20		addition
14	Area #4	50	16	800		Actual		145	30	14	19	50	40	main
15	Area #5	30	3	90		Valley		86	50	16	20	15		dormer
16	Area #6	30	3	90		LF of Ridge		141	26	16	20	15		dormer
17	Area #7			0		Ridge Vent LF		66		16		18		tie in
18	Area #8			0		LF of Winterguard		386		16				
19	Area #9			0		Rolls of winterguard		12		20				
20	Area #10			0		# of Pipe Flashing		2		20				
21	Roof Area Subtotal			2620		Chimney Vent Flashing		1						
22	Hip And Ridge	Bundles	5	165		# of can vents to replace								
23	Total SF			2785		Tin Shingle areas		4						
24	Total Squares			278		Satellite Dish		1						
43	Winterguard 1 Sq roll	\$ 49.00	12	\$ 588.00		8/12 simple		\$ -						
44	Metal			\$ -		8/12 with valley		\$ -						
45	Gutter Apron	\$ 7.29	15	\$ 109.35		12/12 with Valley		\$ -						
46	ODE	\$ 9.49	146	\$ 1,385.54				\$ -						
47	W-Valley 20"	\$ 23.50		\$ -		Ridge Vent		\$ -						
48	Flashings			\$ -		Cut for ridge vent		\$ -						
49	Chimney Flashing	\$ 55.00		\$ -		Tin Shingles		\$ -						

Alasdair
Gilchrist

Spreadsheets

To

Cubes

Advanced Data Analytics for the Small Medium Business

Copyright © 2017 Alasdair Gilchrist

Table of Contents

[Copyright Page](#)

[Spreadsheets To Cubes \(Advanced Data Analytics for Small Medium Business\)](#)

[Diagram of the OLAP Dimensional Model](#)

[Rule coverage versus accuracy.](#)

[Also By \[alasdair gilchrist\]\(#\)](#)

Introduction: Data Analytics and Data Science as a Strategic Asset for SMB

Chapter 1 - What is data and why is it so important?

Quantitative Data: Continuous Data and Discrete Data

Qualitative Data: Binomial Data, Nominal Data, and Ordinal Data

Chapter 2 – Deriving Knowledge from Information

Company Culture

Chapter 3 – Data management and Governance

Why Manage Data?

Chapter 4 – The Data Life Cycle

Understanding how data flows through an Organization

Plan:

Collect

Assure

Describe:

Preserve

Discover, Integrate, and Analyze

Chapter 5 -Working with Data

Data Technologies

Processing Data

Accessing and Presenting

Spreadsheets and Tables

Chapter 6 - Introducing Microsoft BI Reporting and Analysis Tools

Introduction to Excel Analytics

Self-Service Analytics

Self-Service BI

Microsoft Excel

Power Query for Excel

PowerPivot for Excel

Power View for Excel

Microsoft 3D Maps for Excel

SharePoint 2013 Sites

Power BI

Power BI Desktop

Chapter 7 –An Introduction to Relational Database Systems

Database models

Problems of multiple models in the enterprise

Multiple models, one database

Chapter 8 -Transactional Systems (OLTP)

Designing the Database

OLTP (Online Transaction Processing)

Database Design State

Chapter 9 - Operational Databases

Column-Oriented Databases

What is a column-oriented database?

Database Vendors

In-memory database solutions

Chapter 10 – Alternative OLAP Data Schemas

The Star Schema

Star Schema

The Galaxy Schema

The Snowflake Schema

Snowflake Schema

The Starflake Schema

Cubes and Multidimensional models

Chapter 11 - Designing a Data Warehouse

Data Warehouse Infrastructure Architecture

Contrasting OLTP and Data Warehousing Data Models

Data Warehouse Architectures

Creating a Logical Design

Star Schemas

Design Concepts in Star Schemas

Chapter 12 - Overview of Multi-Dimensional Data Modeling

OLAP Cubes

Dimensions and Members

Data Storage

Navigating the Cube

Slicing and Dicing and Multiple Data Views

Designing a Multidimensional Model

Data Warehousing Physical Design

Physical Design Structures

Overview of ETL in Data Warehouses

ETL Basics in Data Warehousing

BI for Data Warehouse for SMB (Retail, Logistics or with Branch Offices)

Typical SMB Data Warehouse Infrastructure

Chapter 13–Advanced Data Analysis & Business Intelligence

Chapter 14 -Data Mining for BI in the SMB

Introduction to Data Mining Algorithms

New Approaches to Data Mining

Chapter 15 - BI and Microsoft Azure Cloud Technologies

Azure Machine Learning

Azure Stream Analytics

Cortana Analytics Suite

Introduction: Data Analytics and Data Science as a Strategic Asset for SMB

Too often Data Analytics and specifically Data Science are referred to as being only relevant to big technology enterprises or Fortune 100 companies with deep pockets and large research and development budgets. However, small, medium businesses (SMB) must not just ignore the opportunity that data analytics presents as it is far too important. The business world and the way companies interact with their customers have gone through profound changes over the last decade and customers are now far more informed about the goods and services that they require. As a result it is becoming more difficult to retain customers and consequently, companies are striving to improve their products through customer personalization. However effective personalization requires knowledge about the customers. Their likes and dislikes and that data cannot be found in spreadsheets or transactional sales reports for that we need to deploy advanced techniques such as predictive or prescriptive analytics where personalization is realized through the study of enriched customer information.

Currently, many SMB still use the traditional methods of spreadsheet or transactional reporting to gain insights into their customer base and operational efficiencies. However the limitation is that only transaction activity is analysed. Those methods do not include the analysis of other high value data that when pieced together reveals a much more complete understanding of a customer.

The solution is to adopt a more thorough data analytics methodology as we now require the insights produced by analysing transactional and non-transactional data across all traditional and digital channels. This omni-channel approach allows the SMB to improve customer engagement through providing improved marketing, sales and service. This comes via on-demand access to insights and by providing recommendation services relevant to each customer. Despite most SMB understanding this they still feel that Data Analytics is out with their budgets or technical abilities. The very mention of Data Science will have them lampooning the notion as they consider all the hype and therefore the costs to be prohibitive. However, that is not necessarily the case as data science is often used as an umbrella term that covers the more precisely defined disciplines of Business Intelligence, Data Analytics or even Data Science itself. These disciplines by definition do not need to be based upon exorbitantly priced technology or industrial scale infrastructure or an expensive suite of applications. Instead data science can be just about performing research and investigation using common algorithms or processes built on spreadsheet or public cloud infrastructure and that does not require a team of data scientists backed with a research budget to make a financial director's eyes water. Instead, Data Analytics can be simply the process of investigating collected, acquired or generated data for probability modeling, to reveal insights, causations, and correlations or to predict trends. Therefore, as data analytics is not dependent on a technology, software or even an industry best practice hence it is within the budgets and wherewithal for SMBs as much as it is with international enterprises and web scale technology companies, the only difference is that of scale.

However, we have to acknowledge that most SMBs have limited resources available for non-core business technology, both in finance and people. These practical limitations means that they cannot be expected to splurge money on data science teams along with million dollar software and technology. Instead, SMBs will mostly have to work with what they have and interestingly they very often have just enough talent and resources to get by all they need is the motivation to get started and a few pointers along the way.



SMBs have the capability and the resources to be anywhere on the Data Maturity scale shown above, where on that scale your company resides is down to you. In this book we will examine each stage in the Data Maturity model from as the title suggests Spreadsheets to Cubes, which are multidimensional data structures use in

advanced analytics, and provide the directions for the paths that lead to data maturity, and ultimately the capacity for advanced data analytics.

Unfortunately there is one obstacle in the way of SMB data analytics, which is not so easily overcome and that involves executive or management buy in. It is all very well having executives and senior managers aware of the concepts and terms of business intelligence, and even the objectives as after all they read the industry and technology news as well as the rest of us. However, where there can be significant differences is with expectation and as any project manager can tell you managing the stakeholder expectation is key to any successful project. Therefore in this book we will see how an SMB can instigate and implement a successful Data Analytics or Business Intelligence project that will meet the expectations of executive management at an affordable price in resources and finance.

Not surprisingly, recent surveys have revealed that SMBs are much less likely to delve into the realms of Data Analytics than larger organizations, with many saying that they do not need further information from their data and that their Excel BI tools are sufficient for their needs. That may well be true and we cover Business Intelligence tools for Excel in a later section but interestingly of those SMB that are about to embark on a Data Analytics project their biggest concern is not the lack of technology knowledge or the tools it is finding a suitable strategy.

This state can arise when the hype over presentation of the technology clouds the purpose. But that should not deter us after all the business case for Data Science, Big Data or whatever term you wish to use for data analytics has not significantly changed from the Business Intelligence models utilized since 2000 albeit with the additional challenge of trying to figure out how to leverage the sheer volumes of unstructured data. This new type of digital content such as photos, sensors, videos, texts and location data amongst many of the other forms of social media and IoT (Internet of Things) sourced data presents as many challenges as opportunities but even though the sources and the format of the data may change the core business case remains the same. Fortunately, providing a strategy for creating value and extracting benefit from data analytics is one of our core aims with this book.

Chapter 1 - What is data and why is it so important?

Data is dumb, it's not about the data it's about the information (Stupid).

Data in itself is meaningless without either context or processing upon which it becomes information. That is the common explanation of data's value and why we need to process it so that it will transform into information. An example of this could be the stream of data contained within computer logs, which to the untrained eye the data is meaningless:

127.0.0.1 user-identifier frank [10/Oct/2000:13:55:36 -0700] "GET /apache_pb.gif HTTP/1.0" 200 2326

It is only when that log snippet is placed in context as being the output from an Apache Web Server log that we can actually gain any understanding from it. Then we can gain information such as the local address of the server, the identifier of the person making the request and the resource they requested. Hence we have transformed raw data, the log, into information by applying context. And this relationship between data and information is the foundation of what is called the Data/Information/Knowledge/Wisdom or DIKW pyramid.



The DIKW pyramid refers loosely to a class of models for representing purported structural and/or functional relationships between data, information, knowledge, and wisdom.

The principle of the Data, Information, Knowledge, Wisdom, hierarchy was introduced by Russell Ackoff in his address to the International Society for General Systems Research in 1989. But the actual first recorded instance of it was from the poem “The Rock” by T.S. Eliot in 1934:

“Where is the Life we have lost in living?

Where is the wisdom we have lost in knowledge?

Where is the knowledge we have lost in the information?”

The DIKW sequence as it became known reinforces the premise that information is a refinement of data and to be more precise that information is the value we extract from data. The DIKW sequence has been generally accepted and taught in most computer science courses. However, some have a problem with this sequence, specifically the middle steps in the sequence between information and knowledge. The general consensus however is that at least the first sequence is correct; as from data we derive information and so that part is accepted.

In order to be able to understand the complex domain of Data Analytics or Business Intelligence and all it entails we need to start by having a good understanding of the basic elements – data.

Data, which is the singular of datum but is typically referred to in the plural, can be defined as discrete, raw or unorganized pieces of information, such as facts or figures. Data is ambiguous, unorganized or unprocessed facts. When a system or process handles data and arranges, sorts, or uses data to calculate something, then it is processing the data. Raw data, which is the term for unprocessed data, are the raw material used to make information. Unprocessed data is the input to an information system that the system organizes and processes. Raw data is fed into a computer program as variables, or strings, that these can be the binary representation on a computer disk, or the digital inputs from electronic sensors that themselves collect data in the form of analogue signals from their environment.

Data can be qualitative or quantitative; the difference being quantitative data – which is measured and objective - can be represented as a number so can be statistically analyzed, as it can be represented as ordinal, interval or ratio scales. Qualitative data – subjective and opinion based – cannot be a number, so it is represented on a nominal scale such as likes or dislikes, etc.

Data can be represented in many formats, but as distinct pieces of information formatted in a specific way such as; character, symbols, strings, numeric, aural (Morse code) and visual (frames in a movie.) Data can also have many attributes, unverified, unformatted, unparsed. It may also have attributes regards its status; verified, unreliable, uncertified or validated.

Quantitative Data: Continuous Data and Discrete Data

There are two types of quantitative data, which is also referred to as numeric data: continuous and discrete. As a general rule, we consider counts to be discrete and measurements are continuous.

For example, discrete data is a count that can't be made more precise. Typically it involves integers and exact figures. For instance, the number of children in your family is a set of discrete data. After all there are no half children, they are or they are not.

Continuous data, on the other hand, could be granular and reduced to finer and finer levels of grain. For example, you can measure the time taken to commute to work in the morning.

Continuous data is therefore valuable in many different kinds of hypothesis tests when comparing figures. Some other analyses use continuous and discrete quantitative data at the same time as it may reveal performance such as time over distance. For instance, we could perform a regression analysis to see if the speed over per meter (continuous data) is correlated with the number of meter run (discrete data).

Qualitative Data: Binomial Data, Nominal Data, and Ordinal Data

We commonly use quantities and qualities to classify or categorize something, and it is easy to categorize quantity – but if you have to relate to Qualitative data it is not so easy. There are three main kinds of qualitative data.

There is the case where results are displayed as Binary data and this is when output is one of two mutually exclusive categories: right/wrong, true/false, or accept/reject.

There is also the situation when we are collecting unordered or nominal data, and we assign individual items to named categories that do not have an implicit or natural value or rank. For example, if I went through a list of results and recorded each that would be nominal data.

However we can also have ordered or ordinal data, in which some items are categorized so that they do have some kind of natural order, such as "Short, Medium, or Tall."

Importantly, there are also three types of Data structure which we should know about:

Structured data: This is data, which is relational and can be stored in a database such as SQL in tables with rows and columns. They have a relational key and can be easily mapped into pre-designed fields. Unfortunately, most data is not structured, and the level of structured data collected by organizations represents only about 5 to 10% of all business data.

There is of course another type of data called Semi-structured data, which is information that doesn't reside in a relational database, due to its lack of correlation but nonetheless does have structure and organizational properties that make it easier to analyze. Some examples of semi-structured data are: XML and JSON documents which are semi structured documents that do not fit an exact data type, NoSQL databases are considered as semi structured data stores.

But what is surprising is that data stored as semi structured data again only represents a small minority of business data (5 to 10%) so the last data type is the most prevalent one: unstructured data.

Unstructured data represent around 80% of all collected data in today's business. It has become as prevalent it will often include video, voice, text, multimedia content, music, social media chats and photos amongst many other formats. Note that while these sorts of files may have an internal structure, they are still considered to be unstructured, because the data they contain doesn't fit neatly in a database schema.

Actually, unstructured data is everywhere and is not just ubiquitous it is in fact the way most individuals and organizations conduct their work and social communications. After all social media chat, voice, text and video are the way that most people live, and they interact with others through exchanging unstructured data. However, to confuse things, as with structured data, unstructured data can be further classified to be either machine generated or human generated.

Here are some examples of machine-generated unstructured data:

Computer Logs: these contain information without knowledge unless you understand the context.

IoT sensors: these will provide streams of binary data from a wide variety of machine or environmental sensors

Satellite images: These include weather data or satellite surveillance imagery.

Scientific data: This includes seismic imagery, atmospheric data, and without context it's difficult to extract any meaning?

Photographs and video: This type of information includes security, surveillance, and traffic video.

Radar or sonar data: This technology allows autonomous vehicles, to take advantage of visual, audio, meteorological, and oceanographic seismic profiles.

The following list shows a few examples of human-generated unstructured data:

Chats, e-mails, documents, and even verbal conversation.

Social media data: This source of data is generated from the social media platforms such as YouTube, Facebook, Twitter, LinkedIn, and Flickr.

Mobile data: This includes shadow IT where users store data, text messages and location information in the cloud.

Website content: This comes from any site delivering unstructured content, like YouTube, Flickr, or Instagram.

The unstructured data group is a vast source of raw data and it is growing quickly and has become far more pervasive than traditional forms of documented text bases files. Unstructured data can be a problem with regards security as it is easily leaked out with the business boundaries but that is not the real issue. Social media and unstructured data such as You Tube videos, chat texts, and social media comments, are typically easily accessed but they actually allow data miners to determine the poster's attitudes and their sentiments Indeed analysis of social media comments for sentiment is a valuable form of information which could help in business decision making.

Another important distinction we must make when evaluating the business and that is what type of data we are storing and processing. There are typically a few categories of data that have very distinct characteristics and therefore we must be able to identify them and cater for their specific requirements. In most business use cases where data is being or planned to be analysed for Business Intelligence purpose data will fall into one of three types:

Transactional data: this type of data is derived from application web servers, they may be on-premises, web, mobile or SaaS applications and they will be producing data base records for transactions. These transaction focused systems are data write optimised in order to support high throughput of customer transactions whether that be sales on an ecommerce web server or process transactions on an industrial robotic controller. Their purpose is to record transactions by creating and storing data. A preferred collection and processing method for transactional data is in-memory caching and processing while being stored on SQL or NoSQL databases.

Data Files: this type of data category relates to log files, search results and historical transactional reports. The data is relatively large sized packets and is slow moving so can be managed with traditional disk/writes and database storage.

Messaging & Events: This type of data known as data streams consist of very small packets in very high volumes and high velocity which, require real time handling. This type of data can come from IoT sensors or the internet but is typically collected and managed by using Publish/Subscribe queues and protocols. Data streams may require specific storage requirements if they are related to industrial application that require real-time handling, such as stream enabled tables in NoSQL databases, or stream optimised databases that are designed for stream management, storage and processing.

In addition data also has other important attributes that we need to consider such as it can have velocity where it is fast moving and may be short lived, i.e. has very high entropy where it loses its value very quickly. For example, the revolutions per second reading obtained from a machine sensor. On the other hand some operational data is slow moving but of potentially long term value such as monthly sales figures. We can categorise these types of data as being hot or cold respectively and how we store them will depend on our design trade-offs. Generally speaking hot data is streaming data, which is processed in real-time and processed in-memory or in-cache for fast and efficient processing times. Examples of streaming data would be industrial sensors and other IoT alarms and publish subscribe messages used to control industrial equipment and operational processes. Hot data's value is measured in milliseconds and requires immediate processing.

Cold data on the other hand has high durability very low entropy and can be stored for long periods. Examples, of cold data would be file data particularly historical data like reports and previous year's transaction data which is stored and used for reference. Another characteristic of cold data is it may require only infrequent access and this is important to consider when designing appropriate storage.

A summary of the qualities of Hot and Cold data is shown in the table, with some of their respective operational qualities – some data such as transactional data will fall somewhere along the spectrum within the warm area.

	Hot	Warm	Cold
Volume	MB-GB	GB-TB	PB-EB
Item size	B-KB	KB-MB	KB-TB
Latency	ms	ms, sec	min, hrs
Durability	Low-high	High	Very high
Request rate	Very high	High	Low
Cost/GB	\$\$-\$	\$-€€	€

As we have seen there are several types of data which we are required to manage and handle proficiently and that requires an understanding of several components. Firstly we should consider the data structure, for example does it have a fixed schema which makes it suitable for standard relational SQL or is it JSON (schema free) or Key/Value style unstructured or semi-structured data in which case NoSQL or in-memory storage or processing will be the appropriate method. We will discuss this further later on, but for now all we need to understand is that we must make the correct choice that matches our requirements. Therefore there is a need to plan how often we will require to access the data and with what latency, and this will depend on the data's characteristics, is it hot, warm or cold. As a general rule we should always store the data in the same method we wish to access it, i.e. if we only require infrequent access store in cold or warm storage. Lastly, and very importantly we should ensure that we handle and manage the data in the most cost effective manner that meets our operational requirements – there can be significant unnecessary costs if you choose an inefficient or inappropriate method. We will go into all of these design choices much later when we consider cloud deployments and the plethora of tools and choices we have to fit our design requirements in a design efficient and cost effective manner.

Chapter 2 – Deriving Knowledge from Information

The European Committee for Standardization's official "Guide to Good Practice in Knowledge Management" says: "Knowledge is the combination of data and information, to which is added expert opinion, skills and experience, to result in a valuable asset which can be used to aid decision making."

In the introductory chapter to this book we saw that Information can be distilled from raw data but we did not examine in any depth how we manage this feat. The answer is of course through the application of the techniques and the subject of this book – data analytics.

Data Analytics

From spreadsheets to cubes we will investigate the data analytical methodologies and technologies that are feasible for SMBs in the pursuit of business intelligence. Most small medium businesses run on spreadsheets and that isn't an issue as they perform more than adequately for the consumers of strategic, tactical and even operational information. Indeed so successful are spreadsheets that you will find weaning executives, managers and decision-makers of their favourite analytical tool are easier said than done.

Spreadsheets provide a way for managers and executives to analyse data and importantly get that data away from the control of IT. Spreadsheets allow managers to do their own data preparation and analysis it also provides a means of self-sufficiency and the ways that spreadsheets can accomplish this feat are through:

Financial Modelling: Spreadsheets are great for the kind of assumptions and testing needed to put together month-by-month forecasts of financial performance.

Hypothesis testing: Spreadsheets are fast and easy to use so are perfect for on the spot calculations or hypothesis checking on a new data set.

One-Time Analysis: Spreadsheets are great for one-time modelling as you can quickly load source data, run an analysis, and draw conclusions quickly.

There are several types of data analytics that are commonly used in business. Descriptive analysis is the first type of data analysis that is usually conducted as it describes the main aspects of the data being analyzed. For example, it may describe how well a certain model of phone is performing by contrasting its number of sales compared to the norm. This allows for comparisons to be made among different models of phones and helps in decision making as it aides in predicting what stock holdings are required per model.

There is another common type of data analysis which is called exploratory analysis and here the goal is to look for previously unknown relationships. This type of analysis is a way to find new connections and to provide future recommendations and is commonly used in supermarket basket analysis to find interesting correlations between the products bought by a customer on a specific visit.

Predictive analysis as the name suggests predicts future happenings by looking at current and past facts. This sounds very grand but can also be as simple as trending analysis which graphs performance against time so that a researcher can see straightaway if there is a recognisable and predictable pattern or not.

There is also inferential analysis where a small sample is used to infer a result from a much larger sample, this method is commonly used in the analysis of voters in exit polls. Causal analysis is used to find out what happens to one variable when you change some other variable. For example, how are sales affected if a product is placed in a different location, adjacent to another product or on a higher/lower shelf?

Spreadsheets cannot provide us with all these types of analytics but they are surprisingly good at financial modelling, trend analysis and reporting and presentation. Hence their huge adoption and popularity within the financial, logistics and sales/marketing departments in the SMB market space.

Spreadsheets are used to perform elementary data analytics, which provides the methods for managers to make better decisions because if for no other reason they are based on evidence and facts. Therefore, by using common tools like spreadsheets for data analytics decision-makers can distil information from the raw data at their disposal and justify their decision. The importance of that is that not all decision makers are managers, indeed most in the work place are not, so being able to justify a decision is a handy thing.

Evaluating Information

Gaining information through data analytics however, may be a better basis for decision making than nothing – or just gut feeling – but it isn't at this level really knowledge as some quantity of that was required to understand the problem initially. Furthermore, knowledge was required to comprehend what data would be required to provide the information to prove or disprove a hypothesis, and then knowledge had to be referenced in order to comprehend and validate the information. Hence, we can consider that although Knowledge may sit above data and information in the DIKW pyramid it may not be a perfect fit. It may be considered to be an actionable component derived from the original data and information. However for now the focus is on data and most importantly how we derive information from the raw data that we collect, sometimes not just from the by-products of our operations but by the social interactions with our customers.

There has recently been an epiphany regards the worth of data and its value to an organisation. Data is something most companies, even SMB's have potentially vast quantities of, typically they have no use for it so they have previously only stored what was necessary for historical reporting and ditched the rest. Over the last decade, the cost of storage has plummeted and with the advent of Big Data and cloud storage it became both possible to store and analyse vast quantities of all sorts of data. However, cheap cloud storage supplemented with new analytical techniques and tools, which promise to reveal insights mined from this data through predictive and prescriptive analytics have changed executive thinking. Now there is potential value to even the small business in the provisioning of intelligent and informed decision making apparatus based on predictions and trends. Consequently, business and technology leaders have begun the process of building data warehouses and even in some cases data lakes in order to harvest and hoard this precious commodity of raw data. An interesting aside though is that although just over 52% of SMB companies that have actively pursued a BI or data analytics program say they derived benefits few seem so far to have generated quantifiable success. This of course may be down to the fact that only around 1% of all data they harvest is actually analysed.

For several years the procedures and techniques of data mining or the manipulation of Big Data was perceived to be the domain of big enterprises with big budgets. However, open source tools allied with cloud computing services have brought data analytics into the reach of even the most financially constrained SMB. Also for SMB to succeed in such an endeavour requires that they understand what data is, how it is analysed, for what purpose, and very importantly how it is managed. This is because for all the rhetoric data analytics is only as good as the data you feed it. With data analytics the computer maxim of garbage in-garbage out is a certainty and if we are looking for quality information then we need to ensure quality data at the source. In addition, we also need to know the right questions to ask of our data, and that is where most companies stumble – as they just don't actually know what they want.

The Information that SMB businesses require comes from many sources and has many characteristics some of these may be desirable, essential, comprehensive and accurate but others may not be so; for example the information derived may well be incomplete, unverified, cosmetic or extraneous. In order to evaluate the differing characteristics of information when considering its value we have to have a process to put that information through in order to characterize its attributes as valid information. Today we have vast amounts of new information that is being sourced from data collected from many diverse sources such as social media, online news sites and forums as well as coming in many formats such as video, text, or photographs and these are not so easily classified and verified especially when we are doing text or sentiment analysis.

To complicate matters further the marketplace is changing due to the advent of social media and ubiquitous mobile communication, which make customers far better informed. This proliferation of information comes about through active collaboration as customers will provide product reviews and exchange views on goods and services with potential customers on social media sites. As a result, customers are making more informed decisions based upon data that they feel they can trust, which ultimately makes retaining customers more difficult. Indeed many

customers in retail are now actively targeting retaining customers through improved goods and services by taking a more proactive position on social media marketing.

Therefore when we are evaluating the credibility, veracity and quality of the information we must also consider its provenance or source especially if it is coming from the internet. There are several key areas to consider:

Authority of the source or the publisher (this is especially important when taking data from social media sources)

Objectivity of the author (again social media sources can be difficult to verify and objectivity or bias hard to quantify)

Quality of the information

Currency of the information

Relevancy of the information

These were the generally accepted steps taken to verify and authenticate information in the pre-internet social media days of the printed press, journals and libraries. However these steps are just as important today when we have to authenticate information from the internet. For instance when we evaluate Authority we have to ask several questions of the source, and this does not just apply to social media authors as data can come from many diverse locations such as sensors, servers, third party distributors/aggregators and the Internet of Things, and any of these can be fraudulent;

Who is the source?

What are their credentials?

What is the sources reputation?

Who is the publisher, if any?

Is the source associated with a reputable institution or organization?

To evaluate objectivity we ask the following questions, especially of data obtained through third party brokers;

Does the author of the data, information or algorithm exhibit a bias?

Does the information appear valid and true?

To evaluate quality;

Is the information well structured?

Is the information source legitimate?

Does the information have the required features and attributes?

In addition we may take into account the Information characteristics themselves, for example, is the information;

Accurate

Comprehensive

Unbiased

Timely

Reliable

Verifiable

Current

Valuable

Relevant

These are all characteristics we should check for when dealing with information especially when sourced from the internet. Information coming from the internet especially should be checked for being timely and current and hence relevant. One of the issues with search engine algorithms is that they typically rate the most popular relevant entries, but these cannot be the most timely and current. Hence you may end up supporting your hypothesis with data that is ten years old.

Bias and completeness should also be considered, as often information is supplied in social media sites, in a one sided manner, in order to support the publisher's agenda. This phenomenon isn't particularly new as traditional printed media were at this game for centuries but it has become more pronounced and prevalent where social media on the internet is concerned. The concept of an echo chamber is how it is best described where like-minded individuals congregate and exchange their similar views, which is not a bad thing – but it is when it is to the exclusion of any opposing point of view. Of course this isn't a healthy environment and it is a Petri dish for cultures of false news and the propagation of deliberate falsehoods to support an agenda. This appears to have exacerbated the all too human condition, best explained in the Simon and Garfunkel song The Boxer, "All lies and Jest, still a man hears what he wants to hear and disregards the rest".

We must be careful how we handle Information as can be seen through the proliferation and perceived damage cause by 'fake news' as it can have many characteristics.

Within a business or organization, information may come from several sources and be categorized by the role that information will play. Hence, within a SMB we may find that some forms of information have roles and can be separated into five main categories;

Planning - A business needs to know what resources it has (e.g. cash, people, inventory, property, customers). It needs information about the markets in which it operates and the actions of competitors.

Recording – Financial transactions, Sales orders, stock invoices and inventory all need to be recorded.

Controlling – Information is required to apply controls and to see if plans are performing better or worse than expected.

Measuring – Performance in a business need to be measured to ascertain if sales and profits are meeting targets and operational costs are controlled.

Decision making – Within the decision making group we find subsets of information that further separates information into three classes, operational, tactical and strategic, which is dependent on the information's role and purpose.

(1) Strategic information: this is highly summarized information used to help plan the objectives of the business as a whole and to measure how well those objectives are being achieved. Examples of strategic information include:

Profitability of each part of the business

Size, growth and competitive structure of the markets in which a business operates

Investments made by the business and the returns (e.g. profits, cash inflows) from those investments

(2) Tactical Information: this is used to decide how the resources of the business should be employed. Examples include:

Information about business productivity (e.g. units produced per employee; staff turnover)

Profit and cash flow forecasts in the short term

Pricing information from the market

3) Operational Information: this information is used to make sure that specific operational tasks are executed as planned/intended (i.e. things are done properly). For example, a production manager will want information about the extent and results of quality control checks carried out in the manufacturing process.

In this chapter we have learned that knowledge, information and data differ mainly in abstraction, with data being the least abstract and knowledge the most. Information is data that has been organized for a specific purpose, is deemed timely and accurate and can be presented within a context that give it meaning and relevance that leads to an increase in knowledge. Conceptually information is the message, the message is made up of a sequence of symbols (data), and the message conveys some knowledge to a recipient that can understand it.

However like any other structure the pyramid is reliant on sound foundations. Hence, the importance of managing and assuring the quality of our data is vital. The discipline of Data Governance is outside the realms and capabilities of the SMB, but the requirement is not. However, in order to understand what measures can be implemented realistically within an SMB organization we need to understand the company culture.

Company Culture

An SBM can take the form of many organizational models and corporate cultures, which approach they take will ultimately decide which form of data management we will be able to implement and the approach taken, i.e. whether to go for a top down, or bottom up approach.

A corporate culture model is the organizing principle behind the values, beliefs and attitudes that characterize a company and guide its practices. Corporate cultures are often categorized by the degree of emphasis placed on various defining elements such as hierarchy, process, innovation, collaboration, competition, community involvement and social engagement.

To some extent, a company's culture model, if not articulated outright, may be categorized by details contained in the company's mission or vision statement. Small and mid-size companies with relatively flat hierarchies may apply the principles of more than one model, or change emphasize for which model to value most, depending on business needs. Larger companies with deep hierarchies may find it more difficult to institute wide-spread changes in company culture and may, instead, seek to implement cultural changes at the division level.

Popular corporate culture models include:

Adhocracy - values the ability of the company to adapt quickly to changing conditions. Adhocracies are characterized by flexibility, employee empowerment and an emphasis on individual initiative. Although corporate levels exist in an adhocracy, they are less strictly defined than in more hierarchical environments.

Clan culture - embraces a family-like or tribe-like environment that values consensus and commonality of values and goals. This type of culture often puts mentoring programs in place to foster employer engagement and loyalty.

Hierarchical - has a fairly rigid and fixed organizational structure. Hierarchical cultures depend upon structure, rules and top-down control to guide business practices and activities. An adherence to identified best practices, controlled processes and considerable oversight are considered essential to productivity and success.

Holacracy - has a flat management structure that distributes authority. The goal of a holacracy is to ensure that those responsible for completing work are given the authority to decide how that work should be carried out.

Keiretsu - extends the corporate culture to include partners. In Japanese, the word keiretsu means "group." In a company that embraces keiretsu as a culture, each member of the group—including those with independent financial interests—works closely with other members of the group to ensure each other's success.

Market culture - emphasizes competitiveness not only between the organization and its market competitors but also between employees. An emphasis on individual performance is thought to lead to greater achievement for the individual employee and, as a result, greater success for the organization.

Startup culture –here the values of creative problem solving, open communication and a flat hierarchy are strong. Startup cultures may also be referred to as innovation cultures. Workplaces that foster a culture of innovation generally subscribe to the belief that innovation is not the province of top leadership but can come from anyone in the organization.

Identifying the cultural organization will aid in successful implementation of a Data Governance program especially within an SMB environment where structure may be fluid and the dynamics less structured than in larger organizations.

However regardless of which culture the SMB embodies it is imperative for any data analytic project to realize the fundamental necessity for good data management. Without confidence in the provenance, veracity and quality of the source data then there can never be any confidence in the results obtained from it. The only way you can having reliable data is through robust and diligent data management.

Chapter 3 – Data management and Governance

In this book we will shy away from the term Data Governance as it implies a very strict and grandiose method of executive control through intimidating structures such as committees. This form of governance is a perfect fit for the traditional enterprise and government style bureaucracies but which grates with today's SMB and start-ups. Data Governance with its highly formal methods doesn't fit well with the SMB model; hence it is only really practical for very large organizations. Instead we focus on data management in the SMB and will try to take the relevant and extremely important principles of Data Governance and translate them into an SMB model of management that retains the valid principles and processes but sheds the rigid organizational structures, hierarchy and roles.

Why Manage Data?

Regardless of the size of your organization, whether that is a SMB in retail, manufacturing or research it will undoubtedly benefit the business to protect and manage data as you would any other asset. Of course that requires the understanding and perception that data is indeed an asset and has some residual value to the business.

In some cases there will be SMBs that actually collect and trade their data so the value will be prominently displayed. However, more often than not, data is merely a by-product of an operation and is collected more by default than by informed intention and in such cases the value is not always obvious to either management or even the technologists handling the data.

In many SMB environments there is a closer connection between IT and Operations than you might witness in larger organizations and this normally comes about where IT is a service support department rather than a profit centre in its own right. This is simply because the IT and operations personnel work closer to their colleagues in other departments of the company to provide a service and their roles may even overlap resulting in a better understanding of each other's requirements and constraints. In these tight-knit organisations data may well be openly shared across departments and at all levels of staff (data democracy) and its value will be recognized by both the creators and the consumers of that information. Unfortunately, the downside to this will be the lack of access control to departmental data and therefore the potential for data leakage and loss.

Conversely, in organizations with Technology centric business models where IT may determine and set policy, data access may be very tightly controlled and in many cases off-limits to other departments, which reduces its value, even if that is actually contradictory to the organizations business case.

Consequently, Business of all sizes have come to realize that there must be processes in place that allows the removal of barriers to the free flow and maximization of data value whilst at the same time ensuring its access is controlled and it is secured through diligent data management.

In general, when you have a good data management program in place an SMB will understand its data; where it is located, how it is protected, how it can be accessed, who is responsible for it, who it can and cannot be shared with, where it is going and what its value is. And that value is recognised not just in dollars and cents, but in the actual value of usability and reusability.

With a robust data management process, an SMB can have all of this information. The company can have confidence in the completeness and accuracy of its data and importantly in the quality and veracity of the reports and insights it derives through analytics. Because here is the crunch, without sufficient data management you will never really have any confidence in the veracity of the reports that each department produces. As without robust data management there will be no confidence in the information, and so the departmental transaction reports let alone the business intelligence reports will not be trustworthy.

SMBs therefore should look to establishing how they will collect, document, organize, manage, and preserve their operational data as part of their security audit and should look to include a procedure into future projects as part of the overall project plan as this has many benefits. Initially this may seem an onerous task but once accomplished you will spend less time on data management and more time on providing services, support and research.

Unfortunately many SMB's enthused by the hype driving Data Analytics or BI dive straight in and try to implement self-service Business Intelligence tools that have no clearly defined methodology and then wonder why they do not get the results that they had hoped for. The problems typically can be traced back to poorly defined or documented metadata and this combined with the lack of managed data will make any self-service BI very difficult to implement.

Unfortunately, Business Intelligence, and Data Analytics are not mature technologies where several decades of experience has produced out-of-the box solutions and this is what most people expect BI tools to be. They expect that these self-service tools or cloud based APIs will somehow return splendid graphs, displaying business insights and discoveries of cunningly disguised causations and correlations with only a few clicks of the mouse.

Instead the reality is much more mundane and a lot of work still must be done manually such as cleaning and preparing the correct type of data before you can even consider running the analytical algorithms. Importantly by implementing a diligent data management policy throughout the organization an SMB can find, use, and analyze that data much more efficiently and it will be easier for their stakeholders and partners to understand and use the data thereby increasing its value.

Further, although data management plans may differ, dependent on the nature of the business, in format and content, several basic elements are central to managing data effectively. Ideally, data should be managed so that any interested and authorized person and not only the collector or data originator can be able to discover, use and interpret the data, even after a considerable period of time has passed. Consequently, a key component of data management is the comprehensive description of the data and contextual information that future data users and analysts may need to understand and use the data, this is called metadata.

Metadata describes data it is this description, which is particularly important when storing and managing data. Metadata is important because storing large data sets can be expensive and tedious and the natural tendency is for the information content inherent in a data set or database to undergo entropy over time (i.e. data will lose value over time), ultimately becoming meaningless to analysts and other users. For example in retail, retaining the last five years data for sales is mandatory for analyzing year on year performance, but do you need the last ten or fifteen years of data, is it even relevant to today's products? Metadata labelling, categorizing and indexing of data-sets can aid in answering those questions and by viewing metadata analysts can quickly see what data is relevant to their project and what isn't. However in manufacturing or scientific research an effective data management program may well extend to 20 years or longer in the future to discover, access, understand, and use particular data. Hence, in this chapter we will summarize the elements of a data management program suitable for an SMB that would satisfy this timescale but it is up to the individual business to recognize and implement the steps necessary to prevent data entropy and mitigate the storage of worthless data.

Compliance, like Governance, is another word that can make many in IT and Technology shudder and with good reason. Today, many governments and industry watchdogs are coming down hard on organizations that do not handle data in a secure and best practice manner. In general, - although in many countries personal privacy is being eroded without much public resistance - governments (ironically) and industry watchdogs are becoming far more rigorous in demanding the assiduous storage and protection of customer's personal information. In the EU the General Data Protection Regulation (GDPR) is insisting on a very high and demonstrable standard for businesses providing services or monitoring users within the European Economic Area (EEA) that are handling EU citizens' personal data. The loss or compromise of customer personal information may result in businesses facing massive penalties, so the E.U.'s message is clear, only store personal data if it is required, and if so protect it diligently. Those goals can only be achieved through robust data management.

Last but not least, as to why we should control data is that data management is not some mundane housekeeping operation that is no longer a requirement in the age of Cloud Computing and just about everything as a service. Indeed the popular but erroneous view that technology start-ups are gung-ho and cavalier operators relying solely on inspiration to drive their innovation could not be further from the truth. For the successful amongst their ranks innovation has been built on a foundation of solid data management practices that recognize the importance and worth of data analytics to the success of the business.

Now that we have seen a few high level reasons why SMB's should implement Data Management processes and procedures let's move on to see how this can be achieved in practice. This section highlights the basics of data

management. It provides guidance about how to organize, manage, and preserve your data.

However, before we dive in deeper it is prudent to keep in mind that implementing a data management program even or perhaps more specifically within an SMB environment will not always go easily. Therefore it is best to endeavour to start out with a coalition of the willing as at least that will have a greater chance of success if everyone is at least on board with the initiative to begin with. The problem being as with many cross departmental initiatives politics and rivalry can soon raise their ugly heads making progress difficult and agreements hard to broker. Therefore a sense of realism is essential, you can only achieve so much from a bottom up approach and only then if good-will is in plentiful supply. In case where a rule or regulation needs to be agreed and implemented across several departments it will perhaps require a top down approach.

Chapter 4 – The Data Life Cycle

From a business perspective we can look through the Data Life Cycle in the same way that we can consider a project management or a software development life cycle so if we take a high level overview of the data life cycle we can see that it has eight components:

Plan: description of the data that will be compiled, and how the data will be managed and made accessible throughout its lifetime

Collect: observations are made either by hand or with sensors or other instruments and the data are placed into a digital form

Assure: the quality of the data are assured through checks and inspections

Describe: data are accurately and thoroughly described using the appropriate metadata standards

Preserve: data are submitted to an appropriate long-term archive (i.e. data centre)

Discover: potentially useful data are located and obtained, along with the relevant information about the data (metadata)

Integrate: data from disparate sources are combined to form one homogeneous set of data that can be readily analyzed

Analyze: data are analyzed

However, when starting out designing and implementing a data management process that will fit the SMBs individual culture and characteristics it is first necessary to conduct a Data Flow Analysis.

Understanding how data flows through an Organization

The initial step that any organization must take when contemplating the implementation of a data driven business model is to identify and classify their existing data. This is important because many organizations have little data governance in place and their data generating and processing systems have generally evolved over time or through acquisitions. This leaves many businesses with little idea let alone knowledge as to what data they process or even where it is kept. The way the organizations come to terms with this dilemma is through a discovery and documentation process termed a data flow analysis.

Data Flow Diagrams

Data flow diagrams assist in identifying and visualizing business logic and processes when analyzing a system. At its simplest, a data flow diagram looks at how data flows through a system. It concerns things like where the data will come from and go to as well as where it will be stored. However, it does not have information about the processing timing (e.g. whether the processes happen in sequence or in parallel).

Data Flow Diagrams Components

External Entity

An external entity can represent a human, system or subsystem. It is where certain data comes from or goes to. It is external to the system we study, in terms of the business process. For this reason, people use to draw external entities on the edge of a diagram.

Process

A process is a business activity or function where the manipulation and transformation of data takes place. A process can be decomposed to finer level of details, for representing how data is being processed within the process.

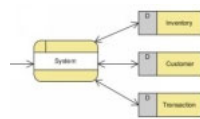
Data Store

A data store represents the storage of persistent data required and/or produced by the process. There are several examples of data stores: membership forms, database table, etc.

Data Flow

A data flow represents the flow of information, with its direction represented by an arrowhead at the end(s) of flow connector.

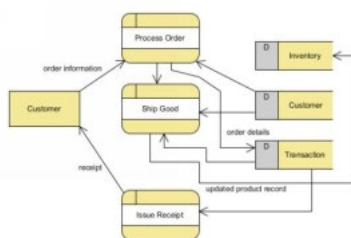
Together, all these diagram elements can be used to form Data flow Diagrams such as the one below showing three data sources;



A Contextual Diagram

Data Flow Diagrams start off as contextual diagrams like the one above which is a simple representation of the whole system. To elaborate more we drill down to a Level 1 diagram and we expanded by adding detail to show the major functions of the system.

A Level 1 Data Flow Diagram



Drilling down further to a Level 2 diagram adds more detail and processes for analysis. Data Flow Diagrams can go down to layer's 3 and 4 but that is not common.

Data Flow Analysis has become more important lately due to increasingly strict regulations regarding individual's personal data and their rights to privacy. Regulations such as the EU GDPR has made collecting and processing personal data a high risk business and organizations will need to wake up to the fact that individual's personal data must be handled transparently, meaning that they must be informed of the specific purpose for collection, the length of time it will be retained for and who it may be shared with. Consequently many organizations with international business scope are reviewing their data handling programs to ensure that they follow the best practices of minimization and transparency.

Ultimately a Data flow analysis is important because it will highlight the location of stored data and every touch point as it flows through the organization. By doing this a data flow analysis indicates who interacts with the data at each stage, why and what they do with it and importantly should they be interacting with it? It is necessary to

document these data flows as it will be required for any evidence of compliancy but more relevantly during categorization and labelling of the data at a later stage.

With the Data Life Cycle stages in mind we can then focus on applying Data Management best practices throughout each relevant stage of the life cycle.

Plan:

During the Plan stage we can introduce into the project our required data management techniques and practices. These are best introduced at the proposal stage as the project is being developed. This will be an ongoing process that will require frequent revisits during the life-cycle of the project and making changes as necessary. We should consider the following:

- Data collection: Based on the project objectives and the data collection plan, what data will be collected or generated?

How will the data be collected and analyzed? Provide descriptive documentation of collection rationale, obtained permissions for which analysis objectives and methods as well as any relevant contextual information. Remember Personal Identifiable Information PII requires specific security measures and in some cases explicit permission to collect (The EU for example). Also consider what techniques will be used to collect and verify the lawfulness of the data collection methods? What cyber infrastructure will be required to protect and secure the project data at rest and in transit? Also determine how long you need to retain the data for and for what purpose?

- At this stage it will also be necessary to decide on a suitable data repository: In order to select the correct option of data repository (i.e. spreadsheet, SQL or NoSQL database, cloud or data centre) that is most appropriate for the data you will collect, share or generate and for the community that will make use of the data. Talk with colleagues and projects sponsors about the best repository for the type of data you will be handling. Check with the repository, i.e. if it is a third party such as a cloud service provider or to IT if it is a local database server about requirements for submission. These may include required data documentation, metadata standards, and proof of lawfulness if the data is PII and any other possible restrictions on handling (e.g. intellectual property rights). Repository specifications, especially those under the auspice of GDPR in the EU, will help guide decisions about what data practices you will need to adhere to.

- Organizing, Categorize and label your data: You must decide early on how data will be stored and what formats will be used, and the types of data products for example reports you will generate. Consider whether a relational database or other type of NoSQL database might be more appropriate or even use another data organization strategy might be most appropriate for your project.

- Describing your data: How will you produce a metadata record? Where will you deposit the metadata? What standards will the repository require, what tools might be needed? Consider your community's and repositories standards when deciding on the metadata standard and data centre.

- Sharing your data: Develop a plan for sharing data with the project team, with other stakeholders, partners and business units, and whether the data may be shared with the broader open source community. It is imperative that you document under what conditions data will be released to each of these groups especially if it is PII?

- Preserving your data: As files are created, implement a short-term data preservation plan that ensures that data can be recovered in the event of file loss (e.g., storing data routinely in different locations).

- Consider your budget: What types of personnel will be required to carry out your data management plan, don't try to do this all by yourself or with just a skeleton crew, make sure you have sufficient resources to handle the data with proper diligence. What other expenses might be necessary, such as data centre cloud subscription

payments? As with any other project component the budget prepared for your project should include estimated costs for data management.

Collect

It is important to collect data in such a way as to be lawful and to ensure its usability later. Careful consideration of methods and documentation before collection occurs is important.

- Consider the lawfulness of the process for collecting any data that may be considered personal and under any privacy protection regulations. PII (personally identifiable information) requires special treatment and consideration – only collect and handle this data if it is essential. If you require the owner's permission make sure the process is documented and verifiable.
- Consider creating a template for use during data collection. This will ensure that any relevant contextual data are collected, especially if there are multiple data collectors. Ensuring clean data is collected and entered is one of the biggest failings in most data collection projects.
- Use consistent data organization:

Data can be collected in row or a spreadsheet format where each row in a file represents a complete record and the columns represent all the parameters that make up the record. This makes it easy to enter into a spreadsheet or a relational database.

Assure

Perform basic quality assurance and quality control on your data, in practice around 80% of the time spent in running data analytics is spent on cleaning and preparing data for analysis. Much of this time can be reduced by verifying during data collection, entry, and analysis that the correct data is being entered. If possible identify values that may be ambiguous and enforce strict field verification on database entry to ensure only the correct syntax is permitted.

Describe:

Comprehensive data documentation (i.e. metadata) is the key to future understanding of data. Without a thorough description of the context of the data file, the context in which the data were collected, the measurements that were made, and the quality of the data, it is unlikely that the data can be easily discovered, understood, or effectively used. Consider the following when documenting your data:

- Describe the digital context
- Name of the data set
- The name(s) of the data file(s) in the data set
- Date the data set was last modified

By maintaining comprehensive metadata descriptions this enables others to discover, understand, and use your data in the future. As a result, the Metadata should be generated in a common metadata format commonly used such as metadata editing tools, Metavist, Mercury Metadata Editor, or Morpho to generate comprehensive descriptions of the data.

Preserve

Work with a data centre or cloud provider service that is experienced in managing SMB business intelligence projects. After all they will provide guidance as to how to prepare formal metadata, how to preserve the data, what file formats to use, and how to provide additional services to future users of your data. They should also provide guidance for the most appropriate and cost effective storage solutions for your data. For example, they will be able to advise as to, which database stores provide the best solution for hot, warm and cold data. An example of this is

Amazon AWS services, which has a whole plethora of tools such as Kinesis for hot streaming data, S3 for warm data and Glazier for cold slow moving data.

Discover, Integrate, and Analyze

A variety of tools are available that support data discovery, integration, analysis, and visualization. These tools and techniques will be covered in detail later.

However it is important at this early stage to not become beguiled by the technology or the currently trending hype. Many SMBs jump on the hype bandwagon of the technology train without any real consideration for their actual requirements. This is not to say that an SMB does not require a cloud based Hadoop storage array or a Spark analytic platform but it will be the exception rather than the rule. For most SMB use-cases traditional data warehouse architecture will more than suffice and make the implementation and learning curve far less onerous.

There are cases where the nature of the business of some SMB where Big Data requirements will be necessary especially if processing very large quantities of images or non-structured streaming data but for most SMBs it is just irrelevant hype as they do not need a solution capable of processing true Big Data quantities of raw data. Instead most SMBs still work on multiple small disparate systems and use excel spreadsheets to collate information on a monthly basis. So they require assistance in finding their way around databases, perhaps before eventually progressing towards a data warehouse to provide a homogeneous data source for data mining. Perhaps they may progress towards a cloud based Hadoop ecosystem, with a Cassandra database for large scale transactional operations or NoSQL systems for unstructured data. But then they would need data analytical tools such as MapReduce or Spark to interrogate and discover insights from these cloud data storage systems. However, that is where the issues can arise, as all those names are readily reproduced in discussions on data analytics but their respective purpose is not intuitive or easily understood. Furthermore, often the SMB will be given short technical descriptions of each component in the cloud architectures role or functionality that does not clarify the position but simply adds more layers of unnecessary complexity and confusion to the architecture. The danger of delving to deep into the hype and plethora of technologies currently available is in comprehension and communication. Consequently many SMB IT people fail to convince executives of the use case for Data Analytics as they are sketchy on business potential benefits but worse unconvincing on the technology front as they cannot articulate the requirements of the architecture. Hence executives steer clear of data analytics as they cannot justify the time nor expense on something they cannot understand. However it doesn't have to be this way and with a little time and effort learning some cloud and computer science architectures it can all be explained in a manner (hype-free) that is a lot clearer for the executives.

Chapter 5 -Working with Data

In business both spreadsheets and databases are used to store and manage sets of data and manage information. After all the basic content in a spreadsheet or a database is a set of data values that we wish to store and perhaps share. Where spreadsheets and databases vary is in how they store and manipulate the data. A spreadsheet stores data values in cells, with multiple cells represented in a system of rows and columns. Cells can refer to other cells, and the spreadsheet can include cells that carry out processing on other cell values. A database typically stores data values in tables. Each table has a name and one or more columns and rows. A row in a table is called a record. A single record includes a value for each column in a table. Databases can enforce relationships between records in different tables.

Data Technologies

Spreadsheets and databases generally use different methodologies and structural technologies. The most widely used spreadsheet program is Microsoft Excel, which is part of the Microsoft Office suite of programs and it has evolved to provide business intelligence functions. Other spreadsheet programs form part of OpenOffice and GoogleDocs suite of office applications. Regardless, all the spreadsheet technologies are designed for the trained individual's ease of use. On the contrary, databases are designed to be distributed, run on servers perhaps in a data centre and perhaps accessed over wide area networks, including the Internet. In most cases however, a database will therefore be inaccessible to the individual but it will have a software application built on top of it – a management system - providing user access to the data via an accessible language such as SQL.

Databases, which may include Microsoft Access, Oracle, MySQL and SQL Server, amongst many others, are therefore designed, built and managed by software programmers, Web developers, and Database administrators. These skilled individuals often design the database using a relational database structure and access the data using a language such as SQL (Structured Query Language). Database applications can be written in many programming languages, including Java, PHP, ASP and others but they typically are relational database management systems.

Of course a database is simple a repository for data which is stored within a predefined schema and as the data is stored in such a predictable manner it can be accessed via a query language. This of course makes data in a database as theoretically accessible to a user as the data stored within a spreadsheet or an Excel table although sometimes experience and accessibility may favor the latter.

Processing Data

As we can see a database may not be so accessible to the user as spreadsheets or tables. Indeed both spreadsheets and tables offer a range of processing functions that are far more suited to the typical office use. After all users have been taking advantage of Excel and the like, to manipulate, sort and filter data as well as to produce business reports since the first IBM PC reached the desktop. Many, if not all financial or business users are capable in Excel, even if it doesn't mean they are experts in writing formulas or devising spreadsheets they are capable of handling and manipulating data in order to produce reports. Indeed it is one of the most annoying facts of business life that even though analysts or database administrators provide detailed and complex SQL queries and reports that extract and present the required data as requested, the most common request is for them to present it in a spreadsheet format so it can be directly imported into Excel. This is the power of spreadsheets, users want the data and they want to manipulate, format and present that data as they see fit and to their own timelines - free from the burden of requiring IT cooperation.

Databases on the other hand offer a greater range of flexibility, longevity and agility but unfortunately also complexity in terms of data manipulation and this must be expressed in programming or SQL code. In essence, for basic data processing, spreadsheets provide users with a range of automated functions, analysis and reporting, which are accessible to people who do not have much technical experience but have vast operational experience i.e. they know exactly how the business runs – such as finance and sales. The problem is that some data management requirements and reports are only possible using databases, and this requires a totally different set of skills. Those skills are also in short supply or the data sources have restricted access for example only to IT but this makes other departments reliant on IT to create reports and this is not very flexible at all – hence the preference to take a DIY approach using spreadsheets such as Excel.

Accessing and Presenting

Access to a spreadsheet is sometimes limited to one person at a time, the sheet is at times updated and shared but the veracity or the integrity of the data becomes dubious. This is a major weakness with individually prepared and shared spreadsheets and consequently they are poor for collaboration.

With database management systems, however, several people can access the same data set. Such systems manage the editing of data so that two people cannot alter the same data values at almost the same time. This preserves data integrity, which is harder to manage when using spreadsheets. Therefore data integrity and presentation can vary with spreadsheets and databases and there can be a conflict of information. Consequently, with a spreadsheet, you can generate charts and graphs using automated software tools and manipulate the figures on your desktop spreadsheet to suit. With a database however, such output is normally a result displayed by the data base application. Databases generally require a higher level of technological expertise to both use and abuse.

Spreadsheets and Tables

In an SMB business the spreadsheet is king, every manager wants to work with them and no matter how diligently you construct and provide reports from the operational systems such as ecommerce, CRM or ERP they still want the reports to be in Excel format. Therefore it is sensible to first address SMB business intelligence from a spreadsheet focus - after all that is the tool used by the majority of the business audience. So can we run effective Business Intelligence using spreadsheets?

Surprisingly, much to the ire of IT, the answer is a resounding yes!

In order to examine how this is not just possible but fully feasible we need to examine the tools and techniques that already exist and are common place within the SMB office environment. For example, Microsoft Excel, the ubiquitous spreadsheet application favored by accountants and managers has capabilities for business data analytics that may surprise you.

Somewhat confusingly the analytical tools that Microsoft have developed and released have been done so in a somewhat haphazard fashion for example as add-ons for Excel over several versions. The issue with SMB is that very often these organizations will be using often outdated versions of Excel and are often reluctant or slow to upgrade. For example, Microsoft's move from static spreadsheets to pivot tables and then the quantum leap to Power Pivot Tables combined with Power Queries, which are external data connections, which facilitated import of data from diverse data sources was a huge leap that embraces data analytics, but few SMBs were aware let alone using the technology.

As a result the Business Intelligence enabler tools in question, Power Query, Power Pivot, Power View and Power BI, which were initially released as separate tools are now available under Excel 2013. For those SMB that do not have that version then they are also available as a free download called BI for the Desktop, which is a collation of all the Power tools and a tight integration with the Azure Cloud based BI analysis and presentation suite, BI.com.

Consequently, all those BI tools that Microsoft has developed for Excel over the years to become a very capable data analytical tool for business intelligence are now available as a single free download and this is especially useful for SMBs. If at the time of writing (2017) we consider what tools Excel provides us to perform not just basic but quite advanced data analytics then it is quite astonishing. Hence the first technology that we should first evaluate when considering the infrastructure, tools and methodology regards implementing a Business Intelligence project for an SMB organization is to take a good look at the options already freely available to us and most importantly have huge business user buy-in – notably in the Excel's BI add-ons.

Chapter 6 - Introducing Microsoft BI Reporting and Analysis Tools

As this book is focused upon SMB and their relative constraints with regards finance and resources we take a perspective regards the most cost effective BI tool available. Hence we do so by learning about the different strengths of the analysis and reporting tools on the market. However, just about every small medium and even large business operates using spreadsheets and the most prevalent when it comes to analysis and reporting is Excel. Regardless, of the operational applications in use and their reporting capabilities almost all managers ask for reports to be in Excel format. Therefore it is sensible for most SMBs to consider Excel and its BI add-on tools as an appropriate Business Intelligence tool-kit. Fortunately for the SMB managers Microsoft does provide a variety of BI tools that can address key workloads, which enables advanced enterprise level data analytics.

Introduction to Excel Analytics

Self-service analytics is the intent of most SMB business intelligence programs as the aim is to assist small businesses with choosing the best-fit business intelligence (BI) tools that meet their organization's BI needs. In most businesses Excel spreadsheets are the predominant mechanism for reporting and analysis, and just about every head of department spends hours if not days collating the data and manipulating the figures from a vast array of sources in producing these monthly management reports. Excel being the flagship product for accountants and managers has many if not all proponents and their expertise in producing spreadsheets means that it is difficult if not neigh impossible to shift them to another format. As a result Microsoft provides a variety of BI tools that can address existing reporting workloads yet also provide the key analysis benefits offered by more expensive and hence often unobtainable analytical software.

However, Microsoft has not singularly launched a data analysis product rather it has been an evolution from the flagship Excel product. This is not in itself a bad thing as the product has developed over time to encapsulate many on the tactical tools analysis require. For example, Excel had for a long time data connectors to other data sources other than spreadsheets and they also had Pivot tables for over a decade but few actually used them due to their unintuitive operation. Many of these issues have been addressed through Power Query and Power Pivot which made everything a lot more intuitive for the business user.

As a result, although many might still scoff at Excel being used for Business Intelligence it is actually for the SMB a fantastic tool especially for those that require self-service and independence from IT.

Self-Service Analytics

To initially address the term self-service we need to understand the inherent constraints that most SMB business units work under. Spreadsheets are wonderful for those that understand them but they do have issues, for example keeping the data up to date and ensuring the integrity of the figures when sharing a spreadsheet. They are also complicated, to such an extent that only the accountants and managers that frequently work with them actually understand how they work, macros for example, – most IT support people have no idea how these work let alone assist when something doesn't work. Furthermore, spreadsheets are terribly vulnerable to errors when updating as they are range based – for example 'sum cells 1 – 10' – so formulas need updating along with every incremental update such as when rows 11 to 17 are added - and the more complex the spreadsheet the more cells throughout the spreadsheet or workbook require the formula to be upgraded. The result being that surveys of SMBs routinely discovered that 80% of spreadsheets used in the business contained errors.

Excel worked around many of these issues by shifting to a table format, and that did mitigate many of the data integrity issues as updates worked similar to a database table structure and that did address many SMB reporting issues regarding data integrity. However, spreadsheets, even the most complex workbooks are not databases – well not in the strictest terms – so are not very good for running analytics except on present data. So how can an SMB that works mainly on spreadsheets perform historical reporting let alone predictive analysis?

The industry has come up with the following version of typical business workloads for SMB.

Self-Service BI – This as the name implies is aimed at SMBs and managers that wish to be involved in running their own free-form reporting and analysis. Being in a self-service format requires that the report criteria is easily understood and manipulated and that it enables users to integrate data from disparate sources and drill-down and understand the root cause for data anomalies. The major benefit of self-service is that it allows managers to run ad hoc reports without having to make IT requests or more likely have to develop their own application expertise to construct queries and reports. Instead they can intuitively perform their own reporting and analysis without relying on IT or others. Some typical criteria we can use to identify Self-service environments are:

Users are familiar with the business data and have strong Excel skills, and have minimal specialized technical skills or access to databases and languages such as SQL, MDX, or other query languages.

Users want to easily drill down, pivot, filter, and format the data.

Frequently information is integrated from a variety of sources, and data sources can include corporate sanctioned and IT managed data sources.

Frequently small-to-medium sized data sets are used.

Distributing the information on a regular basis is not typically the primary purpose, although the end analysis, conclusions and reports might be shared with others.

SMB BI – This is altogether at a more business umbrella level, which involves formatted reports that are typically based upon approved company data, and then shared more broadly with managers, teams, or departments. Typically these reports are overseen by a company data governance mechanism and they are distributed and monitored as being a single source of truth – an aggregation or summary of all departments' reports. The validity of these reports is subject to the integrity and quality of the reporting environment and data layer upon which the reports are built. A popular type of report is a dashboard, which allows managers to monitor the performance and health of their business in near real-time, and this often results in advanced pro-active correction action.

Another type of report is scorecards. Scorecards are highly summarized views with Key Performance Indicators (or KPIs) measured and scored against predefined targets such as a balanced scorecard. These reports are generally a part of a performance management program, and can be highly motivational and are often displayed on large wall displays. However, scorecards can also be thought provoking and result in discussions being raised with regards the results, so are often displayed in communal areas. They can also be used to measure operational performance over a longer term than the dashboard.

Advanced Analytics– This is a discipline not often used by SMB but it should be as it involves tools and techniques that are used to forecast future outcomes and behaviors, which will have a direct impact on the business. One such technique is Data mining, although Artificial Intelligence, Machine Learning and predictive analytics are some of the analytical categories that are more commonly discussed today when exploring the potential of advanced analytics.

If we take the liberty of using all those terms as being synonymous then Data mining uses mathematical analysis to derive patterns and trends that exist in data. Typically, these patterns cannot be discovered by traditional data

exploration because the relationships are too complex or because there is just far too much data. However using some clever mathematical models these obscure patterns and trends can be revealed, defined and displayed as a data model.

The following are the standard tools that an SMB can utilize to fulfill all of the above disciplines:

Microsoft Excel

Power Pivot

Power View

3D Maps

Data Mining Client for Excel

SharePoint 2013 Sites (optional)

Power View for SharePoint 2013

SharePoint 2013 Dashboards

PerformancePoint Services in SharePoint 2013

Power BI

Q & A

Power BI Apps for Mobile Devices

Groups

Data Catalog

Excel Workbooks

Power BI Desktop

SQL Server Analysis Services

Data Mining in Analysis Services

SQL Server Reporting Services

Self-Service BI

In order for SMBs to undertake a Self-Service BI project involves data analysis and reporting without reliance on IT or the necessity of additional infrastructure and additional resources. This is not as restrictive as you might first envisage as only Excel and BI for Desktop are the key requirements and some additional components are recommended but are optional, such as BI.com the Azure Cloud based analysis and visualization engine:

Typical Self-Service Infrastructure

The basic infrastructure requirements for self-service BI are minimal, and you can generally develop and share reports and information with little to no IT support, although IT infrastructure is a helpful mechanism for report distribution and collaboration. Implementations of SharePoint Server with Excel Services or Power Pivot for SharePoint enabled, allow more efficient management and distribution of the self-service analysis solutions if there is no web access to Azure BI.com.

The following tools support self-service BI.

Microsoft Excel

PowerPivot

Power View

3D Maps

Data Mining add-Ins for Excel

SharePoint 2013 Sites (optional)

Power BI

Q & A

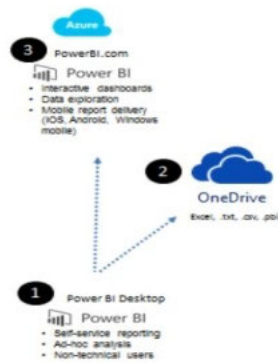
Power BI Apps for Mobile Devices

Groups

Data Catalog

Excel Workbooks

Power BI Desktop



Microsoft Excel

As you already are aware Microsoft Excel enables you to collect and analyze data, and to develop reports. However as we have previously discussed there are some drawbacks, such as maintaining data integrity and keeping the reports up to data and that is before we even consider the issues with publishing, sharing and collaboration. However Excel Services enables you to render a workbook in a browser window and refresh the associated data connection. This mitigates most of the issues with keeping data relevant and up to date as it can be updated automatically, say once a day. This is especially important where others may reference or have access to the workbook or specific parts of it, as they must have confidence in the validity of the data.

Importantly, by having a means of automating the updates from the data sources without the reliance on others to add updates or being able to modify the workbook or calculations thereby ensures that there is a single version of the workbook that is maintained and viewed by everyone. This does not mean that the individual data sources are correct, far from it, but it does ensure that the workbook is a trustworthy reflection of the data it has collected. Hence, a user can query an Excel Services workbook with confidence, in the knowledge that no structural changes have taken place to the underlying formulas that may compromise the workbooks integrity. In addition it is also possible to perform live, interactive analysis through any supported browser, including sorting and filtering of data, as well as expanding or collapsing of PivotTables. The importance of this will become clear when we investigate the strengths of Power Pivot when it comes to slicing and dicing dimensions.

The latest version of Microsoft Excel 2016 for Windows has all the functionality and features required but with some added data analysis and BI features and enhancements. Several of these new and improved features include the following.

New chart types to help you create some of the most commonly used data visualizations of financial or hierarchal information, or for revealing statistical properties in your data. The new chart types include Treemap, Sunburst, Waterfall, Histogram, Pareto, and Box and Whisker.

Built-in functionality that makes it easy to quickly get and transform your data. These new capabilities, previously only available as a separate add-in called Power Query, can be found natively within Excel.

There are enhancements to the PivotTable authoring environment, such as automatic relationship detection, automatic time grouping, and field searching.

Select multiple items in an Excel slicer on a touch device.

Share your data analysis with your workgroup or clients by publishing to Power BI with one button and then use your data models to quickly construct interactive reports and dashboards.

The “Tell me what you want to do” text field enables you to enter words and phrases related to what you want to do next and quickly get to features you want to use or actions you want to perform. You can also choose to get help related to what you're looking for, or perform a Smart Lookup on the term you entered.

With Smart Lookup, you can quickly locate definitions, Wiki articles, and top Web search results related to a selected word or phrase.

With Ink Equation you can use your finger, touch stylus, or a mouse to write math equations by hand, and Excel will convert it to text.

You can share your spreadsheet with others on SharePoint, OneDrive, or OneDrive for Business, and use History to see a complete list of changes that have been made to your workbook and access earlier versions.

Data Loss Protection (DLP) is introduced in Excel to enable real time scan of content based on a set of predefined policies for the most common sensitive data types.

Power Query for Excel

Power Query is an Extract, Transform and Load tool (ETL) that helps Excel and Power BI users connect to diverse data sources, transform the shape of the data as required and then load the data for further use, typically into Excel, Power BI, CSV etc. ETL tools as we will examine in detail later when we move on to data warehousing are important tools required to collect and transform data from diverse data sources and transform them into a common acceptable format. ETL tools are not new – there are many open source and enterprise strength tools in the marketplace as collecting and collating data from heterogeneous data sources such as Excel Spreadsheets, Access, Oracle, MySQL or the plethora of other Databases, is a common business requirement. What makes Power Query such a handy tool is it is the first tool built into Excel specifically for business users use rather than system integrators and hence is also intuitive to use yet is not short on functionality.

Power Query may not seem like a particularly exciting product after all it is basically a gallery of connectors but it is hugely useful when integrating heterogeneous data sources and that usefulness increases at the same rate as the diversity of data sources to business. Previously, working with Excel we would only consider connecting to data sources from other spreadsheets, or CVS files which were the product of transformation from a database report. What Power Query facilitates is the direct connection to many sources for the seamless – to the user – extraction, transformation and loading of data from a vast array of diverse data sources. Power Query even has connectors that let the user scrape websites and input the data into a workbook.

What Power Query can do is ETL but in plain language that means simple it can be used for the following (and probably many more things too).

A data loading tool from a wide selection of data sources

A data cleansing tool (remove unwanted columns, rows, characters etc)

A data reshaping tool including a very powerful “unpivot data” function

Further Power Query will be beneficial to users who may:

Spend hours of repetitive work combining data from multiple files (CSV, Excel) into a single Excel workbook.

Have data in a file but need to spend time “reshaping” the data before you can use it.

Write lots of VLOOKUP formulas over multiple tables of data to “join” the data together.

Manually have to “cleanse” or redact data.

Need to get data from web pages into Excel in order to analyze the data.

PowerPivot for Excel

Pivot tables have been in Excel for many years but were heavily underutilized due to several factors, lack of ease to use; lack of a compelling use case; limited data set and spreadsheet size; and lack of general awareness, are just some examples that curtailed Pivot table’s general use. Power Pivot, on the other hand is a different beast altogether as it is a user-friendly data modeling tool that lets you create data models by slicing and dicing dimensions, thereby establish relationships and cutting out the noise. Slicing and dicing lets you focus only what is important and this is especially useful when creating and using calculations. Power Pivot also allows you to work with very large data sets, build extensive relationships, and create complex calculations, which are of course especially important to data analytics.

With regards data analytics we can use Power Pivot to filter data, omit columns, rename columns and manipulate source tables while importing the data – we only extract what we wish. Further, we can create perspectives to limit the number of columns and tables that your workbook consumers see and this is particularly important in today’s security conscious but share and collaboration style workplace. In addition we can write advanced formulas with the Data Analysis Expressions (DAX) expression language as well as create key performance indicators (KPIs) to use in PivotTables and

Power View reports

When you select a data set to manage using Power Pivot, you can view and manage the data model, add calculations, establish relationships, and see elements of your Power Pivot data model. A data model is a collection of tables or other data, often with established relationships among them. In addition you can also establish, and graphically represent, relationships between the data included in the model.

Furthermore by using the Data Analysis Expressions (DAX) you can now define custom calculations for Calculated Columns and for Measures (also known as calculated fields). DAX formulas are very similar to Excel formulas. For example, to create one, you type an equal sign, followed by a function name or expression, and any required values or arguments. Like Excel, DAX provides a variety of functions that you can use to work with strings, perform calculations using dates and times, or create conditional values. DAX includes some of the functions used in Excel formulas, and additional functions designed to work with relational data and perform dynamic aggregation.

However, DAX formulas are different in the following important ways:

If you want to customize calculations on a row-by-row basis, DAX includes functions that let you use the current row value or a related value to perform calculations that vary by context.

DAX includes a type of function that returns a table as its result, rather than a single value. These functions can be used to provide input to other functions.

Time Intelligence Functions in DAX allow calculations using ranges of dates, and compare the results across parallel periods.

You can import data into a Power Pivot data model from many different data sources that might support a lot of different data types. Therefore, when you import or load the data it is converted to one of the Power Pivot data types, which you can then use in calculations and in Pivot Tables. A new data type in DAX is the table data type that is used as the input or output to many new functions. For example, the FILTER function takes a table as input and provides as an output another table that contains only the rows that meet the filter conditions. By combining table functions with aggregation functions, you can perform complex calculations over dynamically defined data sets.

You can also create formulas and relational models these are useful when you wish to manipulate the relationship between two tables of data, based on matching data in each table, even when the tables are from different sources. The Power Pivot window is an area where you can view and work with multiple table diagrams of data and connect the tables in a relational model. Within this data model, tables are connected to each other by their relationships, and this lets you interconnect tables of interest and create correlations with columns in other tables. For example, you can create formulas that sum values for a related table and then save that value in a single cell in another table.

Power Pivot is powerful and easy to use and goes a long way towards the aim of self-service as you can prototype the solution quickly before involving IT in building more traditional BI infrastructure like a formal SQL Server Analysis Services cube.

Power View for Excel

Another addition to standard Excel which is a beneficial BI tool is Power View, which is a data visualization tool that lets you create interactive charts, graphs, maps, and other visual presentations. Power View is an essential add-on for BI because the presentation of results is critical to the understanding and hence the perceived value to the audience.

In Excel 2013, when you add tables and create relationships between them, Excel is creating a Data Model, which is a collection of tables and their relationships. These data models reflect the real-world relationships between business functions and processes—for example, how Products relates to Inventory and Sales. You can modify and enhancing data models in Power Pivot, however you can also use them for the basis to make more sophisticated reports by using Power View.

With Power View you can interact with data:

In the same Excel workbook as the Power View sheet.

In data models in Excel workbooks published in a Power Pivot Gallery.

In tabular models deployed to SQL Server 2012 Analysis Services (SSAS) instances.

In multidimensional models on an SSAS server (if you're using Power View in SharePoint Server)

In Power View, you can quickly create a variety of visualizations, from tables and matrices to pie, bar, and sets of charts. A table is the starting point for the visualisation which is converted easily to illustrate the data as Power View uses the metadata in the underlying data model to understand the relationships between the different tables and fields in a workbook or report. As a result of this functionality you can use Power Views understanding of relationships to create a single visualization to filter and highlight all the visualizations in a sheet or view. Or you can display the filters area and define filters that apply to an individual visualization or to all the visualizations in a sheet or view.

Slicers in Excel enable you to compare and evaluate your data from different perspectives. Slicers in Power View are similar. When you have multiple slicers on a view and you select an entry in one of the slicers, that selection filters the other slicers in the view.

You can also sort tables, matrices, and column charts in Power View. Columns are sorted in tables and matrices, with the categories or numeric values in charts, in each case, you can sort ascending or descending either on attributes such as Product Name, or on numeric values.

Microsoft 3D Maps for Excel

We will highlight another powerful but lesser known tool for BI visualization namely Power Map, the 3-D geospatial visualization add-in for Excel 2013. Power Map is now also fully integrated into Excel 2016. What is useful with Power Map is that you can create 3D Maps for Excel as it is a three-dimensional (3-D) data visualization tool that lets you look at information in new ways. 3D Maps allow you discover insights you might not see in traditional two-dimensional (2-D) tables and charts.

SharePoint 2013 Sites

Previously, to make use of all the Excel Power features it was advisable to host a SharePoint server and if you are working in a security conscious workplace and your BI results are strictly for internal eyes only then that is still a good idea. The advantages of SharePoint are that it provides an internal platform to host shared data that will be widely available but access permission will be strictly enforced.

As a result you can also use SharePoint to share work and collaborate with others in a secure manner. It is also a way to organize your projects and teams and to host internal intranet web portals.

The following are some of the types of portals that you can create.

BI Center Sites: A basic SharePoint 2013 site that contains a set of prebuilt templates, lists and libraries to organize BI content.

Document Center Sites: These are websites that are designed for the purpose of large-scale document management. SharePoint provides templates to manage large scale data repositories and support features such as document versions, document IDs, document sets, metadata navigation, and content types.

Power BI

Power BI is a suite of business analytics tools to analyze data and share insights. The first release of Power BI was based on the Microsoft Excel add-ins: Power Query, Power Pivot and Power View. However, Microsoft has built upon this framework and also added many additional features like Question and Answers, enterprise level data connectivity and security options via Power BI Gateways.

The importance of Power BI is that it can unify all of your organization's data, whether in the cloud or on-premises making it available to everyone in the organisation. Using the Power BI gateways, you can connect SQL Server databases, Analysis Services models, and many other data sources to your same dashboards in Power BI.

Power BI dashboards provide a 360-degree view (the past, present and future) for business users and it is a single location to store and analyse their most important metrics, so that they are updated in real time and are available across all of their devices. In today's business world mobility is very important and having access to your data and reports from anywhere is no longer a luxury it is a requirement and Power BI provides this with the Power BI Mobile apps, which update automatically with any changes to your data.

The new Power BI experience includes the following ...

Q & A: Use Q&A to explore your data using intuitive, natural language capabilities and receive answers in the form of interactive charts and graphs.

Groups: Groups in Power BI bring you and your colleagues together to collaborate, communicate, and connect with your data across Office 365. Create a group in either Power BI or Office 365. Then invite co-workers into this group workspace where you can collaborate on your organization's shared dashboards, reports, and datasets.

Data Catalog: You can share information about a query in the Power BI Data Catalog with all users or a specific set of users within the organization. Shared queries appear in search results when users perform an online search in Power Query. This enables the users to easily discover and reuse the shared queries and their underlying data for their data analysis and reporting.

Excel Workbooks: You can bring any Excel workbook (XLSX or XLSM) into Power BI and explore the whole file, exactly as you would in Excel Online. To view and explore whole Excel files in Power BI, your workbooks need to be on OneDrive for Business. Any changes you make to a file on OneDrive appear automatically in the file in Power BI.

Power BI apps for mobile devices

Power BI supports mobile devices and you have several options for viewing your Power BI dashboards, including viewing them in a browser on your phone or iPad. And, you can also download a Power BI app to specific devices, including an iPad app, iPhone app, Power BI app for Windows devices, and Power BI app for Android phones.

The iPad app and iPhone app for Microsoft Power BI for iOS brings the mobile BI experience to Power BI. With live, touch-enabled mobile access to your important business information, you can view and interact with your company dashboards and reports easily, from anywhere.

With Microsoft Power BI for Windows app you can view and interact with your company dashboards and reports easily from anywhere and share them with your colleagues in email.

Power BI Desktop

Delivering on the 360-degree view is not simply about having a unified database of all activity, but rather being able to pull together the pieces of information that are relevant for a specific customer and specific interaction into an intuitive workspace for the agent or the customer, Power BI Desktop lets you create a collection of queries, data connections, and reports that can easily be shared with others. Power BI Desktop integrates proven Microsoft technologies – the powerful Query engine, data modeling, and visualizations – and works seamlessly with the online Power BI service.

You use the Get Data command on the ribbon to connect to a variety of data sources such as Excel, SQL Server, an OData Feed, or Azure SQL Database. There are vast arrays of connectors available for other data sources to choose from.

With Power BI you can bring together in one place dashboards that help you track the performance of your project, business or organization, you can drill down to report details, and connect multiple datasets from a variety of data sources and you can share those dashboards with people in your organization.

The Data Mining Client includes the following features:

Use wizard interfaces to access well known, empirically tested data mining algorithms, including clustering (*K*-

Charts generated by most tools can be saved directly to Excel. Use the **Preparing Models in Excel** (COI Server) link in the left-hand column of the **COI Server** page to learn more.

Save your data mining model to the server, to use in further testing, or to deploy to a production server for greater

In order to implement such a secure data repository we need to look towards implementing a secure database server that can enforce security and privacy rules across the entire data schema. This entails enforcement rules that allow access to files only to those that have a specific business requirement and block any other users from accessing data that they have no requirement. This is of course a lot more awkward to deploy as we need to audit the users who have legitimate access but also categorize the data that they should have access too. This is easier if the organization already categorizes their users through a robust Identity and Access Management program but that is many a flight of fancy in an SMB environment. However, as much as that is rarely the case with SMBs the aim should be the same. To secure the database against unauthorized access and prevent the illicit transportation of data from the company premises whenever possible, and this is where a secure database management system comes into play.

Chapter 7 –An Introduction to Relational Database Systems

People since the stone ages have been storing and presenting information through cave paintings and carvings in stone tablets. The invention of paper and ink led to a more user friendly method of storing and presenting information as it could now be stored and consumed in scrolls, manuscripts and later in books. However as we have seen previously information is not all the same, some information requires to be handled differently than other types.

Each database management system, as we have already seen whether it be a relational or non-relational data base, has its strengths and weaknesses. Indeed, it is often the case where a designer, will choose the database system they know or are most comfortable with, and fit the data into that schema. A schema is the blueprint for the database and is the inter-relationships between tables and data. The schema sets out the names for the tables, columns and rows and which data types can be stored in each – a data type could be a small or large integer or a body of structured text such as a date or just a label or body of text such as a name or a description – hence it is a detailed plan of the database. In relational databases there is also the concept of normalisation (3NF), which strives to reduce the instances of redundant data, by moving instances into their own tables, which will improve its performance and consistency but at the cost of flexibility and complexity.

However, normalisation had its drawbacks because as the schema became more efficient i.e. to accommodate common instances such as customer names, addresses, products, prices, stock numbers, vendors, the list goes on and on so the number of instance of tables grow and grow. The result being is that the schema becomes a labyrinth of interconnected relational tables which only the database designer can possibly understand.

Furthermore, most organisations do not have just one vast database system that controls all business operations as typically there are several each with their own area of responsibility. For example there would be a separate database system for CRM, ERP, HR or ecommerce all heterogeneous (dissimilar and incompatible) systems that cannot naturally integrate and share data and this creates what is known as silos of information.

To compound the problem often these systems will have different underlying databases one might be MySQL another Oracle express and a third PostgreSQL and of course they will have their own unique data schemas as they host applications that perform completely different tasks. But of course often reports need to be run, which will require access to data from several applications. The SQL query will collect and then to collate the retrieved data from several of these systems, and that made writing report queries extremely complicated. These unfortunate disparate structures arose from the fact that several applications would favour and come preinstalled upon different databases, such as CRM on MySQL, ERP on SQL Server, or PeopleSoft on Oracle and there was nothing that could be done about that. However the problem that could be addressed was that they would each have their own analytic and reporting infrastructure that would have to be queried in turn when building a report.

A new thought-process surfaced that suggested that instead of fitting the data to the database schema, what if we let the database schema fit the data? So instead of juggling several diverse schemas, across several physical servers and then have to start joining tables from dissimilar applications we could build a central database with a schema that would accept data from all the other application databases. Hence, there would be the possibility of perhaps connecting all those troublesome data silos. And what if we just got rid of these disparate reporting servers altogether and formed a single warehouse for analytics and reporting that held a repository for all the aggregated data?

Database models

To understand how this problem was resolved and how data warehousing came into being we need to understand how database models are designed to work in more detail. However, this is not intended to be a database tutorial or primer instead it is simply a high level introduction to the underlying technologies and database concepts, so that you will be familiar with the terminology when it arises later in the book. Hence, if you are comfortable with OLTP and OLAP databases and schemas, you can skip most of the next two chapters.

A Data base “Model” refers to how a database stores and organizes the data it has been given and this term is synonymous with a schema. The design of the schema is based typically on the relationship between categories of data, such as Sales, Products, Price and Date and these are referred to as dimensions. Therefore the schema is concerned with how one dimension (a column) interacts with the data stored in a row and how that can be retrieved via a query language.

The relational model

If we initially take a look at the Relational paradigm, which is by far the most common in SMBs, as these are the SQL databases underpinning the applications many of us are familiar with. In the relational SQL database the data is stored in tables, where each row column forms a dimension, such as a product, location or cost and each row contains in the intersecting cell a value or a measure and the column represents a particular attribute of the items.

As we have seen, well-designed relational databases conventionally follow third normal form (3NF), in which no data is duplicated or deemed redundant in the system. To accomplish this lack of redundancy, any references to an entity is made using foreign keys. So, for example, in an ecommerce system with an orders table and a customer table, the row referring to a single order will reference the ID of the customer, instead of storing the customer’s information directly as this would incur many instances of redundant data.

Of course that is great when we are writing transaction data into the database as only one instance or location of the data will need updating. However when we come to reporting, the issue is that the order table doesn’t have the customer information, so a join is needed to pull that information into one query statement.

Furthermore, due to normalisation the data schema ensures that the order table can’t store a list of items in the order; that information is on the order items table. But the order items table only references product keys, so getting details like the name of the product requires looking at the products table. So, for something as simple as viewing order details, we have to join across at least five tables. Now we can clearly see why spreadsheets are as popular and prevalent in SMB environments as even to construct a simple query requires skill and intricate knowledge of the database schema. The problem is further compounded by the fact that each report or query has to be pre-constructed and even for an experienced DBA (database administrator) this is not a trivial task so it makes business-user ad-hoc queries just about impossible.

However, this deconstruction of information into the relational model has advantages:

With a homogenous data set, it is highly space efficient. This was an extremely valid point a decade or two ago when local hard disk storage capacity was at a premium. Nowadays disk capacity is vast and cheap but consumption levels are escalating. The more you give a user the more they use.

The data integrity of 3NF ensures consistency about what is considered “true” in the system. Again a valid reason when hard disk technology performance was an issue, today’s solid state drives and in-memory processing – where the entire database is loaded and runs in memory – negates much of this latency issue, but of course these systems are out with the financial budgets of SMB and even smaller enterprises.

A well-designed schema, for a domain where such a thing can exist, often reveals the shape of the domain and can guide application logic. This is very true but such is the complexity of most schemas it would take a long time of studying to become familiar with the schema let alone have it guide the application logic.

There are disadvantages, too:

Data integrity requires a high degree of dependency across tables. This can cause scaling problems because 'sharding' across multiple machines may physically separate data tables that have to be joined in a query to be useful. This is only really relevant to very large databases that are distributed across several machines, so are rarely seen in SMB environments.

Relational data schemas are complex and hard to change, and have to be designed and implemented before data is added to the system. If you can't predict what the incoming data will look like, it is impossible to create a meaningful schema for it. This can certainly be an issue with in-house applications where updates and custom changes are commonplace.

There is a mismatch between how data exists in the real world, and how it is stored in the database, and how it is used by any consuming application. This can cause a loss of context and detail, making the data less meaningful.

The integration of systems also becomes difficult as we cannot just interconnect disparate systems and schemas as we will need to translate data from one system's schema into another schema and format, and that is before we can load it and make use of it in another database or application.

As we can see integrating disparate applications that run upon databases at the backend is not a trivial task. In addition the problem if anything is growing as in order to accommodate the diverse nature and characteristics of data many specialist forms of database management systems have appeared on the scene. There are many types of databases available, which can be categorized by their content or their application;

An in-memory database is a database that primarily resides in main memory, but is typically backed-up by non-volatile computer data storage. Main memory databases are faster than disk databases, and so are often used where response time is critical, such as in telecommunications network equipment

A cloud database relies on cloud technology. Both the database and most of its DBMS reside remotely, "in the cloud," therefore the end-users access the DB through a web browser and Open APIs. The advent of Cloud databases has radically reduced the total cost of storage of data and created business opportunities for SMBs that previously – only a generation ago – would have not been feasible.

Data warehouses: These are databases used to aggregate and archive data from operational databases and often from external sources such as social media, data distributors and direct market research firms. The warehouse becomes the central source of data for end-users who may not have access to operational data as running queries and analytics on an operational database can have a profound detrimental impact. Some basic applications of data warehousing include achieving, analytics and presentation of data which, includes the retrieving, analyzing, mining, transforming, loading and management of data. The purpose is to archive and manipulate data to make them available for further use.

A distributed database is one in which both the data and the DBMS span multiple computers, we will see these later in the case of the Hadoop distributed file-system ecosystem for handling Big Data and vast data sets across an array of physical servers.

A document-oriented database is designed for storing, retrieving, and managing document-oriented, or semi structured data. Document-oriented databases are one of the main categories of NoSQL databases and we will see several of these such as Cassandra, and other popular Cloud NoSQL databases when we take a look at cloud storage solutions.

An embedded database system is a DBMS, which is tightly integrated with application software that requires access to the stored data in such a way that it is transparent to the end user. However embedded databases are also common in Retail POS systems and lately have become popular in IoT and industrial systems but they retain the same characteristics of transparency and light-weight design.

End-user databases consist of data developed by individual end-users. Examples of these are collections of documents, spreadsheets, presentations, multimedia, and other files.

Hypertext databases: In a hypertext or hypermedia database, any word or a piece of text representing an object, e.g., another piece of text, an article, a picture, or a film, can be hyperlinked to that object. This type of database is particularly useful for organizing large amounts of disparate information. For example, they are useful for organizing online encyclopedias, where users can conveniently jump around the text. A perfect if extremely large version of a distributed hypertext database is the World Wide Web.

A knowledge base is a special kind of database for knowledge management, providing the means for the computerized collection, organization, and retrieval of knowledge. Also a collection of data representing problems with their solutions and related experiences as often used by Call Centers and Technical Support.

A mobile database can be carried on or synchronized from a mobile computing device such as a mobile phone or tablet.

Operational databases store detailed data about the transactional operations of an organization. They typically process relatively high volumes of transactions which entail updates of a data base using many small 'write' procedures. Examples of operational or transactional data bases (Online Transaction Processing) include customer relation databases (CRM) that record contact, credit, and demographic information about a business' customers, ecommerce data bases that record online sales transactions, enterprise resource planning (ERP) systems that record details about product components, parts inventory, and financial databases that keep track of the organization's production, logistics, inventory and assets, accounting and financial transactions.

Probabilistic databases employ fuzzy logic to draw inferences from imprecise data.

Real-time databases process transactions fast enough for the result to come back and be processed immediately.

A spatial database can store the data with multidimensional features. The queries on such data include location based queries, like "Where is the closest hotel in my area?"

A temporal database has built-in time aspects, for example a temporal data model and a temporal version of SQL. More specifically the temporal aspects usually include valid-time and transaction-time.

As can be seen from the list the one significant driver than has changed the way we need to store data within databases has come from the types of data available. There are three types of data that we work with:

Structured – relational data that is easily stored within a relational database (approx 10%)

Semi-structured – Data that has some relational structure but does not readily conform to the relational database schema, XML or CVS for example (approx 10%)

Unstructured – Videos, Photographs, social media chats, etc (approx 80%)

Therefore we have to find suitable storage and processing methods for all of these types of data and this is especially important as the most troublesome of the three, which is unstructured data, consists of approximately 80 per cent of the data that organizations collect and now require to analyze. Previously relational databases using structured data only accounted for 10% of the data that we are now required to handle, process, analyze and store securely. Consequently in recent times there has been a shift towards non-relational databases or NoSQL databases especially in the fields of data analytics and large scale cloud data storage across distributed systems.

Non-relational models (NoSQL)

In addition to having to compensate for the disadvantages of the relational model data engineers have over the last decade needed to solve new sets of problems created by the size and scope of the internet. As a result, a new model for database design has emerged and these are called NoSQL databases, and the most popular NoSQL databases are document oriented.

Document databases

The beauty of NoSQL databases is that they are conceptually easy to understand and use. Hence their common use in what are termed Document databases. This type of database is designed to store complete, usually self-contained representations of entities, such as emails, video clips, social media chat, Twitter tweets, and photographs. For most non-technical users accessing and managing the data contained within Document Databases —which includes anything from social media content to medical records — is actually a very straightforward and natural way to approach storing data.

Graph databases

A graph database is not what it initially seems as it holds the mathematical context of a graph which is a model that takes the form of a web of nodes and the inter-connections (called “edges”) between them. For example, the graphic mapping of IoT wireless mesh networks and the connectivity of the internet through internet routing protocols such as BGP, which provides the route mapping of the web, have provided a number of important uses for graph databases. Additionally, these Graph databases are also finding important new environments due to the prevalence of IoT industrial projects that connect wireless devices across remote wide areas and even smart cities. This is because the graph or map they produce can be easily calculated and updated using robust algorithms and they are also easier to visualise as they are the most natural way to model the network of relationships among a collection of entities.

Key-value store

A key-value store is an associative array or dictionary which relies on a unique key to represent a body of data whether that is a video clip, a music file, some chat text or a photograph. This is because data is stored simply as binary code but the key and the metadata makes the entity unique so it can be written and retrieved to and from storage extremely quickly. Of course this is as long as you already know the key of the data file you are looking for as the process of querying by the values is typically very difficult.

Problems of multiple models in the enterprise

In an SMB, where different departments make decisions and implement applications to meet their own needs, a diversity of database models will prevail. Also, business longevity and the reluctance to close down legacy systems mean that legacy databases will often be in service years after their useful life and this is even more of an issue if companies have been acquired over time.

For most departments using different models for different applications makes perfect sense, as no single model is perfect for all situations. A system managing financial transactions needs absolute consistency, which usually means a relational database and third normal form. An ecommerce or other high volume transactional systems such as a CRM might need a document-oriented system dependent on its product or service catalogue. Even if everyone in an organization used only relational databases, the differing schemas created to solve particular business problems and application requirements would still create difficulty.

Silos and how they get connected

Problems arise with reporting in data silos due to the complexity of acquiring the individual slices of data from disparate systems and tables. For example, if the sales transaction order data is in the sales database on the ecommerce server, but the details that led up to the sale transaction, i.e. the salesman, leads, history and the customer details are in the CRM system, and the information about how that customer first connected with your company is in the marketing database, then retrieving that data through a standard SQL query might pose a significant problem to even an experienced DBA. There are a few common ways organizations try to overcome this gap.

Report building

This is one of the most common methods in the SMB as managers manually query the various data stores and loads that information into a spreadsheet every month and it is a time wasting monthly chore that is dreaded by most. The aggregated data from across the business becomes the basis for a report which is then read by the company executives, but the veracity of the report is often questionable.

Problems with this manual extraction and spreadsheet presentation approach:

The new information created by building the report cannot stay up to date. The process has to be repeated any time you want to see new information.

The copy-paste-manipulate process has a lot of opportunities for human error and bad judgment also data can be easily manipulated whether intentionally or not to hide a multitude of sins.

It is extremely time consuming and worse it is boring and frustrating which leads to errors.

Reports can only answer questions that report builders thought to ask ahead of building the report i.e. they are only barely feasible for standard business reports. New insights that might come from being able to “play with” the data or through ad hoc queries are just not feasible.

Data aggregation

A welcome advancement from the tedious manual aggregation of operational data into a report every month is to aggregating the data into a new database, typically a relational database with a schema designed to answer particular types of analysis questions i.e. a warehouse with an Online Analytical Processing (OLAP) capability. An OLAP as they are termed differs from a standard Online Transaction Processing (OLTP) database due mainly to its data schema being multidimensional as opposed to the traditional 2-dimensional relational data schema. The advantage of this approach is that IT or the database administrator can write scripts to extract, transform and load the data out of the other operational systems (CRM, POS, ERP, etc) into the warehouse, and then pre-prepared reports can be scheduled to run every day or even hour, which ensures that the data in the reports is always fresh.

Problems with this approach:

Defining a schema that will work well for data coming from a variety of sources can be very difficult.

The ETL process may rob data of its original context and format, which may affect its meaningfulness.

Like the reports, the new database will often only be able to answer the types of questions that were thought of when designing the schema.

Changes in the way data is organized in the originating databases will require changes to the aggregating database.

Multiple models, one database

A multi-model database can solve these problems by providing a single storage solution for all the different types of data an organization handles. A multi-model database supports multiple data models in their native form within a single, integrated back end, and uses data standards and query standards appropriate to each model. Queries are extended or combined to provide seamless queries across all the supported data models. Indexing, parsing, and processing standards appropriate to the data model are included in the core database product.

A multi-model database has relational storage, document storage, graph storage, and key-value storage together in one place. Data does not need to be transformed before it is loaded, and a relational schema does not need to be designed before data is brought in.

The multi-model database allows querying across data of different types, without losing the context and formatting that often gives meaning to individual pieces of information.

The major advantage to the multi-model approach is that it allows you to keep data in the form the data wants to be in, it minimizes or eliminates the need to transform data to fit new formats, and it allows you to consume data before knowing how it will be organized. As we already live in a multi-data model world it makes sense that a multi-model database system can bring that world view into a single application.

Chapter 8 -Transactional Systems (OLTP)

Having seen some of the other type of specialized data bases models available it might seem strange that the most commonly deployed by far in business is still the relational database. The advantages that RDBM databases have over tradition file systems is quite clear and they are typically; speed, efficiency, and cost. In addition, they also provide for improvements in data quality over spreadsheets it their; completeness, validity, consistency, timeliness and accuracy. However, when we consider why relational databases are still used in operational systems, when compared to other types of database. Then we find that RDBM databases can also be optimized for very fast write functions for small portions of data making them extremely efficient for logging high volume transactions where write performance is critical and read less so.

Designing the Database

Whatever the type of database the first step a developer is required to take is to design a conceptual data model that will reflect the structure of the data to be stored in the database. A tried and tested way to make the conceptual model is with entity relationship diagrams and using UML. However to construct an accurate data model an in-depth understanding of the application and its related workflows and procedures is required in order that the data model reflects the real world the application will mirror.

Therefore designing the database initially requires an in depth knowledge of the application flow, which typically will be determined from the developers Software Requirements and Specifications. In this document the application developers will explain through use case and data flow charts how the application will function and it will be based upon this document that a database designer will construct the initial schema. However, the designer still needs to determine the tables required as well as the names for instances and attributes stored as columns. Where the database is interactive and dynamic for instance when it will input data from an online form the designer will also need to consider which fields are required. For example, do you add a US State field and drop down selector in a Customers data record, and if so, do you make it a required field? If you do, then you may have unintentionally restricted your customers to the US only. Another example, of what level of detail is required in determining the fields and data assets collected is to imagine the detail that may be required when creating an instance for a product. In this case, there will be certainly a product name, a product ID, a description, a model and a price, but what about a photo, a choice of attributes, a unique serial number, an inventory level, different prices (stock, retail, discount) all these possible attributes and relationships have to be taken into account at the data conceptual modeling stage.

Once the conceptual data model has been completed, it is then used as the input for the logical data model. This is the time to translate the conceptual model into an actual database schema, which will accurately represent the model and implement the relative data structures in the database.

When designing the database schema it is important to remember there are two ways in which a database is used. One is Online Transaction Processing (OLTP) and the other is Decision Support Systems (DSS). Depending on the application, the choice is whether to normalize or denormalize the data.

OLTP (Online Transaction Processing)

An OLTP database is generally an SQL relational database manager (RDBM) such as Oracle or MySQL and it stores structured data in a relational 2-dimensional data structures called tables. Although most database systems use a form of SQL, most of them also have their own additional proprietary extensions that provide valuable product differentiation and as such are usually only available for use on their system. However, the standard SQL commands provide all the functionality required such as "Select", "Insert", "Update", "Delete", "Create", and "Drop" and these limited commands can be used to accomplish almost everything that one needs to do with a database.

Such are the popularity of SQL databases they are almost always used in transactional operational database systems for SMB, Enterprise, Commerce and Business applications such as CRM (Salesforce), ERP(SAP),

Finance (Oracle) and HR (PeopleSoft) systems. In addition to providing the back-end storage for business applications there are also embedded and PC based version in retail POS systems and in industrial controlsystems. Therefore, SQL databases make up the majority of operational databases found in the business and in operational environments, which require an OLTP database.

What makes SQL databases so suitable for OLTP is that they are highly optimized for write processing of transactional data into their 2-dimentional table structures. Visualizing a SQL table is much like looking at a spreadsheet, which consist of Column, Rows, and data fields. The complexity comes with normalization where tables grow rapidly and the interconnections between them grow exponentially creating a complex lattice of many small tables sometimes numbering into the hundreds or thousands.

Alas, before we get ahead of ourselves it is important to recognize that at the fundamental level, a relational database system contains one or more objects called tables. The data or information for the database is stored in these 2-dimensional data tables, which are uniquely identified by their name which identifies the instance, such as Customer, Product, or Sales and are comprised of columns and rows. Columns contain the attributes of the table instance, so for example the Products instance may have columns, such as Product ID, SKU, Model, Count, which are attributes, as are the data type, and any other attributes for the column. Rows contain the records for the columns, so in our example the rows of records may be Kettle, Iron, or Spoon. The intersection between the column and row will contain the data. A very simplistic example below shows a sample table for the instance called "Daily Temp".

We can see that in this table city, state, high, and low are the names of the columns. The rows contain the records for this table and the intersection, the cell, contains the measure for each column/row:

Annual Temperature			
City	State	high	low
Atlanta	Georgia	72	53
Los Angeles	California	75	56
Chicago	Illinois	59	41
San Diego	California	70	60
Houston	Texas	80	60

The designer will also have to specify which Data types and what the type of data can be used for that particular column. For example the column in the Annual Temp table called "City", is used to hold the City names, so that particular column should have a "varchar" (variable-length character) data type. Similarly, the column "High" which holds the max temperature figure should be assigned a data type of "number" with a size of (3).

Here are the most common Data types:

char(size) Fixed-length character string. Size is specified in parenthesis. Max 255 bytes.

varchar(size) Variable-length character string. Max size is specified in parenthesis.

number(size) Number value with a max number of column digits specified in parenthesis.

Date Date value

number(size,d) Number value with a maximum number of digits of "size" total, with a maximum number of "d" digits

When tables are created, it is common for one or more columns to have constraints associated with them. A constraint is basically a rule associated with a column that the data entered into that column must follow. For example, a "unique" constraint specifies that no two records (rows) can have the same value in a particular column, such as "Product ID". They must all be unique to that column. The other two most popular constraints are "not null" which specifies that data for that column can't be left blank, and "primary key". A "primary key" is a very important constraint as it defines a unique identification of each record (or row) in a table.

Primary Key (PK)

A PK is assigned to a specific column with a unique value for each row. Good design dictates every table should have one. Good prospects for the unique Primary key are unique identifiers such as Product_ID or SKU (a stock identifier). Sometimes the primary key requires two or more columns to make a unique primary key. Some other notable keys that relate to the primary key are:

Candidate key: A field that is considered a possibility for becoming the primary key. However, only one field must be chosen as the primary key. Candidate keys are entirely optional.

Alternate key: Any candidate field that was not used as the primary key.

Foreign Key (FK): A Foreign Key defines relationships between tables. When you want a row in one table to be linked to a row in another table, you place a FK column in the child table and use the value of the parent row's PK as the value of the FK field. The links between the foreign keys and the Primary keys is what establishes the relations in the database schema and allows normalisation.

Composite Key

This key is made up of more than one column. Typically, this is used when you want to prevent a table from using the same combination of values twice.

When discussing SQL keys it is a good starting point to answer the question, 'what is a key in SQL?'

According to the SQL standard; 'a key is a subset of columns in a table that allow a row to be uniquely identified'. In other words, a column or group of columns that hold a value or set of values that is unique to each row, within the boundaries of that table. Therefore, a key can be one or more columns that identify any single row in a table.

However, if a key contains more than the minimum number of keys required to identify a single row then SQL standards designates that key to be a super key. On the other hand, if a key comprises the minimum number of columns required to identify a single row within the table then it is a minimal super-key.

Consider the table below.

Title	First Name	Family Name	D.O.B	email	Location
Manager	Rachel	Braun	18:03:1988	RB@rgconsulting.com	London
Manager	John	Blackley	28:07:1987	jb@rgconsulting.com	New York
Manager	Simon	Garfunkle	12:12:1988	sg@rgconsulting.com	Rome
Manager	Morris	Whitely	31:08:1980	mw@rgconsulting.com	Paris
Manager	Roberta	Redford	15:06:1985	rr@rgconsulting.com	Milan

In SQL, a super-key is a set of columns whose combined values differentiate a single row from any other row, i.e. the combination will be unique for every row in the table.

Possible super-keys in the table managers are;

Family Name,

Family Name, D.O.B

Family Name, D.O.B, email

Family Name, D.O.B, email, Location

In this small table, it is easy to find unique combinations; however, in a table of millions of records it will be far more difficult to find a unique combination of columns that match just one row.

The possible choices of minimal key, which have the minimum number of columns that uniquely identify a row, are: Family Name, D.O.B, email& Location. The minimum key 'email' is the only key likely to scale if the table grows.

What is a Natural Key?

A natural key is comprised of columns that have a logical relationship with one another within the table. In the example table, Managers, all the columns, Family Name, D.O.B, email& Location have a logical relationship. Therefore, any key composing of any of those column(s) would be a natural key as the columns already existed, the DB designer did not need to create a column to provide a unique characteristic, as in the case of a surrogate key.

A surrogate key is an artificial key whereby the column value has no relationship with the other columns in a table. A surrogate key is usually just a custom column that the designer has added and holds as a value a simple sequential number. The column's row value increments by 1 as a new row are added. By providing each row in the table with a unique number as an identifier, the designer has created an artificial key for the table. Now every row in the table has a unique identifier regardless of changes in policy, coincidence or growth.

Consider the Table Managers after adding a surrogate key.

Company ID	Title	First Name	Family Name	D.O.B	email	Location
101	Manager	Rachel	Braun	18:03:1988	RB@rgconsulting.com	London
102	Manager	John	Blackley	28:07:1987	jb@rgconsulting.com	New York
103	Manager	Simon	Garfinkle	12:12:1988	sg@rgconsulting.com	Rome
104	Manager	Morris	Whitely	31:08:1980	mw@rgconsulting.com	Paris
105	Manager	Roberta	Redford	15:06:1985	rr@rgconsulting.com	Milan

By adding the surrogate key Company ID, the designer has ensured that the table has a unique minimal key, Company ID, regardless of change in email policy or new hires sharing the same family name or D.O.B, the key will stay unique. By creating a surrogate key the designer now has a perfect candidate as the Primary key for the table, with the other candidate key 'email' now becoming a secondary key.

What are the specific types of key?

IN SQL there are unique keys, contender keys, foreign keys and the most important of all, the primary key. The one thing they all have in common is that they share the same basic principle in so much as they can be constructs of one or more columns. One area where they do differ is in their purpose, especially the foreign key.

The foreign key differs from the other key types, as its relevance is out-with the local table. A foreign key is really a pointer to a unique key in another table. A foreign key within a table (A) references a unique key in another table (B), which defines a relationship that links the tables.

A unique key is simply that, a unique key within the table, there can be several unique keys in a table.

There are some basic rules regards foreign keys & unique keys;

A table can have multiple unique and foreign keys

A foreign key must reference a unique key in another table

A foreign and a unique key can be NULL

A primary key cannot be NULL

Each table should have a primary key

Each table can have only one primary key

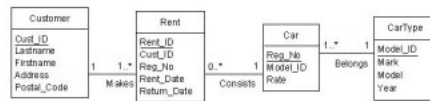
It is very important to select a strong primary key, such as in the example above that will remain unique even during times of change. A primary key uniquely identifies every record in the database, and needs to be unique. The DB designer selects a primary key from an existing strong stable natural key or more commonly from a custom surrogate key. There can be several candidates for the role of Primary key, but there can only be one Primary chosen, therefore it should be chosen wisely.

Database Design State

Before you can define the database tables using a specific database management software and DDL (Data Definition Language), you must write a relation schema for each table. The set of relation schemas form the relational database schema which is used as input when defining the database.

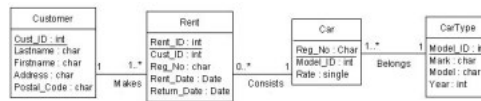
Car Rental Database

Here is a class diagram of a Car Rental database below. This diagram has been drawn using UML classes and associations. The name of the entity is in the upper box. The attributes for the entity are written in the lower box. The primary key candidates are underlined.



The Class Diagram of the Car Rental database.

A customer has made one or more (1..*) car rentals. A certain car rent always belongs to a certain customer (1) and there is always a specific car (1) that is rented. A car is rented out several times. If the car is a new one, it might not be rented out yet (0..*). A car has a certain car type (1). There is at least one car in each car type (1..*). In the class diagram data types can be written. It might be a good idea to think about data types at this point. You have to define the data type for each attribute before you can implement the database.



The relation schemas for the Car Rental database are:

```
CUSTOMER(Cust_ID, Lastname, Firstname, Address, Postal_Code)
RENT(Rent_ID, Cust_ID, Reg_No, Rent_Date, Return_Date)
CAR(Reg_No, Model_ID, Rate)
CARTYPE(Model_ID, Mark, Model, Year)
```

The relational process uses a methodical approach known as normalization. The goal of normalization is to ensure that each elementary "fact" is recorded in only one place, so that insertions, updates, and deletions automatically maintain consistency. Redundant data wastes disk space and creates maintenance problems. Relational databases improve performance but also data integrity and consistency. If data that exists in more than one place must be changed, the data must be changed in exactly the same way in all locations. There are a few rules for data normalization and each rule is called a normal form but first

Unnormalized - when a database has not yet been broken apart by business entities* into separate tables (Normalized).

Normalized - when a database has been broken into several tables so column values don't repeat (see description below)

Denormalized - when a database was first Normalized, then, for strategic reasons, carefully modified to violate Normalization rules to increase reporting speed.

* A business entity is a unique identity with both state and behavior. An example might be Customer and Order.

Rules for Normalization

When normalizing a schema there are several levels or standards of normalization we can apply and these are called Normal Forms:

First Normal Form

Eliminate repeating groups in individual tables.

Create a separate table for each set of related data.

Identify each set of related data with a primary key.

Second Normal Form

Create separate tables for sets of values that apply to multiple records.

Relate these tables with a foreign key.

Third Normal Form

Eliminate fields that do not depend on the key.

Chapter 9 - Operational Databases

The databases we have described at a very high level so far are relational databases, which we would commonly see utilized in standard transaction based OLTP data bases. This model of databases are designed for inserting data efficiently by writing data one record at a time albeit extremely quickly. As their name suggests OLTP databases are online and constantly receiving transaction data such as sales records, log files or sensor status updates. However, as we know from everyday experience a database does not just store data it provides views and reports of the input data it holds in the tables. What is more the database can generate its own data through arithmetic techniques such as the SUM and AVG functions. Indeed in SQL databases this set of arithmetic functions termed the Aggregation Functions can generate new data about the stored data in the internal schema tables. By performing these aggregate functions on data gives SQL very powerful reporting and data aggregation and manipulation techniques. Therefore in most SMB reporting is done on the OLTP data sources such as the Financial, ERP or CRM databases directly. But in larger organizations or where there is large data sets present, or heavy transactional use it is no longer either feasible or advisable to run SQL Queries on the OLTP data base as it is optimized to write transaction data and not to read and process vast amounts of data as output. Running SQL queries can have a serious negative impact on the OLTP performance and so another solution for reporting is required.

Traditionally, the solution to the burden of reporting and analytics was to pre-process reports and schedule them to run out-of working hours and that is still feasible today. However for many systems there is no quiet periods as they operate 24/7 an example would be an ecommerce website. In this case the solution is to host another server that cloned the transactional database via either a cluster or by data replication on to another server. Using either of these options allowed reports to be run on a slave data base thereby mitigating the detrimental effects of overloading the resources of the main transactional database.

Utilizing clusters or through data replication techniques provides another benefit in that it facilitates a live backup of the data. This also mitigates the risk in the case of a primary database crash. Without a clone or online data replication many of the transaction replays would be required to restore a single transactional data base server and consolidate its database. (The issue being if you run an incremental back-up even once an hour, there is a possibility of the backup being up to one hour out of date and the Data Base Administrator would need to manually replay all the transactions missed but logged during that period).

Having dual servers in a cluster or in an online replication configuration is feasible and can even be used on different transactional servers. For example, a slave CRM reporting server can be the slave ERP Reporting server and vice versa and this reduces both the capital and operational overheads. All that is needed there is for the designated slave reporting server to run an instance of the RDBM for each application and hold a copy of the schema for each database. This may be sneered at as just a poor man's data warehouse, but it is perfectly feasible for an SMB with limited resources and low transaction rates.

There are some issues with the shared reporting server model as although it seems a good solution for the SMB with light workloads across several applications, issues do arise. Online data replication works really well and under light load is both quick and reliable. However the initial issue was that running intensive reports or data analysis on an operational application server may have been detrimental to the application's performance and that was quite true. The issue now was how would running those same reports, but this time for all the applications on just one server, affect the operation of the online replication.

The only other option is to double up each transactional operational server with a slave reporting server and that is not financially feasible for a lot of SMBs, so that wasn't ideal either.

There had to be another solution which would lower both the Capex and Opex overheads and provide a reporting and analysis platform that could deal with high transaction workloads without the risk of overloading the

transactional servers. Again depending on the workloads two or three instances could be run without any issues but conflict may arise due to the ambiguous nature of the server – is it a reporting server or a backup server?

What was needed to solve these issues was a database architecture that could alleviate some of the inefficiencies of read operation performed on a write optimized platform. The solution was to think out of the box and pivot the tables, so that rows became columns.

Column-Oriented Databases

Increasingly businesses are realizing a one size fits all model isn't working for databases. It seems we can have issues with reporting and analysis performance even on a dedicated reporting server albeit one performing replication and reporting services for several disparate applications. The issue appears to be down simply to the way RDBM servers operate. Interestingly, it didn't need to be this way, by simply pivoting the tables tremendous improvements in performance could be leveraged. For example, when you want to analyse large amounts of data, with analytical queries that span 1000's of rows, column-oriented databases can provide a 100x speedup. Particularly for time-series data, which is transactional data, column-oriented databases have been successfully used to deliver very high performance fast storage and analysis.

What is a column-oriented database?

Column Oriented Database

date	price	size
2011-01-20	10.1	10
2011-01-21	10.3	20
2011-01-22	10.5	40
2011-01-23	10.4	5
2011-01-24	11.2	55
2011-01-25	11.4	66
...	...	...
2013-03-31	17.3	100

A column-oriented database stores each column continuously i.e. on disk or in-memory each column on the left will be stored in sequential blocks.

For analytical queries that perform aggregate operations over a small number of columns retrieving data in this format is extremely fast. This is because of the way that PC computer BIOS and hard disk drive storage is optimized for block access. Simplistically, designers when they were deliberating the read/write operations of hard-disk access on PCs had some performance issues due to the issues of precision. So they decided that once a disk-drive's head was roughly positioned over the track it was to read data from it was easier and quicker to just Hoover up all the data as a large block and sort out what was required in memory. This was both simple and brilliant. Therefore, if we take advantage of this and the fact that column data is very often stored beside each other on the hard drive we can exploit the physics of what is termed the locality of reference. On hard disk drives this is particularly important due to their relatively sluggish performance characteristics, due to the mechanical constraints of physically moving the read/write head assembly. Hence, column based databases running on hard disk drives provide optimal performance due to the sequential disk access limiting head movement.

Row Oriented Database	Table of Data	Column Oriented Database																																																																								
<table> <tr><th>date</th><th>price</th><th>size</th></tr> <tr><td>2011-01-20</td><td>10.1</td><td>35</td></tr> <tr><td>2011-01-21</td><td>10.3</td><td>35</td></tr> <tr><td>2011-01-22</td><td>10.5</td><td>40</td></tr> <tr><td>2011-01-23</td><td>10.4</td><td>5</td></tr> <tr><td>2011-01-24</td><td>11.2</td><td>55</td></tr> <tr><td>2011-01-25</td><td>11.4</td><td>60</td></tr> <tr><td>2011-01-31</td><td>17.3</td><td>100</td></tr> </table>	date	price	size	2011-01-20	10.1	35	2011-01-21	10.3	35	2011-01-22	10.5	40	2011-01-23	10.4	5	2011-01-24	11.2	55	2011-01-25	11.4	60	2011-01-31	17.3	100	<table> <tr><th>date</th><th>price</th><th>size</th></tr> <tr><td>2011-01-20</td><td>10.1</td><td>35</td></tr> <tr><td>2011-01-21</td><td>10.3</td><td>35</td></tr> <tr><td>2011-01-22</td><td>10.5</td><td>40</td></tr> <tr><td>2011-01-23</td><td>10.4</td><td>5</td></tr> <tr><td>2011-01-24</td><td>11.2</td><td>55</td></tr> <tr><td>2011-01-25</td><td>11.4</td><td>60</td></tr> <tr><td>2011-01-31</td><td>17.3</td><td>100</td></tr> </table>	date	price	size	2011-01-20	10.1	35	2011-01-21	10.3	35	2011-01-22	10.5	40	2011-01-23	10.4	5	2011-01-24	11.2	55	2011-01-25	11.4	60	2011-01-31	17.3	100	<table> <tr><th>date</th><th>price</th><th>size</th></tr> <tr><td>2011-01-20</td><td>10.1</td><td>35</td></tr> <tr><td>2011-01-21</td><td>10.3</td><td>35</td></tr> <tr><td>2011-01-22</td><td>10.5</td><td>40</td></tr> <tr><td>2011-01-23</td><td>10.4</td><td>5</td></tr> <tr><td>2011-01-24</td><td>11.2</td><td>55</td></tr> <tr><td>2011-01-25</td><td>11.4</td><td>60</td></tr> <tr><td>2011-01-31</td><td>17.3</td><td>100</td></tr> </table>	date	price	size	2011-01-20	10.1	35	2011-01-21	10.3	35	2011-01-22	10.5	40	2011-01-23	10.4	5	2011-01-24	11.2	55	2011-01-25	11.4	60	2011-01-31	17.3	100
date	price	size																																																																								
2011-01-20	10.1	35																																																																								
2011-01-21	10.3	35																																																																								
2011-01-22	10.5	40																																																																								
2011-01-23	10.4	5																																																																								
2011-01-24	11.2	55																																																																								
2011-01-25	11.4	60																																																																								
2011-01-31	17.3	100																																																																								
date	price	size																																																																								
2011-01-20	10.1	35																																																																								
2011-01-21	10.3	35																																																																								
2011-01-22	10.5	40																																																																								
2011-01-23	10.4	5																																																																								
2011-01-24	11.2	55																																																																								
2011-01-25	11.4	60																																																																								
2011-01-31	17.3	100																																																																								
date	price	size																																																																								
2011-01-20	10.1	35																																																																								
2011-01-21	10.3	35																																																																								
2011-01-22	10.5	40																																																																								
2011-01-23	10.4	5																																																																								
2011-01-24	11.2	55																																																																								
2011-01-25	11.4	60																																																																								
2011-01-31	17.3	100																																																																								

To better illustrate the benefit of columns versus rows let us consider the image depicting the Row Orientated Database. The areas that need to be read when performing a query like "average price" for all dates in row-oriented databases would require that large areas of the disk be accessed as we will have to read every single row in order to extract just one field from each – the price and discard all the other data we collect via a read operation. This is very inefficient and consequently it is very slow in large databases with perhaps 10s of thousands of rows.

However, in a column-oriented database the prices are stored as one column and in a sequential region therefore the drive read operation needs only collect data from a single small region. As a result we discover that when we use column-oriented databases for aggregate functions (sum, average, min, max, etc.) within queries they are extremely fast and this is tremendous for reporting and analysis, which rely heavily on these aggregate functions.

This of course may well beg the question, Why are most databases row-oriented? The answer to that question is due to ease of editing and scaling the schema. If we want to add one row somewhere in the middle of our data for 2011-02-26, on the row oriented database this is not an issue. With a column oriented data base on the other hand we will have to move almost all the data. Fortunately, since we mostly deal with time series new data this requires only appends to the end of the table so is not an issue.

Column-Oriented Vs. Row Oriented Model

Operation	Column-Oriented	Row-Oriented
Aggregate Calculation of Single Column e.g. 'avg' (price)	Fast	Slow
Compression	Higher. As stores similar data together-	
Retrieval of a few columns from a table with many columns	Faster	Slower
Insertion/Updating of single new record	Slow	Fast
Retrieval of a single record	Slow	Fast

As can be seen from the table Column-based databases are fast and agile when reading data from the database because much of the similar data is grouped together and can therefore be swept up in one disk read operation. Where column-based databases excel is exactly where reporting and analysis functions place the most stress on a database making them a great fit for these use cases.

In-memory database solutions

In-memory databases put the working set of data into system memory, either completely, in the case of solutions such as SAP Hana, or partially, based on the identification of tables that will benefit most from DRAM speed.

There is an obvious performance benefit in the reduced latency which in-memory database solutions bring, even over heavily cached systems as these can only optimise database read requests.

The performance benefits that in-memory databases bring are more subtle than that just read/write performance uplifts. This is because when all the data is kept in memory, the issues that naturally arise from the use of traditional spinning disks disappear. For example, in-memory databases provide an opportunity to optimise the way data is managed compared to traditional databases on disk-based media.

What this means in terms of performance is that, for example, there is no need to maintain additional cache copies of data and manage the synchronisation between them. In addition, the data can also be compressed and decompressed in memory more easily, resulting in the opportunity to deliver both high security performance and space savings over the equivalent disk copy.

Sometimes, we see an attempt to compromise and a database administrator may create a RAM disk in memory and move the database to this virtual volume in a bid to get many of the benefits of in-memory databases but without the cost overhead.

This can be done, but the performance benefits do not really manifest themselves due to the internal algorithms of the database are still managing the data as if it were on disk –the virtual disk drive is transparent to the algorithms as they should be. Therefore the underpinning database management algorithms still perform tasks such as pre-fetching, caching and lazy writes and that doesn't result in any optimal gains in terms of performance and sometimes they may even use more processor time. Instead, in-memory database solutions have logic specifically adapted to work with data in DRAM.

The biggest problem with in-memory databases is that system memory is volatile, which means in-memory databases only conform to three out of four of the ACID model of database characteristics - atomic, consistent, isolated and durable. Of these, durability cannot directly be served by in-memory database solutions, because data is lost when power is removed from the server.

Overcoming the shortcomings of volatile memory

But there are solutions to the problem. These include keeping additional copies of data in clustered and scale-out databases that allow systems to keep running by replicating updates to one or more standby systems.

Some database systems also perform periodic commits-to-disk to maintain state to a point from which recovery can be performed in the case of a server crash. Here there is a trade-off between the time between commits (and subsequent recovery) and the overhead of the commit process on performance.

In-memory database technology has largely been avoided for general OLTP applications due to the lack of durability and instead targeted at specific data types or analytics requirements (including batch analysis and reporting) where re-running transactions can easily be achieved.

Having said that, in-memory databases are set to move into the OLTP world as the acceptance and adoption of the technology continues. Indeed a major foothold in the market has been already gained with the release of Microsoft SQL Server 2014, which offered out-of-the-box in-memory capability. SQL Server makes use of "memory optimised tables", which allow portions of a database to be placed into system memory thereby accelerating the performance of analytical calculations.

Storage for in-memory databases

Although the operation of in-memory databases occurs in system memory, there is a need for permanent storage media. There are three main in-memory database storage requirements: first, to provide permanent media to store committed transactions, thereby maintaining durability and secondly, to provide a mechanism for recovery purposes if a database does need to be reloaded into memory, and; thirdly to provide permanent storage to hold a copy or backup of the database in its entirety.

With regard to the requirement to reload databases back into memory, then of all the latest disk drive technology flash is possibly the most beneficial. The issue may come down to cost, and with flash being significantly more expensive than hard disks per gigabyte and, in the case of in-memory database use, the efficiency and necessity of such performance will come into question as the data will be accessed very infrequently. But there is no doubt that reading an entire database into memory from flash will always be much faster than from spinning disk and it is still cheaper than Solid State Drives. Therefore you might want to undertake a cost/benefit risk audit to decide on that all important price/performance balance. However, there are ways around the storage issues such as using clustered or replicated environments, but that will still require significant investment so a shared flash-based storage solution may still be best.

Chapter 10 – Alternative OLAP Data Schemas

As we have examined in the previous chapters, OLTP database systems typically are designed around the RDBM model which can be readily optimised for efficient transaction handling. We also learned that these types of database were not ideal for reporting and analysis and often the solution was to offload the burden onto another reporting server which held a clone of the operational data. This slave reporting system architecture works well on a one to one basis and is the foundation of the concept of the Data Warehouse.

A data warehouse (DWH) is a system used as a central repository in which to collect data from one or more disparate operational systems in order to collate, transform and store information for use in data analysis and reporting. Data marts are specialized areas of a data warehouses and they can be internal or external to the physical warehouse and they are used to store information needed by a single department or even by an individual power user.

Why are data marts needed? All relevant data collected from several disparate operational systems, such as CRM, ERP and HR is stored inside the company DWH. Most users, however, only need to access certain subsets of data, like those relating to sales, production, logistics or marketing. Data marts are subdivisions of this large pool of data and only contain specific information for a particular department, project or group. The architecture is deemed important from both a security standpoint as we can enforce strict access control and from a performance aspect as the amount of data handled during queries is significantly reduced.

There are two different approaches to the data warehouse-data mart relationship:

Top-Down: Data marts are created from the data warehouse. (This is something that Bill Inmon, would agree with, along with the idea that warehouses should be in 3NF.)

Bottom-Up: Data marts are created first, and then combined into a data warehouse. (This approach is closer to what Ralph Kimball, a data warehouse and dimensional modeling expert, advocates.)

Regardless of the warehouse/mart model deployed there still is the non-trivial problem of moving data from heterogeneous operational systems and physically storing it within a common schema within the central warehouse. In order to accomplish this task requires that data be extracted from each operational systems database and then transformed in order to fit the new schema of the central repository, as this is likely to require table names, columns and data types to be translated or changed to suit before it can be successfully loaded. This ETL process is used to add “new” data to the warehouse system on a regular basis. ETL is an acronym for Extract, Transform and Load and that describes perfectly its functional purpose. As the name hints, we’ll extract data from one or more operational databases, transform it to fit our warehouse structure, and load the data into the DWH.

Remember when we spoke about the idea of having the schema fit the data rather than have our data fit the schema. Well this is where we get the chance as when creating the warehouse we need to consider whether to leave the schema in a 3NF (normalised relational format) schema (lots and lots of tables) or to change the schema to a much simpler structure called a Star schema which is not only simpler, but also not normalised. The downside is it could possibly have some data redundancy and consistency issues. However if we consider that the performance of the technologies available to us today such as very high capacity high performance disk drives, Solid State Drives and In-memory optimised Databases then we might be willing to accept that risk.

Therefore, if we consider an alternative schema to get away from the constraints and complexity of the 3NF Relational schema there are several options available to us. Designing the warehouse data schema is described as dimensioning the warehouse or dimension modelling:

Dimensional modelling, which is a fundamental and significant part of data warehouse design, results in the creation of the data schema. It is described as being dimensional modelling because there are two types of tables

involved:

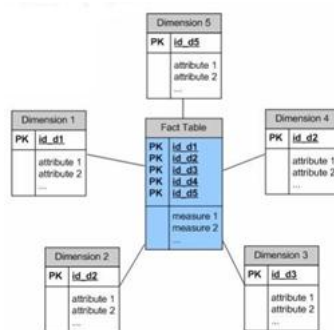
Dimension tables are used to describe the data we want to store. For example: a retailer might want to store the date, store, and employee involved in a specific purchase. Each dimension table is its own category (date, employee, and store) and can have one or more attributes. For each store, we can save its location at the city, region, state and country level. For each date, we can store the year, month, day of the month, day of the week, etc. This is related to the hierarchy of attributes in the dimension table.

In the star schema, we'll usually find that some attributes are a subset of other attributes in the same record. This redundancy is deliberate and done in the name of better performance. We could use date, location, and sales agent dimensions to aggregate (the transform part of the ETL process) and store data inside DWH. In dimensional modelling, it's very important to define the right dimensions and choose proper granulation.

Fact tables contain the data we want to include in reports, aggregated based on values within the related dimension tables. A fact table has only columns that store values and foreign keys referencing the dimension tables. Combining all the foreign keys forms the primary key of the fact table. For instance, a fact table could store a number of contacts and the number of sales resulting from these contacts.

With this info in place, we can now dig into the star schema data model.

The Star Schema



What is star schema? The star schema architecture is the simplest data warehouse schema. It is called a star schema because the diagram resembles a star, with points radiating from a centre table, which is called the Fact table and the tables at the points of the star are the dimension tables. Usually the fact tables in a star schema are in third normal form (3NF) whereas dimensional tables are de-normalized. The fact that the star schema is the simplest architecture is probably why it is most commonly used nowadays and is recommended by most system integrators and vendors. Bearing that in mind let's take a look at how it is formed.

Fact Tables

A fact table typically has two types of columns: foreign keys to dimension tables and measures, which are data that contain numeric facts. A fact table can contain the fact's data on a detailed or aggregated level.

Dimension Tables

The definition of a dimension is that it is a structure usually composed of one or more hierarchies that categorizes data. If a dimension hasn't got a hierarchy and levels it is called a flat dimension or a list. The primary keys of each of the dimension tables point towards their corresponding foreign key in the fact table, which are part of the composite primary key of the fact table. Dimensional attributes help to describe the dimensional value. They are normally descriptive, textual values. Dimension tables are generally much smaller in size than fact tables, which can grow very large.

Typically, the fact tables store transactional data such as the facts and figures about individual transaction for example a retail sale while dimension tables contain data about attributes and objects that relate and provide context to the facts such as a store name, region, clients, products, times, channels.

The main characteristics of star schema:

Simple structure -> easy to understand schema

Great query effectiveness -> small number of tables to join

Relatively long time of loading data into dimension tables -> de-normalization, redundancy data caused that size of the table could be large.

The most commonly used in the data warehouse implementations -> widely supported by a large number of business intelligence tools

The best feature of the star schema is its basic form and it is the simplest model used in DWH and hence the easiest to understand and design. For this reason we come across Star schemas very often in data marts.

Star Schema Example: Sales

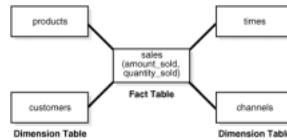
The sales report in business is one of today's most common reports. As we mentioned before, in most cases we could generate sales reports from the live system. But when data or business size makes this too cumbersome, we'll have to build a data warehouse or a data mart to streamline the process. After designing our star schema, an ETL process will get the data from operational database(s), transform the data into the proper format for the DWH, and load the data into the warehouse.

Star Schema

Each dimension in a star schema is represented with only one-dimension table.

This dimension table contains the set of attributes.

The following diagram shows the sales data of a company with respect to the four dimensions, namely product, time, customer, and sales channel.



There is a fact table at the centre. It contains the foreign keys to each of the four dimensions.

The fact table also contains the attributes, namely dollars sold and units sold.

Each dimension has only one dimension table and each table holds a set of attributes. For example, the Channel dimension table may contain the attribute set {Channel_id, name, street, city, region, country}. This constraint may cause data redundancy. For example, "City" and "Region" will contain many instances of the same data.

Advantages and Disadvantages to the Star Schema

There are plenty of advantages to using the star schema. The fact table is related to each dimension table by exactly one relation, and we don't need any additional dictionaries to describe dimension tables. That simplifies queries and decreases query execution time. We could produce the same report directly from our OLTP system, but the query would be much more complex and it could impinge on the overall performance of the system. The following sample query for the sales model will return the quantity of all phone-type products type sold in Berlin stores in 2016:

```
SELECT dim_store.store_address, SUM(fact_sales.quantity) ASquantity_sold FROM fact_sales INNERJOINdim_f
```

The biggest disadvantage of the star schema is redundancy. Each dimension is stored in a separate dimension table, and this causes denormalization. In our example, city belongs to a region or state, which belongs to a country; we don't store that relation as a rule in our database, but we continually repeat it. This means we'll spend more disk space and have a data integrity risk.

The Galaxy Schema

We can combine star schemas, or typically data marts together where each of them consists of only one fact table and a few dimensional tables. If we wanted, we could combine two or more data marts into one model. This type of schema, containing several fact tables and sharing some dimension tables, is called a galaxy schema. The big advantage is that by combining several star schemas we can then share common dimension tables and this can considerably reduce database size, especially where shared dimensions have many possible values. This is considerably easier to accomplish, if both data marts have the common dimensions defined in the same format i.e. they share all of the common attributes. If that's not the case, we'll have to adjust the dimensions to fit both needs.

The issue with the Galaxy schema and why it isn't use extensively is that there is often little requirement as although some space will be saved this way, this is only realised if there is justification for hosting the two or more star schemas (marts typically in practice) together. Therefore, before we join two data marts into one galaxy schema:

Is there any business logic or user defined requirement for hosting them together? For example, will both marts data be used by the same department, and from a security perspective should it?

Do we need the same dimension and granulation for the schemas and data marts?

The star schema is one approach to organizing a data warehouse and is the most common due primarily to its simplicity. The drawn back as we have seen is inefficiency in data storage and write consistency but If we don't have to worry about disk space and we take good care of data integrity, then the star schema is a viable first and best choice. If we do need to save disk space and simplify cross department (mart) queries we can use a Galaxy approach but that can raise access issues with security. If we need to save on disk space yet retain access control and insist on high data integrity, we should think of another approach and the common alternative dimensional model is the Snowflake schema.

The Snowflake Schema

The snowflake schema's name comes from the fact that the schema starts like a Star schema but the dimension tables may branch out and have several layers of child dimensions and this can look something like a snowflake. The fundamental idea behind the snowflake schema is that dimension tables are completely normalized and this is radically different to the design concepts of the Star schema where dimension tables are denormalised. Hence, each dimension table can be described by one or more lookup tables. Each lookup table can be described by one or more additional lookup tables. This is repeated until the model is fully normalized. This process of normalizing the original star schema dimension tables is called snowflaking.

Snowflake Schema

Some dimension tables in the Snowflake schema are normalized.

The normalization splits up the data into additional tables.

Unlike Star schema, the dimensions table in a snowflake schema are normalized.

Advantages and Disadvantages of the Snowflake Schema

There are two main advantages to the snowflake schema:

Better data quality (data is more structured, so data integrity problems are reduced)

Less disk space is used than in a denormalized model like the Star Schema or its other variants.

However, there is the major issue that we have simply gone back to the original normalized schemas and all its inherent complexity. Hence, the obvious disadvantage of using the snowflake model is that it requires more complex queries through all those additional joins. These more complex queries make the feasibility of self-service reporting and analysis of data far more unlikely due to the increased number of joins, which could also decrease performance significantly. Therefore, a compromise approach is to design a hybrid model using the best features of the Star and Snowflake, and this is the Starflake schema.

The Starflake Schema

A Starflake schema is a hybrid schema which combines features of the snowflake and the star schemas. Basically it is a reduced snowflake schema that has some of the dimension tables denormalized. When used correctly, the Starflake schema can give a best-of-both-worlds approach. Since this schema aims to reduce the number of normalised dimensions and consequently the number of query joins needed, it will reduce some of the complexity and may even improve performance.

The snowflake schema is often used in data modeling. It may be the right choice in situations where disk space is more important than performance. If we want a balance between space-saving and performance, we can use the Starflake schema. Still, the right fit for any specific problem depends on many parameters. This is one of the areas in IT where we can 'play' with factors to come up with the best solution.

Cubes and Multidimensional models

Database systems can be roughly divided into two categories: operational and reporting systems. Operational systems are often called Online Transaction Processing (OLTP) systems. In contrast, reporting and analytical systems are referred to as Online Analytical Processing (OLAP). In almost all cases the OLTP systems support business processes as they are optimised for high volume transactions. They are designed to work with "live" operational data, are highly normalized, and manage high volumes and intensity of small write style transactions and in addition they can respond very quickly to user actions. On the other hand, the primary purpose of the OLAP systems is reporting and to a lesser degree analytics. However, OLAP systems are required to perform predominantly read type processes and these are slow on an OLTP system. Consequently, OLAP systems are used to upload or clone and then summarize data, which is usually placed in a denormalized data warehousing structure like the star schema. As we have seen previously denormalization is having redundant data records within tables for the sake of better performance but at the cost of storage and write consistency.

An OLAP cube is a data structure that overcomes the limitations of relational databases by providing rapid analysis of data. Cubes can display and sum large amounts of data while also providing users with searchable access to any data points. This way, the data can be rolled up, sliced, and diced as needed to handle the widest variety of questions that are relevant to a user's area of interest.

As a result an OLAP system often uses a different database model for dimensioning, called a multi-dimension model and these data structures are often represented as cubes. The difference is that the traditional 2-Dimensioned table is replaced by a multi-dimensional cube.

The first question that may arise is what is a multi-dimensional cube? The simplest high-level answer is that a cube is a multidimensional model, and these are constructed using data according to user preferences. This means that multiple tables of interest can be held within the same multi-dimension structure but they are still defined by

dimensions and facts, which makes pre-processing easy and queries very efficient. For example, in disciplines such as finance and accounting where reports and analysis tend to follow a classic structure i.e. the information required rarely deviate out with the common range of financial standard reports. This makes it feasible to think ahead and if the cubes are preconfigured by collating all the relevant data tables together it is then relatively easy to pre-process the queries allowing for almost instantaneous retrieval of reports. As a result, multidimensional cubes are optimal where user query reports are standard and predictable as they can be pre-processed. Consequently cubes and multidimensional databases are most commonly used in data warehouses and OLAP systems optimised for reporting and analysis.

To understand cubes, how they operate and how we can configure or query them we need to understand how they came about. Rather strangely they came into being as a result of the need for simplicity. Indeed the issue that cubes resolve - or at least strive to - is that the requirement for technical competency in order to make even a simple database query is not ideal. The relational model for all its worth is rather a poor solution and for the majority of all users within an organization it is a drawback, so the goal should be towards ease of use. The multi-dimensional model, the cube, is important because it enforces simplicity. And, it is that simplicity which allows users to understand the database and for the database system to efficiently navigate and handle the data it contains.

The problem has been that traditionally in relational database analysis there is one basic storage structure, a flat table. This object, the table, takes on many different roles depending on how it relates, or joins, to other tables. Even in a much simplified structure such as a star schema, where fact tables and dimension tables describe the relationship between the various tables within the schema, it can still be confusing.

In multi-dimensional analysis the same basic structures also exist and the fact table is usually the main driver within the query. Indeed, the fact table is referred to as a cube, and the columns within the table are referred to as measures. Each cube has additional structures over and beyond a simple table. The cube has edges, which are referred to as dimensions. Each dimension is a grouping of common or related columns from one or more tables into a single entity. For example a Product dimension could be based on a single or collection of tables containing columns that refer to categories, sub-categories, and products.

All these structures (cubes, dimensions, hierarchies, levels and measures) interact with each other to provide an extremely powerful reporting environment. Each object adds new levels of interactivity when it is fully exploited. (We will discuss cubes in detail later).

Users want to interact with their data using business terms and objects without having to comprehend the underlying storage model. The abstraction of the physical storage model from the logical presentation model is the key to the success of multi-dimensional models.

The benefits on multi-dimensioning are that cubes enable interactive analysis of large quantities of data by rapidly processing the data within the database and providing “just in time” information for effective decision making.

Software vendors or information technology (IT) developers with a working knowledge of OLAP cubes can create management packs to define their own extensible and customizable OLAP cubes that are built on the data warehouse infrastructure that was provided in Service Manager 2010. These cubes are stored in SQL Server Analysis Services (SSAS). Self-service business intelligence tools such as Excel and SQL Server Reporting Services (SSRS) can target these cubes in SSAS, and you can use them to analyze the data from multiple perspectives.

As we have seen over the course of the chapter there are several alternative schemas we can use for OLAP type reporting and analysis database design. What has become clear though is that for even the SMB to collect, process and report on operational data when it is sources from heterogeneous sources is going to be a serious issue without some central repository for reporting and analysis.

A common example in an SMB scenario for the need for a solution to the vagaries of heterogeneous reporting is in retail where a business may have many remote stores scattered around the country or even the world and Point of Sales data for sales transactions and stock inventory control must be managed in a timely manner. Traditionally this may have been managed by spreadsheets and end of day reports filed with the head office for a central manager to process. But that is an onerous task and worse has to be replicated diligently each evening by a sales

manager (daily sales reports, per store, region and total), a logistics manager (stock control, replenishments), Finance (cash control, financial reports), HR (employee attendance, overtime, sick/holidays absence) something we should be able to automate.

At the end of the month the problems are even greater because prior bitter experience has taught them that the spreadsheets value is only for that day or two and when it comes to the month end the spreadsheet are always inaccurate when compared to the reports generated from the CRM, ERP, HR or Finance servers so they have to source and collate the data once more, but this time for the monthly data and reconcile each transaction for the month in order to trace the discrepancies.

Month after month figures between spreadsheets, POS and Application servers would refuse to tally and to compound matters no two departments could ever agree, with Sales and Logistics constantly at loggerheads over disputed transactions. The only time all the department heads would unite in common agreement was when they blamed the data on the systems!

Therefore, it falls back to IT to address these issues of incompatible data collected from diverse systems and load them into a common schema which can be accessed and queried more easily and on a self-service basis.

Chapter 11 - Designing a Data Warehouse

Now that we have had a brief high-level overview of the major architectural design patterns and concepts regarding data warehouses and data marts we can move on to consider some of them in greater detail. As we have seen running extensive SQL queries against an operational database such as an On Line Transactional Processing (OLTP) ecommerce store database can prove detrimental to the systems performance causing it in extreme cases to be too busy handling the query to handle incoming transactional data which is its core function. Therefore it is prescient to design a data warehousing model which will provide for an offline solution where reporting, standard and ad hoc queries, analysis and data manipulation or transformation can be performed without any adverse effect on the operations of the real time transactional systems.

Data Warehouse Infrastructure Architecture

A data warehouse is conceptually simply a shared data repository typically a database designed to enable offline business intelligence activities such as ad hoc and advanced reporting. Therefore the reason we wish to implement a warehouse into the IT infrastructure is to help users understand and enhance their organization's performance through greater access to information via self-service analytics and reporting. The conceptual design of the warehouse is to optimise query and analysis (read operations) rather than for transaction processing (write operations), and so it holds large amounts of historical data derived. This historical operational data comes from the business transaction data, but commonly today contemporary warehouse designs can and often do include data from other sources such as the internet. Therefore we could summarise, the fundamental purpose of a data warehouses is to separate analysis workload from transaction workload and hence to enable an organization to consolidate historical and topical data from several sources.

The benefits of such a design are in:

Maintaining historical records (Time variant)

Analyzing the data for business intelligence in order to gain a better understanding of the business operations dynamics and hence to improve the business

In the simplest infrastructure design, a basic blueprint of a data warehouse may be just an addition to a relational database utilising a master slave mechanism and data replication, which provides a clone of the database on another disk or host. This may be a common implementation where an SMB has an ecommerce transactional system on one or more web servers or an ERP system hosted on several host but sharing a common database schema. In that case these would be referred to as a homogeneous system and the process of extracting, transforming and loading data into our cloned schema would be relatively straightforward.

However, in most cases a data warehouse is a repository for more than one operational system such as where an organisation, still an SMB might have CRM, ERP, HR, and web based ecommerce systems that do not share a common database management system let alone a common schema. For example one may be MS SQL, another MySQL or Oracle Express and yet another PostGRES. In this heterogeneous environment a data warehouse environment can become vastly more complex but also much more rewarding.

Therefore, a data warehouse system in a heterogeneous environment will need considerably more planning and forethought as the ingestion of data from diverse sources into a common schema is no trivial task. It will require as a matter of physics to include in addition to the issues presented in the transportation of the data an extraction, transformation, and loading (ETL) solution, in order to integrate and provide a platform for all the disparate systems.

There have been tremendous advancements in what are termed ETL technologies over the last two decades and although it is certainly no longer as threatening a prospect it is still something that cannot be considered plug and play. Therefore we need to spend time considering the options and testing out the potential solutions. Spending effort at the design stage in getting this foundational technology correct will save us a lot of doubt and worry later. It is also important because the business users must have confidence in the veracity of the data. As it will be on this warehouse infrastructure, or this centralised data repository platform that the business will perform statistical analysis, reporting, perform data mining. Therefore the users must have confidence to unquestioningly run client analysis tools that manage the process of gathering data, transforming it into useful, actionable information, and delivering it to business users. Hence, a data warehouse system will be optimized to consolidate data from many sources to achieve a key goal: it becomes your organization's "single source of truth".

To achieve the goal of creating a platform for enhanced business intelligence, within the budgets and resources of an SMB, requires that the data warehouse works with data collected from multiple sources on affordable technology. The source data may come from internally developed systems, purchased applications, third-party data syndicates and other sources such as social media. The data provenance may be from transactions from sales, production, marketing, human resources and other departments. In addition in today's world of big data, the data may be many billions of individual clicks on web sites or the massive data streams from sensors built into complex machinery and the Internet of Things. There is great value in having a consistent single source of data that all users can look to with confidence; it prevents many disputes and enhances decision-making efficiency.

Data warehouses are distinct from online transaction processing (OLTP) systems and require different design specifications not just a larger more powerful box. With a data warehouse you are going to separate analysis workload from transaction workload. Thus data warehouses are very much read-oriented systems. They have a far higher amount of data reading versus writing and updating. This enables far better analytical performance and avoids impacting your operational transaction systems. A data warehouse usually stores many years of data to support historical analysis, which is in sharp contrast to an operational transaction server which requires archiving data regularly to prevent a decline in performance. The archiving of data from the operational servers can now be accelerated so that they only hold a few months of live data. Instead, the historical data in a data warehouse is typically loaded through an extraction, transformation, and loading (ETL) process from multiple data sources at the beginning and then incremental transaction data is perhaps uploaded on a daily basis. In fact with most modern data warehouses the extract, load, transform (ELT) architecture in which all or most data transformation is performed resides on the database that hosts the data warehouse so they tend to trickle feed data in at almost real time. Because of this there is increasingly no need for an (operational data store) ODS as a staging area but most designs still include these artefacts. It is important to note that defining the ETL process is a very large part of the design effort of a data warehouse. Similarly, the speed and reliability of ETL operations are the foundation of the data warehouse once it is up and running.

The advantageous of having a direct ETL feed into the warehouse is that users of the data warehouse can then perform data analyses on present rather than historical data. Users often run queries that are time-related. Examples include consolidation of last year's sales figures, inventory analysis, and profit by product and by customer. Having fresh data in the warehouse provides a picture of the present as well as the past, which better equips analysts to predict the probability of events occurring in the future.

But time-focused or not, users want self-service, the popularity of the spreadsheet and the way managers cling to that technology like a comfort blanket shows us that. They want to play with the data to "slice and dice" their data however they see fit and a well-designed data warehouse will be flexible enough to meet those demands. Users will sometimes need highly aggregated data, and other times they will need to drill down to details. More sophisticated data analytics will often include trend analyses and data mining, which are analysis techniques that use existing data to forecast trends or predict futures. However, with today's focus on data privacy with security, confidentiality and integrity being of keen interest to authorities and industry watchdogs we cannot just provide a platform for open access to data. There will have to be ways to ring fence sensitive data in addition to the application of traditional identity and access management techniques deployed on databases.

For this purpose we may consider the suitability of a data mart, which serves the same role as a data warehouse, but it is intentionally limited in scope. It may serve one particular department or line of business. The advantage of a data mart versus a data warehouse is that it can be created much faster due to its limited coverage but that attribute is also hugely beneficial. The limited scope of data marts is a perfect way to ring fence particularly

security sensitive data such as HR and Finance data. Data marts provide natural barriers and demarcation points which greatly assist with the creation and enforcement of identity and access management policy. However, data marts also create problems with inconsistency. It takes robust data management and tight discipline in order to keep data consistent across several data marts. We have to be very careful here that we do not in our attempt to mitigate the issue of departments all having their own version of the truth make this worse. To address this issue two distinct types of data marts have evolved. First we have the concept of the independent data mart which receives the data directly from data source. These marts are easy to deploy but have a tendency to turn into islands of inconsistent information. On the other hand we have what are termed dependent data marts and these in contrast to the independent marts are fed from an existing data warehouse. Dependent data marts will usually avoid the problems of data inconsistency, due to the source provenance but they require that a data warehouse already exist.

There can be much confusion as to the definition of a data warehouse as it can physically and logically have many manifestations hence it can be difficult to differentiate a warehouse from a mart, an ODS or even a slave reporting and replication system. The confusion is often compounded by the plethora of middleware that has sprouted up that interfaces user visualization and presentation tools with backend source databases. In order to clarify and pin down a definition of a Data Warehouse several characteristics have been set forth by William Inmon A man widely credited with the title of 'Father of the Data Warehouse'.

Subject Oriented

The first characteristic is that data warehouses are subject orientated they are specifically designed to help the data user and data consumer analyze data. For example, to learn more about the company's sales performance, a user can build a data warehouse that concentrates on just the sales department's transactions. Using this data warehouse, they now have a platform for answering questions such as "Who was our best customer for this item last year?" or "Who is likely to be our best customer next year?" This ability to define a data warehouse by subject matter, the sales transactions in this case makes the data warehouse subject oriented.

Integrated

Integration is closely related to subject orientation. It is not just restricted to ETL because Data warehouses must go several steps further by translating data from disparate sources into a consistent format. Therefore they must resolve additional issues such as naming convention conflicts and inconsistencies among units of measure, such as the date format. Only when this has been completed is it possible to claim that they are integrated.

Non-volatile

Non-volatile means that, once data is entered into the data warehouse, data should not change. The data should be immutable as this is only logical as the fundamental purpose of a data warehouse is to enable the analysis of historical data and hence to determine what has occurred and why.

Time Variant

A data warehouse's places great emphasis on historical data and the relationships, events or changes that may occur over time and this is what is meant by the term time variant. It is necessary for a data warehouse to exhibit this characteristic in order for it to have the capability to discover trends and identify hidden patterns and relationships in business.

Key Characteristics of a Data Warehouse

To summarize some of the other key characteristics of a data warehouse they are as follows:

Data is structured for simplicity of access and high-speed query performance.

End users are time-sensitive and desire speed-of-thought response times.

Large amounts of historical data are used.

Queries often retrieve large amounts of data, perhaps many thousands of rows.

Both predefined and ad hoc queries are common.

The data load involves multiple sources and transformations.

In general, it can be distilled down to fast query performance with high data throughput as being the key performance requirements for a successful data warehouse.

Contrasting OLTP and Data Warehousing Data Models

As we have just seen there are several important differences between an OLTP system and a data warehouse. The significant difference between the types of data models is that data warehouses are not exclusively in third normal form (3NF), which is a type of data normalization technique or strategy predominantly found in OLTP environments.

Due to the performance expectations, the roles they play and difference in workloads Data warehouses and OLTP systems have very different requirements. Here are some examples of differences between typical data warehouses and OLTP systems:

Workload

OLTP systems are designed with a specific application to support so they are optimised for only predefined operations – the applications specifications and requirements. As a result the OLTP acts as a back-end for its application so might be specifically tuned or designed to support only these operations to the detriment of other functionality.

Data warehouses are designed to accommodate a wide range of purposes and are generalist by nature so they can equally serve ad hoc queries and data analysis. The designer of a warehouse might not know the workload expectations of the data warehouse in advance, so a data warehouse should be optimized to perform well for a wide variety of possible query and analytical operations.

Data modifications

An OLTP system performs well when end users or other systems routinely issue individual data modification statements to the database i.e. as its name suggests high volumes and/or velocity of small transactions. Therefore, the OLTP database is always going to be up to date, and reflect the current state of each business transaction in real time.

A data warehouse in contrast is typically updated on a scheduled basis by the ETL process (run nightly or weekly) using bulk data modification techniques. Modern warehouses can trickle feed updates in real time but most still run on a scheduled basis. Importantly though, the end users of a data warehouse do not directly update the data warehouse except perhaps when using analytical tools. An example is when performing analysis techniques such

as data mining, or in order to make predictions with associated probabilities, assign customers to market segments, and developing customer profiles.

Schema design

Data warehouses often use 3NF schemas or partially denormalized schemas such as snowflake, which does have many normalised dimensions. But they do often, Marts especially, use the common Star schema where only the Fact table is normalised, in order to optimize query and analytical performance.

OLTP systems often use 3NF fully normalized schemas to optimize update/insert/delete performance, and to guarantee data consistency at the expense of performance and complexity.

Typical mode of operations

A typical data warehouse is optimised to read (query) and scan thousands or millions of rows. For example, "Find the total sales for all mobile telephones last month."

A typical OLTP operation is to write (update) thousands of transactions into rows but only reads (query) a handful of records. For example, "Retrieve the current stock holding for iPhone 7."

Historical data

Data warehouses usually store many months or years of data. This is to support historical analysis and reporting.

OLTP systems usually store data from only a few weeks or months. The OLTP system stores only historical data as needed to successfully meet the requirements of the current transaction.

Data Warehouse Architectures

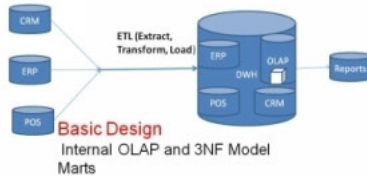
Data warehouses can fit many IT architectures and so they vary in implementation depending upon the specifics of an organization's or projects requirements. However, generally there are three common architectures, which are:

Data Warehouse Architecture: Basic

Data Warehouse Architecture: with a Staging Area

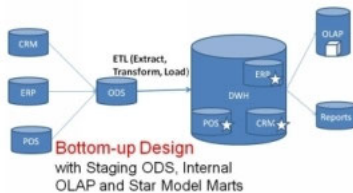
Data Warehouse Architecture: with a Staging Area and Data Marts

Data Warehouse Infrastructure Architecture:

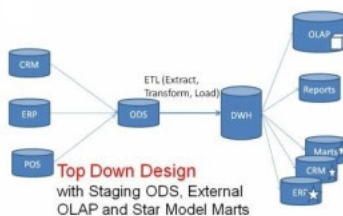


The Basic Design is suitable for an SMB warehouse. In this architecture the operational servers connect directly through ETL process to the warehouse and the data is stored in 3NF in individual data marts for each application. This design will provide a central repository for reporting and analysis using scheduled pre-prepared or ad hoc reports from the individual marts or from the OLAP Cube for preprocessed Financial and Accounting reports.

An alternative design is the bottom-up design where we have a staging area and an ODS handling the Extract, Transform and Load process. In this design the data is loaded into marts that support a common Star schema, the DWH is then build from the aggregation of data held in the Marts using a common schema and common bus.



In contrast to the bottom-up architecture the top down approach, which is the most common architecture deployed, supports a staging area for ETL but it has external data marts which are built from the data held in the central warehouse. The OLAP server has also be moved to a dedicated server to host its multi-dimensional (Cube) schema.



Designing Data Warehousing Logical Design

Now that we have learned about the characteristics and some of the architectural patterns that we can use as reference it is time to consider designing the warehouse. Once the decision to build a data warehouse has been made - after due consideration of the well-defined business case, specifications and requirements - the next step is to agree upon the scope of the business goals, and create a conceptual high-level design. Typically, with most in-house projects this is done during the initial project planning stage, however with external partners this is usually done during the RFP (Request for Proposal) but either way this is the time to translate the projects specifications and requirements into a system deliverable. To do so, the designers will need to create the logical and physical design for the data warehouse. The job for the designer then is to define:

The specific data content

Relationships within and between groups of data

The system environment supporting your data warehouse

The data transformations required

The frequency with which data is refreshed

The major difference between the logical design and the physical design is that the former is more conceptual and abstract than the latter. In the logical design the focus is on the logical relationships among the objects that will be held in the database. On the other hand, during the physical design, the focus is on the most effective way of storing and retrieving the objects.

The primary design goal is always to meet the application's specifications and requirements, which should be well defined and documented. If there is no such document, which is quite common in SMB development then start by writing one up and get the stakeholders to understand and agree by signing it off- as this document will be the only benchmark for success or failure. In an SMB id no formal documentation or sign up is possible then the only course of action left is to orient the design toward the needs of the majority of the end users. Unfortunately, the end users in SMB's either have unrealistic expectations or have no specific requirements other than to have reports in spreadsheet format. A safe middle ground is not going to be easy to find but in general users want to create standard reports, some ad hoc queries, perform analysis on their data, such as financial modelling and to be able to visualise and present the reports in spreadsheets for meetings and in graphical format for presentations. That necessitates looking at aggregated data, rather than at individual transactions, which a well-planned design should accomplish with aplomb but it is wise to allow for growth and changes as the requirements of the users shift and evolve over time.

By beginning with the logical design, you can focus on the information specifications and requirements and save the implementation details for later.

Creating a Logical Design

As we have seen the logical design is conceptual and abstract but a useful technique used to model the SMB organization's logical information requirements is a technique called entity-relationship modelling. This design technique involves identifying the things of importance to the business (entities), the properties of these things (attributes), and how they are related to one another (relationships).

In short form the process of logical design involves arranging the organisations data into a series of logical relationships called entities and attributes. In relational databases, an entity often maps to a table. An attribute is a component of an entity that defines the characteristics of the entity. In relational databases, an attribute will map to a column.

Just as we saw earlier with relational schemas we need to ensure that the working data is consistent, therefore we must use unique identifiers. A unique identifier, such as a surrogate key, which we discussed earlier, is something

you add to tables so that you can differentiate between similar items and positively identify an item when it appears in different places in the schema.

Remember that entity-relationship modelling is a purely logical exercise but it is applicable to the various common physical schema modelling techniques found in data warehousing environments, namely normalized (3NF) schemas in Data Warehousing environments, star or snowflake schemas in data marts. It will be the model of the source data and the specifications and requirements of the project that will drive the design process of the data warehouse schema. Of course being a logical exercise doesn't mean that we can forget about the constraint that will be applied during the physical implementation of the logical data warehouse model. For example we cannot lose sight of the constraints, budgetary or physical, that may require some changes to adapt the schema to the actual system parameters—size of computer, number of users, storage capacity, type of network, and software. A key part of designing the schema is whether to use a third normal form, star, or snowflake schema.

Third Normal Form Schemas

We were introduced to the concepts of the Third Normal Form design during our primer on relational database schema. So we are aware that the concept seeks to minimize data redundancy and avoid anomalies in data insertion, updates and deletion. 3NF design has a long heritage in online transaction processing (OLTP) systems, but many of the drives for normalisation such as conserving disk space and disk drive mechanical performance limitations are much less of a concern today. Despite perhaps disk conservation being less of a compelling reason to adopt 3NF normalisation as it surely was back in the 90's it is still undoubtedly best practice in OLTP system design to mandate the maximization of performance and accuracy when inserting, updating and deleting data. To this end we must ensure that transactions are handled as quickly as possible or the systems may be unable to handle the flow of events, queues may overflow, sessions can be lost resulting perhaps in the loss of sales. Working to those goals, 3NF designs strive to avoid redundant data, which in turn minimizes table locks as less redundant data requires to be updated at diverse locations, both of which can slow inserts, updates and deletes. However, 3NF designs, do act as an abstraction layer of sorts, though not by reducing the complexity but they do abstract the data from the specific application needs. This can be demonstrated when new types of data are added to the environment, as the data model can accommodate these additions with relative ease and minimal impact to existing applications. Likewise, if you have a new analyses technique to perform in the data warehouse, a well-designed 3NF schema will be able to handle them without a refit of the underlying data structures.

3NF schemas provide great flexibility, but it also has many disadvantages. With the kinds of queries involved in data warehousing, 3NF models are not ideal, as they will often require the joining of many tables, this inherent design issue imposes performance penalties. Furthermore, it can be complex for query builders, and not just DBA's but even business intelligence tools and applications struggle to make sense of the complexity of the schema and successfully join the tables needed for a given piece of data. Unfortunately, we cannot escape the reality that the more tables there are in a schema means more potential data access paths and inter-connections, which makes the database query optimizer's job harder, which invariably result in slow query performance.

Design Concepts for 3NF Schemas

The high level goals of the 3NF design for a data warehouse are to ensure data where ever possible is not redundant and is only stored in one place. This will ensure high write performance and data integrity and consistency. We discussed earlier how to design at a high level a relational 3NF data schema, but in this section we will revisit a few key points in order to understand there relevance or importance to data warehouses.

Some key 3NF schema design concepts that are relevant to data warehousing are as follows:

Identifying Candidate Primary Keys

Foreign Key Relationships and Referential Integrity Constraints

Denormalization

Identifying Candidate Primary Keys

A primary key is an attribute that uniquely identifies a specific record in a table. Primary keys can be identified through single or multiple columns. If the data model does not lead to a simple unique identification through its attributes, or you would require too many attributes to uniquely identify a single record, or the data is prone to changes, the usage of a surrogate key is highly recommended.

Specifically, a 3NF schema relies on a proper, simple and unique identification method since queries tend to have many table joins and all columns necessary to uniquely identify a record are needed as join condition to avoid row duplication through the join.

Foreign Key Relationships and Referential Integrity Constraints

3NF schemas in data warehousing environments often resemble the data model of its OLTP source systems, in which the logical consistency between tables is enforced through primary key - foreign key relationships, also known as parent-child relationship. A foreign key resolves a 1-to-many relationship in relational system and ensures logical consistency: for example, you cannot have an order line item without an order header, or an employee working for a non-existent department.

While such referential are considered mandatory and enforced in OLTP system, data warehousing systems often implement them as declarative, non-enforced conditions, relying on the ETL process to ensure data consistency. Foreign keys and referential integrity constraints therefore should where ever possible be defined as non-enforced conditions, since it enables better query optimization and cardinality estimates.

Denormalization

One of the biggest issues with normalisation regardless of its many forms is that a normalized model tends to decompose logical entities - such as a customer, product, or an order - into many physical tables. The issue is that these entities are closely related and decomposing that tight bond makes what should be a simple query require the join of many tables for joining information that is always used together. While this may not be a problem from a query processing perspective, it makes the notion of self-service analysis and reporting a nonsense, further it can unnecessarily stress the DBA who has to write the query and destroys all notion of ad hoc reporting. Therefore, it should be possible to have some sensible level of denormalization in the 3NF data warehousing models, so that information that is used together stays together in a logical form as views or in a physical form through slightly denormalized tables.

Star Schemas

In data warehousing systems with embedded logical or physical data marts the star schemas are often used as the data structure. The term star schema refers to a "dimensional modelling" method or approach to defining your data model in a denormalised manner. The strategy behind the concept of dimensional modeling was to build a basic model based on each major business process such as sales, logistics or marketing and then build a star schema for each. In this design each star schema could be considered as an independent data mart, and what was surprising was that only around 20 data marts maximum could cover the business intelligence needs of even a large enterprise.

What is so attractive regards the star schema is its visual simplicity. Compared to 3NF designs, the number of tables involved in dimensional modeling is tiny and many star schemas for even quite large and complex business process will have only around ten or less. In addition to the individual simplicity is another compelling reason for selecting the star schema and that was they could be easily aggregated to form galaxies where common dimension tables might be shared and this reduced some of the disk space saving concerns. What is more, some strategies took this idea a step further and advocated that the star schemas should use a common design for their dimension and fact tables which would facilitate easy merging of star schemas. Thus, users are able to get data from multiple star schemas with minimal effort.

This idea of having conformed dimensions and conformed facts tables is the basis of the bottom up warehouse design where individual star configured data marts based on a single business process are brought together to form the main data warehouse. Any subsequent data mart is simply assimilated into the main body due to the conformed tables being a perfect match. This way of designing a warehouse by combining the smaller data marts is as we mentioned earlier the basis of the popular bottoms-up approach. The benefits of this approach especially for the SMB are the simplicity of the implementation for you can start building one business process at a time and do so in isolation of other business units and processes. Further, the bottoms-up approach facilitates the low-hanging fruit approach whereby you can collect early successes by selecting and concentrating the efforts onto the business units likely to be easiest to deploy and provide the maximum early visible returns.

An interesting point to note is that during the deployment planning process the discussions of star schemas are more straightforward and much less abstracted from the physical database than 3NF descriptions. This is due to the schema being far easier to understand due to the pragmatic emphasis of dimensional modeling on the needs of business intelligence users.

The star design principles have a firm foundation in their inherent structural simplicity and hence their high performance in running user query. The acceptable time taken to process queries is of course qualitative as it is dependent on the expectations of the user. However most queries in today are generated by user defined reporting tools embedded in browsers and applications. Indeed some of the analytical and reporting tools require that there is a star schema so it's important to make the query generation engines easy to integrate, convenient to develop and reliable for the tools and application developers.

Facts and Dimensions in Star Schemas

Star schemas divide data into facts and dimensions. Facts are the measurements of some event such as a sale and are typically numbers. Dimensions are the categories you use to identify facts, such as date, location, and product. Fact tables have measurement data. They have many rows but typically not many columns. Fact tables for a large enterprise application can easily hold billions of rows. For many star schemas, the fact table will represent well over 90 percent of the total storage space. A fact table has a composite key made up of the primary keys of the dimension tables of the schema.

A fact table contains either detail-level facts or facts that have been aggregated. Fact tables that contain aggregated facts are often called summary tables. A fact table usually contains facts with the same level of aggregation. Though most facts are additive, they can also be semi-additive or non-additive. Additive facts can be aggregated by simple arithmetical addition. A common example of this is sales. Non-additive facts cannot be added at all. An example of this is averages. Semi-additive facts can be aggregated along some of the dimensions and not along others. An example of this is inventory levels stored in physical warehouses, where you may be able to add across a dimension of warehouse sites, but you cannot aggregate across time.

In terms of adding rows to data in a fact table, there are three main approaches:

Transaction-based - Shows a row for the finest level detail in a transaction. A row is entered only if a transaction has occurred for a given combination of dimension values. This is the most common type of fact table.

Periodic Snapshot - Shows data as of the end of a regular time interval, such as daily or weekly. If a row for the snapshot exists in a prior period, a row is entered for it in the new period even if no activity related to it has occurred in the latest interval. This type of fact table is useful in complex business processes where it is difficult to compute snapshot values from individual transaction rows.

Accumulating Snapshot- Shows one row for each occurrence of a short-lived process. The rows contain multiple dates tracking major milestones of a short-lived process. Unlike the other two types of fact tables, rows in an accumulating snapshot are updated multiple times as the tracked process moves forward.

Dimension Tables in Data Warehouses

Dimension tables provide category data to give context to the fact data. For instance, a star schema for sales data will have dimension tables for product, date, sales location, promotion and more. Dimension tables act as lookup or reference tables because their information lets you choose the values used to constrain your queries. The values in many dimension tables may change infrequently. As an example, a dimension of geographies showing cities may be fairly static. But when dimension values do change, it is vital to update them fast and reliably. Of course, there are situations where data warehouse dimension values change frequently. The customer dimension for an enterprise will certainly be subject to a frequent stream of updates and deletions.

A key aspect of dimension tables is the hierarchy information they provide. Dimension data typically has rows for the lowest level of detail plus rows for aggregated dimension values. These natural rollups or aggregations within a dimension table are called hierarchies and add great value for analyses. For instance, if you want to calculate the share of sales that a specific product represents within its specific product category, it is far easier and more reliable to have a predefined hierarchy for product aggregation than to specify all the elements of the product category in each query. Because hierarchy information is so valuable, it is common to find multiple hierarchies reflected in a dimension table.

Dimension tables are usually textual and descriptive, and you will use their values as the row headers, column headers and page headers of the reports generated by your queries. While dimension tables have far fewer rows than fact tables, they can be quite wide, with dozens of columns. A location dimension table might have columns indicating every level of its rollup hierarchy, and may show multiple hierarchies reflected in the table. The location dimension table could have columns for its geographic rollup, such as street address, postal code, city, state/province, and country. The same table could include a rollup hierarchy set up for the sales organization, with columns for sales district, sales territory, sales region, and characteristics.

Design Concepts in Star Schemas

Here we touch on some of the key terms used in star schemas. This is by no means a full set, but is intended to highlight some of the areas worth your consideration.

Data Grain

One of the most important tasks when designing your model is to consider the level of detail it will provide, referred to as the grain of the data. Consider a sales schema: will the grain be very fine, storing every single item purchased by each customer? Or will it be a coarse grain, storing only the daily totals of sales for each product at each store? In modern data warehousing there is a strong emphasis on providing the finest grain data possible, because this allows for maximum analytic power. Dimensional modeling experts generally recommend that each fact table store just one grain level. Presenting fact data in single-grain tables supports more reliable querying and table maintenance, because there is no ambiguity about the scope of any row in a fact table.

Working with Multiple Star Schemas

Because the star schema design approach is intended to chunk data into distinct processes, you need reliable and optimal ways to traverse the schemas when queries span multiple tables and schemas. One term for this ability is data warehouse bus architecture. By sharing a common bus – a highway - for transportation and interconnectivity then data exchange between Marts is simplified and this can be achieved if the Marts use conformed dimensions and conformed fact tables.

Conformed Dimensions

Conformed dimensions means that dimensions are designed identically across the various star schemas. Conformed dimensions use the same values, column names and data types consistently across multiple stars. The conformed dimensions do not have to contain the same number of rows in each schema's copy of the dimension table, as long as the rows in the shorter tables are a true subset of the larger tables.

Conformed Facts

If the fact columns in multiple fact tables have exactly the same meaning, then they are considered conformed facts. Such facts can be used together reliably in calculations even though they are from different tables. Conformed facts should have the same column names to indicate their conformed status. Facts that are not conformed should always have different names to highlight their different meanings.

Surrogate Keys

Surrogate or artificial keys, usually sequential integers, are recommended for dimension tables. By using surrogate keys, the data is insulated from operational changes. Also, compact integer keys may allow for better performance than large and complex alphanumeric keys.

Degenerate Dimensions

Degenerate dimensions are columns in fact tables that do not join to a dimension table. They are typically items such as order numbers and invoice numbers. You will see them when the grain of a fact table is at the level of an order line-item or a single transaction.

Junk Dimensions

Junk dimensions are abstract dimension tables used to hold text lookup values for flags and codes in fact tables. These dimensions are referred to as junk, not because they have low value, but because they hold an assortment of columns for convenience, analogous to the idea of a "junk drawer" in the home. The number of distinct values (cardinality) of each column in a junk dimension table is typically small.

Embedded Hierarchy

Classic dimensional modeling with star schemas advocates that each table contain data at a single grain. However, there are situations where designers choose to have multiple grains in a table, and these commonly represent a rollup hierarchy. A single sales fact table, for instance, might contain transaction-level data, then a day-level rollup by product, then a month-level rollup by product. In such cases, the fact table will need to contain a level column indicating the hierarchy level applying to each row, and queries against the table will need to include a level predicate.

'Factless' Fact Tables

The strangely named Factless fact tables are so called because they do not contain measures such as sales price or quantity sold. Instead, the rows of a factless fact table are used to show events not represented by other fact tables. Another use for factless tables is as a "coverage table" which holds all the possible events that could have occurred in a given situation, such as all the products that were part of a sales promotion and might have been sold at the promotional price.

Slowly Changing Dimensions

One of the certainties of data warehousing is that the way data is categorized will change. Product names and category names will change. Characteristics of a store will change. The areas included in sales territories will change. The timing and extent of these changes will not always be predictable. How can these slowly changing dimensions be handled? Star schemas treat these in three main ways:

Type 1

The dimension values that change are simply overwritten, with no history kept. This creates a problem for time-based analyses. Also, it invalidates any existing aggregates that depended on the old value of the dimension.

Type 2

When a dimension value changes, a new dimension row showing the new value and having a new surrogate key is created. You may choose to include date columns in our dimension showing when the new row is valid and when it is expired. No changes need be made to the fact table.

Type 3

When a dimension value is changed, the prior value is stored in a different column of the same row. This enables easy query generation if you want to compare results using the current and prior value of the column.

In practice, Type 2 is the most common treatment for slowly changing dimensions.

The dimensional modeling style is very different from the 3NF approach that's goal is to minimize data redundancy and maximise data consistency. Instead the star schema accepts data redundancy in its dimension tables as this promotes better understanding and an increase in data retrieval performance. However, a common criticism of star schemas is that they limit analysis flexibility compared to 3NF designs. But, a well-designed dimensional model can be extended to enable new types of analysis, and star schemas have been successful and widely adopted for many years.

As noted earlier, the modern approach to data warehousing does not pit star schemas and 3NF against each other. Rather, both techniques are used, with a foundation layer of 3NF - the Data Warehouse of 3NF, acting as the bedrock data, and star schemas as a central part of an access and performance optimization layer.

Chapter 12 - Overview of Multi-Dimensional Data Modeling

A multidimensional database (MDB) is a type of database that is optimized for data warehouse and online analytical processing (OLAP) applications. Multidimensional databases are frequently created using input from existing relational databases. Whereas a relational database is typically accessed using a Structured Query Language (SQL) query, a multidimensional database allows a user to ask questions like "How many electric cars have been sold in Los Angeles so far this year?" and similar questions related to summarizing business operations and trends.

There are several types of OLAP applications that we need to become familiar with the most common access data from a multidimensional database is known as a MOLAP (multidimensional OLAP) application. A multidimensional database - or a multidimensional database management system (MDDBMS) - implies the ability to rapidly process the data in the database so that answers can be generated quickly.

Online analytical processing (OLAP) is a multidimensional, multiuser, client-server computing environment for users who need to analyze enterprise data. Finance departments use OLAP for applications such as budgeting, activity-based costing (allocations), financial performance analysis, and financial modeling. Sales departments use OLAP for sales analysis and forecasting. Marketing departments use OLAP for market research analysis, sales forecasting, promotions analysis, customer analysis, and market/customer segmentation. Typical manufacturing OLAP applications include production planning and defect analysis.

Important to all of these applications is the ability to provide managers the information that they need to make effective decisions about an organization's strategic directions. A successful OLAP application provides information as needed; that is, it provides "just-in-time" information for effective decision-making.

Providing such information requires more than a base level of detailed data. Just-in-time information is computed data that usually reflects complex relationships and is often calculated on the fly. Analyzing and modeling complex relationships are practical only if response times are consistently short. In addition, because the nature of data relationships may not be known in advance, the data model must be flexible. A truly flexible data model ensures that OLAP systems can respond to changing business requirements as needed for effective decision making.

Although OLAP applications are found in widely divergent functional areas, all require the following key features:

Multidimensional views of data

Calculation-intensive capabilities

Time intelligence

Key to OLAP systems are multidimensional databases, which not only consolidate and calculate data; but also provide retrieval and calculation of a variety of data subsets. A multidimensional database supports multiple views of data sets for users who need to analyze the relationships between data categories. For example, a marketing analyst might ask the following questions:

How did Product A sell last month? How does this figure compare to sales in the same month over the last five years? How did the product sell by branch, region, and territory?

Did this product sell better in particular regions? Are there regional trends?

Did customers return Product A last year? Were the returns due to product defects? Did the company manufacture the products in a specific plant?

Did commissions and pricing affect how salespeople sold the product? Did certain salespeople sell more?

In multidimensional databases, the number of data views is limited only by the database outline, the structure that defines all elements of the database. Users can pivot the data to see information from a different viewpoint, drill down to find more detailed information, or drill up to see an overview.

The databases that a business uses to store all its transactions and records are called online transaction processing (OLTP) databases. These databases usually have records that are entered one at a time and that contain a wealth of information that can be used by strategists to make informed decisions about their business. The databases that are used to store the data, however, were not designed for analysis. Therefore, retrieving answers from these databases is costly in terms of time and effort. OLAP databases are specialized databases that are designed to help extract this business intelligence information from the data.

Dimensional objects are an integral part of OLAP. Because OLAP is on-line with respect to the user but not the operational process it collects and processes data from, it is expected to provide answers quickly. This is due to the way that they are used, because they are isolated from the operational process but online to the users, analysts then can pose iterative queries during interactive sessions, and not have to schedule batch jobs that run overnight. Also, because OLAP is analytic by nature and not just a standard report generation engine the queries are often complex. Therefore, designers make good use of the dimensional objects and the OLAP engine in order to design the solutions for complex queries in real time.

OLAP Cubes

The dimensional objects include cubes, which are multidimensional data structures which can contain measures, dimensions, attributes, levels, and hierarchies. Conceptually, a multidimensional database uses the idea of a data cube to represent the dimensions of data available to a user. For example, "sales" could be viewed in the dimensions of product model, geography, time, or some additional dimension. In this case, "sales" is known as the measure attribute of the data cube and the other dimensions are seen as feature attributes. Additionally, a database creator can define hierarchies and levels within a dimension (for example, state and city levels within a regional hierarchy).

The simplicity of the model, which is not always apparent at first glance, is in the way that it defines objects that represent real-world business entities. If we cast our mind back to one of the issues with 3NF normalisation, there was a decoupling of highly associated information that even though the segments of information were almost always used together, normalisation dictated that they would be separated and placed in other tables. The beauty and the simplicity with cubes is that this issue can be largely solved. This is because the business analysts, for example an accountant, know which business measures they are interested in examining, which dimensions and attributes make the data meaningful, and how the dimensions of their business are organized into levels and hierarchies. Therefore, if they relate this information to the cube designer all of the dimensions, attributes along with their hierarchies and measures can be assembled into a multi-dimensional cube.

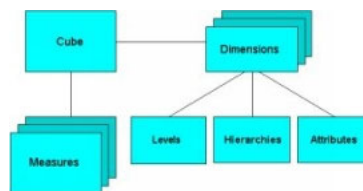


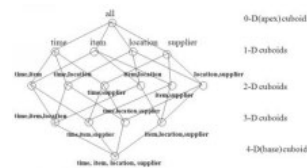
Diagram of the OLAP Dimensional Model

As can be seen from the diagram of the OLAP Dimensional model it is highly structured and structure implies rules that govern the relationships among the data and control how the data can be queried. Cubes are the physical implementation of the logical dimensional model, and thus are highly optimized for custom built dimensional queries. The technology that underlies cubes is based on an indexed multidimensional array model, which provides direct cell access. What this means is that the dimensions are pre-joined to the measures and the OLAP engine leverages this in performing highly efficient cross-cube joins for inter-row calculations, outer joins for time series analysis, and indexing.

The OLAP engine manipulates dimensional objects in the same way that the SQL engine manipulates relational objects. However, because the OLAP engine is optimized to calculate analytic functions, and dimensional objects are optimized for analysis, analytic and row functions can be calculated much faster in OLAP than in SQL.

Cubes in more detail

Cubes usually corresponds to a single fact table or view and are a flexible data structure, which provide a means of organizing measures that conform to the exact same dimensions so that Measures in the same cube can easily be analyzed and displayed together.



Cubes are referenced by their level and as can be seen from the figure above 0-D Cuboid (0-dimensions are the apex and the highest possible level. The next level 0-1 (The first Dimension contains all the high level standard dimensions for instance Time, Item, Location and Supplier). As we progress down the cube we see two and three dimensions at each layer respectively until we reach the base cuboid in this case 4-D, which has four dimensions – this will be the unique reference for this particular cube – the primary key.

In this cube and in many of the simplified examples you will see there will be 4 or 5 dimensions but in the real world cubes can have many more dimensions up to 64 in some instances, hence the term multidimensional is true but often they are not cubes – well not as we know them.

Measures

Measures populate the cells of a cube with the facts collected about business operations. Measures are organized by dimensions, which typically include a Time dimension.

Measures in an analytic database are snapshots of historical data, derived from data in a transactional database or other data sources. A characteristic of these measures are that they are static and remain consistent while analysts are using them to inform their decisions. A critical decision in defining a measure is the lowest level of detail required – the grain. Users will often not drill down far enough to view this detail data, it may be of no interest but it determines the foundation and therefore the types of analysis that can be performed. For example, a market analyst does not need to know that - Jason Cabin, Margate, London, England, purchased a blue pair of jeans, size 30 on July 16, 2017, at 12:27 p.m. from the store in Cannon St, paying by credit card – as that level of detail is too detailed for their purpose. What they want to find out was which colour and size of jeans was most popular in the summer of 2017 in the City of London area.

It is the base level which determines whether analysts can get an answer to their question and this is where hierarchies and levels are so important. For this particular example: the question, “which colour and size of jeans was most popular in the summer of 2017 in the London area”. We have to consider whether we have the data to answer the question, in this case we could if Time dimension was rolled up into months, Customer could be rolled up into regions, and Product could be rolled up into items (such as Jeans) with an attribute of colour and size. Therefore we would have enough aggregated data to answer the question.

However, this level of aggregated data could not answer the question: “At what time of day are students most likely to buy blue narrow-cut jeans? This is due to the fact that the data before it was loaded was aggregated and does not include the attributes for the customer dimension “occupation” or for the Product attribute “Style/Fit” so that granular level of detail has been lost. Therefore, an important decision in the design process is the extent to which the data is to be aggregated before being loaded into a data warehouse.

Dimensions

Dimensions contain a set of unique values that identify and categorize data they are similar to tables in a relational database. The base-level dimension values of the cube correspond to the unique keys of a fact table. Dimensions form the edges of a cube, and thus of the measures within the cube cells. Because measures are typically multidimensional, and a single value in a measure is meaningless on its own we must qualify it by matching it to members of other dimensions for it to be meaningful. For example, the Sales measure has four dimensions: Time, Customer, Product, and Channel. A particular Sales value (21,765.50) only has meaning when it can be matched and qualified by the other relevant measures such as a specific time period (Feb-06), a customer (Bartley Smith), a product (Shoes), and a channel (Website). When we can collate all the measures we have meaningful information.

Hierarchies and Levels

A hierarchy is a way to organize data at different levels of aggregation. In viewing data, analysts use dimension hierarchies to recognize trends at one level, drill down to lower levels to identify reasons for these trends, and roll up to higher levels to see what affect these trends have on a larger sector of the business.

Level-Based Hierarchies

Each level represents a position in the hierarchy. Each level above the base (or most detailed) level contains aggregate values for the levels below it. The members at different levels have a one-to-many parent-child relation. For example, Q1-05 and Q2-05 are the children (quarters) of the parent (Year) 2005.

A data warehouse may well contain snapshots of data scheduled to be collected every 8 hours. However, Analysts might prefer to view the data aggregated into days, weeks, quarters, or years. Thus, we will need to provide the Time dimension with a hierarchy with at least five levels.

A hierarchy typically contains several levels, and a single level can be included in multiple hierarchies. Therefore we can say that hierarchies and levels have a many-to-many relationship.

Each level typically corresponds to a column in a dimension table where the most granular, the level with the greatest detail is at the base level and hence it is the primary key.

Value-Based Hierarchies

Although hierarchies are typically composed of named levels, they do not have to be. The parent-child relations among dimension members may not define meaningful levels. For example, in an employee dimension, each manager has one or more reports, which forms a parent-child relation. Creating levels based on these relations (such as individual contributors, first-level managers, second-level managers, and so forth) may not be meaningful for analysis. Likewise, the line item dimension of financial data does not have levels. This type of hierarchy is called a value-based hierarchy.

Attributes

An attribute provides additional information about the data. Each attribute typically corresponds to a column in dimension table or view. Some attributes are used for display purposes and to provide more meaningful information. For example, you might have a product dimension that uses Stock Keeping Units (SKUs) for dimension members. The SKUs are an excellent way of uniquely identifying thousands of products, but are meaningless to most people. Therefore, you would define attributes for the descriptive labels on graphs and reports.

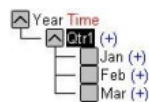
You might also have attributes like colours, model, or sizes. This type of attribute can be used for data selection and answering questions such as: Which colours were the most popular in cars in the summer of 2017? How does this compare with the previous summer?

Time attributes can provide information about the Time dimension that may be useful in some types of analysis, such as identifying the last day or the number of days in each time period.

Dimensions and Members

This section introduces the concepts of outlines, dimensions, and members within a multidimensional database. The understanding of the relationship between dimensions and members is crucial to gaining an understanding of the power of a multidimensional database.

Dimensions and members in a tree structure indicate a consolidation relationship and a dimension represents the highest consolidation level in the database outline. For example, in the figure below we can see a hierarchic structure for the dimension Time.



The core components of a business plan are represented by standard dimensions and often these will relate to departmental functions or processes. For example some commonly used standard dimensions are: Time, Accounts, Product, Channel, and Location. Because these are key representations of core components within a business the dimensions by their nature will change less frequently than their members.

There is another type of dimension, that is called an attribute dimension and these are associated with standard dimensions. The utility of attribute dimensions is that they can be used to group and analyze members of standard dimensions based on the member attributes (their characteristics). For example, you can compare the profitability of fizzy carbonated products that are packaged in glass to the profitability of fizzy carbonated products packaged in cans.

In multidimensional terms Members are the individual components of a dimension. For example, Product A, Product B, and Product C might be members of the Product dimension. Each member has a unique name. The database can store the data associated with a member, or it can dynamically calculate the data when a user retrieves it.

Outline Hierarchies

All database design begins with a logical mental sketch if not with pen and paper by creating a database outline, which accomplishes the following:

Defines the structural relationships between members in a database

Organizes data in the database

Defines the consolidations and mathematical relationships between items

Creating the database outline will solidify the concept of members to be used to represent data hierarchies. Each dimension will consist of one or more members. The members, in turn, may consist of other members. Hence, when you create a dimension, you are describing not just the member but also how to consolidate the values of the individual members. Within the tree structure of the database outline, a consolidation is a group of members in a branch of the tree.

For example, the Year dimension comprises five members: Qtr1, Qtr2, Qtr3, and Qtr4, each storing data for an individual quarter, plus the parent Year which is storing the accumulated summary data for the year. Qtr1 comprises four members: Jan, Feb, and Mar, each storing data for a month, plus Qtr1, which again will be storing the accumulated summary data for the quarter. Similarly, Qtr2, Qtr3, and Qtr4 comprise the members that represent the individual months plus the member that stores the quarterly totals.

Data Storage

Each data value in a multidimensional database is stored in one cell. A particular data value is referenced by specifying its coordinates along each standard dimension.

Consider a simplified database shown in A Multidimensional Database Outline. This database has three dimensions: Accounts, Time, and Scenario:

The Accounts dimension has four members: Sales, COGS, Margin, and Margin%.

The Time dimension has four quarter members, and Qtr1 has three month members

The Scenario dimension has two child members: Budget for budget values and Actual for actual values.

A Multidimensional Database Outline

Data Values

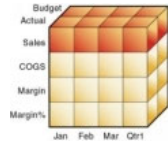
The intersection of one member from one dimension with one member from each of the other dimensions represents a data value. The example below has three dimensions (Accounts, Time, and Scenario); therefore, the dimensions and data values in the database can be represented as a cube. However multi-dimensional cubes can have many more dimensions 5 or 6 is typically the maximum limit ever shown in an example due to technical difficulties and human mental capacity to visualize such as object but cubes can have up to 64 dimensions.

Three-Dimensional Database



As illustrated below, when you specify Sales, you are specifying the slice of the database that contains eight Sales values, where Sales intersect with Actual and Budget.

Sales Slice of the Database



Slicing a database amounts to fixing one or more dimensions at a constant value while allowing the other dimensions to vary.

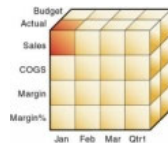
As illustrated in 'Actual, Sales Slice of the Database', when you specify Actual Sales, you are specifying the slice of the database that contains four Sales values, where Actual and Sales intersect.

Actual, Sales Slice of the Database



A data value is stored in one cell in the database. To refer to a specific data value in a multidimensional database, you specify its member on each dimension. In the Figure: 'Sales->Jan->Actual' in this slice of the database, the cell containing the data value for Sales, Jan, Actual is shaded. The path or navigation route to the data value can also be expressed using the cross-dimensional operator (->) as Sales -> Actual -> Jan.

Sales->Jan->Actual Slice of the Database



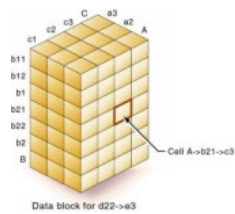
Navigating the Cube

In the simplest terms the database management system can navigate by just following the coordinates such as the indexes it assigns to data blocks (cubes). When the system has to search for a data value, it uses the index to locate the appropriate data block. In the example we show that the requested block is indexed. Then, when the block is located the system looks within the data block, in order to locate the cell containing the data value. Hence, each unique data value can be considered to exist in a cell within a data block. The index entry provides a pointer to the data block. The index handles data efficiently because it includes only pointers to existing data blocks i.e. those that contain at least some valid data within the cells. For combinations of dimensions that have no corresponding data then the system does not create a data block and hence an index does not exist. In this way the system ensures very efficient storage of data by not wasting space creating data blocks for non-existing data.

This is important as data may be very sparsely distributed throughout the data model. For example not all products may be sold at every location therefore it is wasteful to create data blocks that will never hold any data.

Each data block is a multidimensional array that contains a fixed, ordered location for each possible combination of dense dimension members. Accessing a cell in the block does not involve sequential or index searches. The search is almost instantaneous, resulting in optimal retrieval and calculation speed.

For example let us consider the block 'Data Block for d22 -> e3'this represents the three dense dimensions from within the combination of the standard members' d22 and e3. The member combinations are denoted by the cross-dimensional operator. The symbol for the cross-dimensional operator is ->, so d22 -> e3 denotes the block for d22 and e3. The intersection of A, b21, and c3 are written as A -> b21 -> c3.



In the diagram for Data block d22 -> e3 we can see that the multidimensional system has created creates a data block for every unique combination of the members of the sparse dimensions D and E (providing that at least one data value exists for the combination). Therefore by the index we can locate the data block and then look within to find the cell value for the data at the intersection of A, b21, and c3 is written as A -> b21 -> c3.

Manually when we are visualizing cubes this is how we would typically navigate around them using the index for the data block and using the cross-dimensional operator to arrive at the intersection of the dimensions which is the cell value we are looking for.

Slicing and Dicing and Multiple Data Views

Cubes allow us to manipulate and present data in a variety of ways through their inherent multidimensional structure. For instance we could slice only a month we are interested in out of the Time Q1 Dimension, as we can see in the diagram below where we slice February sales figures, but we could just as easily have sliced away January or March.



What is more we can dice the cube as well to isolate only the portion of data of interest such as if we only wish to look at the Profit Margin as shown below.



Designing a Multidimensional Model

Having been introduced to working with cubes and multidimensional data models in the previous sections it is not time to look at how we can design our own. As always the driver will be a real business case or problem we need to address as cubes are designed for a purpose they are not arbitrarily constructed instead they will contain the dimensions and measures we require, no less and no more.

The first step is to identify and analyze the business case. In this example we will use a common business scenario whereby a retail company wishes to improve its profitability and increase its capacity for market analysis.

Therefore, we need to decompose the business case into high-level business goals that the project must meet. For example:

The strategic goal is to increase company profits by increasing sales of higher margin products and by increasing sales volume overall.

The tactical approach will be for the Sales and Marketing department to:

Analyze industry trends and target specific market segments

Analyze sales channels and increase profits

Identify product trends and create a strategy for developing the appropriate channels

Once we have established the strategic business goals and agreed upon the best tactical approach the next stage is to gain further information from the subject matter experts and project stakeholders as to what information we will need to collect in order to achieve the business goals. This interestingly, takes us back to the opening chapters where we saw the relationship between data, information and knowledge, and here we can see that the model isn't perfect. For example, in order to understand what information we need to obtain, requires the higher level knowledge from SMEs to decompose the problem and using their knowledge identify the information we require. We gain this information by asking questions of the SMEs such as:

What is the percent of total sales for any item, product family, or product class in any month, quarter or year, and in any distribution channel?

What are the unit price, unit cost, and margin for each unit for any item in any particular month?

What items contributed the most to total profitability in any month, quarter, or year, in any distribution channel, and in any geographic area or market segment?

The answer to these questions amongst many others will enable us to determine the information that we require to build the Dimensions. In our common scenario of a desire to increase and track profitability and market trends we can identify the following key business areas for analysis:

The company must understand profitability by product, account, market segment, and distribution channel. It also needs to understand profitability trends.

The company has a need for ad hoc sales analysis. Analysis must identify what products are sold to whom, when these products are sold, and how customers buy these products.

The ability to perform trend analysis is important to predict profitability against time over calendar months and the financial year.

Now that we have identified the key analysis requirements, we can decompose them to determine the essential components – the business Facts. These may be many at this stage, but what we are looking for is not individual Facts as such but what category they come under, for example, the following business facts are just some that we might require:

(These facts are ordered by time, product, customer shipment or market segment, and distribution channel)

Sales

Units

Change in sales from prior period

Percent change in sales from prior period

Change in sales from prior year

Percent change in sales from prior year

Product share

Channel share

Market share

Extended cost

Extended margin

Extended margin change from prior period

Extended margin percent change from prior period

Units sold, change from prior period

Units sold, percent change from prior period

Units sold, change from prior year

Units sold, percent change from prior year

These facts are ordered by item and month:

Unit price

Unit cost

Margin per unit

What we can see is that although this is a lengthy list the business Facts fall into just a few categories, Product, Customer, Channel and Time.

Therefore, in this rather simplistic case we can break down the analysis requirements into four dimensions:

Product - shows how data varies by product.

Customer - shows how data varies by customer or geographic area.

Channel - shows how data varies according to each distribution channel.

Time - shows how data varies over time.

Now that we have identified the dimensions we can turn our attention to identify the levels of summarization within each dimension. Again the level of summarization will be driven by the original business goals, and from our business case analysis we can determine that for the analysis to serve the company we require:

Three distribution channels: Stores, Catalogue and Internet. These three values are the lowest level of detail in the data warehouse and will be grouped in the Channel level. From the order of highest level of summarization to the lowest level of detail, levels will be Total and Channel.

Because two of the channel categories are in-direct sales so will require delivery, Shipments and Segment will be two hierarchies in the Customer dimension. In each case, the lowest level of detail in the data model is the 'Ship To' location.

When analyzing along the line of customer shipments, the levels of summarization will be (highest to lowest): Total, Region, Warehouse, and Ship To.

When analyzing by market segmentation, the levels of summarization will be (highest to lowest): Total, Market Segment, Account, and Ship To.

The Product dimension will have four levels (highest to lowest): Total, Class, Family, and Item.

The Time dimension will have four levels (highest to lowest): Total, Year, Quarter, and Month.

The next stage is to identify the hierarchies that organize the levels within each dimension. To identify hierarchies, we will group the levels in the correct order of summarization and in a way that supports the identified types of analysis.

For the Channel and Product dimensions, we require only one hierarchy for each dimension. For the Customer dimension, we will require two hierarchies. Analysis within the Customer dimension tends to be either by geographic area or market segment. Therefore, we will organize levels into two hierarchies, Shipments and Segment. Analysis over time also requires two hierarchies, a Calendar hierarchy and a Fiscal hierarchy.

Finally we are required to identify the stored measures, the actual facts that we wish to store in the cells of the cube. Even though earlier we identified many more required business facts, the number can be greatly reduced due to the fact that all of them can be deduced from just these four:

Units

Sales

Unit Price

Unit Cost

Data Warehousing Physical Design

Moving from Logical to Physical Design requires putting all the logical and theoretical planning effort into practice. Unfortunately, how you start to build out your warehouse database is dependent on a multitude of things and not just the choice of technology or the data schema. Consequently when you decide to transition from the logical design phase into the physical build phase you should be working to a vendors plan or implementation guide. For example if you have opted for Oracle or Microsoft they will guide you through the perils of the build cycle. In this section all that we can do is spell out the initial requirements for transition and explain some off the concepts and terms you will need to know when battling through the vendor's implementation guides.

Logical design is what you plan out in your head and draw up with a pen and paper when you first start dreaming about building your data warehouse. Physical design requires the real world transition from theory and logic into hard coding and dealing with constraints, but the first step is typically to start transforming that logical data model into a real database.

During the physical design process, you convert all the best laid plans, concepts and the data gathered during the logical design phase into a physical database structure. During the logical design phase, you defined a model for your data warehouse consisting of entities, attributes, and relationships. The entities are linked together using relationships. Attributes are used to describe the entities and the unique identifier (UID) distinguishes between one instance of an entity and another. During the logical design phase it was the relationships that were the drivers of the design, but in Physical design decisions are mainly driven by query performance and database maintenance aspects.

Logical Design Compared with Physical Design

Description of "Logical Design Compared with Physical Design"

During the initial physical design process as we can see from the figure above we will need to translate the logical descriptions of the schemas into actual database structures. At this time, we must map:

Entities to tables

Relationships to foreign key constraints

Attributes to columns

Primary unique identifiers to primary key constraints

Unique identifiers to unique key constraints

Physical Design Structures

In order to convert our logical design into a physical design, we are going to have to create some or all of the following structures: tablespaces, tables, partitions on tables or index-organized tables, indexes, views, integrity constraints, materialized views, and dimensions.

Tablespaces in Data Warehouses

A tablespace consists of one or more datafiles, which are physical structures within the operating system you are using. A datafile is associated with only one tablespace. From a design perspective, we can look upon tablespaces as being containers for physical design structures.

Tablespaces need to be separated by differences. For example, tables should be separated from their indexes and small tables should be separated from large tables. Tablespaces should also represent logical business units if possible. Because a tablespace is the coarsest granularity for backup and recovery or the transportable tablespaces mechanism, the logical business design affects availability and maintenance operations.

Partitioning in Data Warehouses

Partitioning is an extremely important functionality for data warehousing, improving manageability, performance and availability. Partitioning allows tables, indexes or index-organized tables to be subdivided into smaller pieces. Each piece of the database object is called a partition. Each partition has its own name, and may optionally have its own storage characteristics. From the perspective of a database administrator, a partitioned object has multiple pieces that can be managed either collectively or individually. This gives the administrator considerable flexibility in managing a partitioned object. However, from the perspective of the user, a partitioned table is identical to a non-partitioned table; no modifications are necessary when accessing a partitioned table using SQL DML commands.

Database objects - tables, indexes, and index-organized tables - are partitioned using a partitioning key, a set of columns that determine in which partition a given row will reside. For example a sales table partitioned on sales date, using a monthly partitioning strategy; the table appears to any application as a single, normal table. However, the DBA can manage and store each monthly partition individually, potentially using different storage tiers, applying table compression to the older data, or store complete ranges of older data in read only tablespaces.

Materialized Views in Data Warehouses

Materialized views are query results that have been stored in advance so long-running calculations are not necessary when you actually execute your SQL statements. From a physical design point of view, materialized views resemble tables or partitioned tables and behave like indexes in that they are used transparently and improve performance.

Overview of ETL in Data Warehouses

Now that we have looked into at some depth the data structure of the proposed data warehouse we now need to consider how we load data. After all, the data warehouse will require being regularly fed with new operational data in order that it can serve its purpose of facilitating business analysis. To do this, data from one or more operational systems must be extracted and copied into the data warehouse. The challenge in data warehouse environments is to integrate, rearrange and consolidate large volumes of data over many systems, thereby providing a new unified information base for business intelligence.

The process of extracting data from source systems and bringing it into the data warehouse is commonly called ETL, which stands for extraction, transformation, and loading. Note that ETL refers to a broad process, and not three well-defined steps. The acronym ETL is perhaps too simplistic, because it omits the transportation phase and implies that each of the other phases of the process is distinct. Nevertheless, the entire process is known as ETL.

The methodology and tasks of ETL have been well known for many years, and are not necessarily unique to data warehouse environments: a wide variety of proprietary applications and database systems are the IT backbone of any enterprise. Data has to be shared between applications or systems, trying to integrate them, giving at least two applications the same picture of the world. This data sharing was mostly addressed by mechanisms similar to what is now called ETL.

ETL Basics in Data Warehousing

What happens during the ETL process? The following tasks are the main actions in the process:

Extraction of Data in Data Warehouses

Transportation of Data in Data Warehouses

Extraction of Data in Data Warehouses

During extraction, the desired data is identified and extracted from many different sources, including database systems and applications. Very often, it is not possible to identify the specific subset of interest therefore more data than necessary has to be extracted, so the identification of the relevant data will be done at a later point in time. Depending on the source system's capabilities (for example, operating system resources), some transformations may take place during this extraction process. The size of the extracted data varies from hundreds of kilobytes up to gigabytes, depending on the source system and the business situation. The same is true for the time delta between two (logically) identical extractions: the time span may vary between days/hours and minutes to near real-time. Web server log files, for example, can easily grow to hundreds of megabytes in a very short period.

BI for Data Warehouse for SMB (Retail, Logistics or with Branch Offices)

When we consider the issues of extracting, transforming and loading information from disparate data silos within the context of the SMB we have to remember that it is such a diverse landscape with a myriad of proprietary, open source or homemade applications. There are also some cases where the Data Analytics for some small organizations may well involve creating highly formatted and distributable reports. Examples of these reports include sales order detail reports from POS, inventory-on-hand reports, and detailed up-to-the minute status reports. Some of the data is traditional quantitative and readily obtainable from the relational database, i.e. for the POS and the inventory-on-hand report. However, many POS systems will have their own embedded database schemas so we will still need to translate the data before loading it into a central reporting system. These business requirements far exceed the capabilities of the self-service Excel-BI model that we described in an earlier chapter. Therefore, for SMB businesses with more complex business requirements, such as distributed retail, we will need to design a database structure that alleviates many of the access issues and much of the complexity in collecting, processing, reporting and analysis of the traditional structured data.

Some key characteristics of SMB organizations with enhanced business requirements are:

Data used to create reports comes from corporate sanctioned and IT managed data sources.

Reports may have to be built from data from dissimilar and incompatible systems such as POS in retail or IoT devices in logistics.

Reports are highly formatted and frequently printed.

Reports may be complex and require special technical skills such as advanced SQL, MDX, or other query languages to collect, merge and build.

Reports are often created with one or more user selectable parameters, but users are not capable of extensive interaction.

Reports are usually authored by IT or BI developers, often because the complexity of the reports exceeds the capabilities of the user base for self-service reporting.

The reports are refreshed on a regular basis and available on-demand.

Reports may be delivered in multiple formats such as PDF, Excel, HTML, and so on.

Reports may be shared broadly across the organization.

Consequently after viewing the business requirements of our more complex or more business constrained SMB we can readily observe that the self-service model will not suffice. However the design goal is not to replace the self-service Excel-BI model but to enhance it through adding functionality, robustness and the capability to support a wide variety of data types, such as structured, semi and unstructured data for a variety of sources both conventional and contemporary such as POS and the multitude of IoT devices.

In order to achieve this requires that we maintain our basic model of excel with its BI add-ons and the Azure Cloud based BI.com integration for self-service analysis and visualization as we must not lose the self-sufficiency it provides while providing a more robust and flexible on-premises based features and services.

Tool Options

The following tools support the larger SMB BI infrastructure:

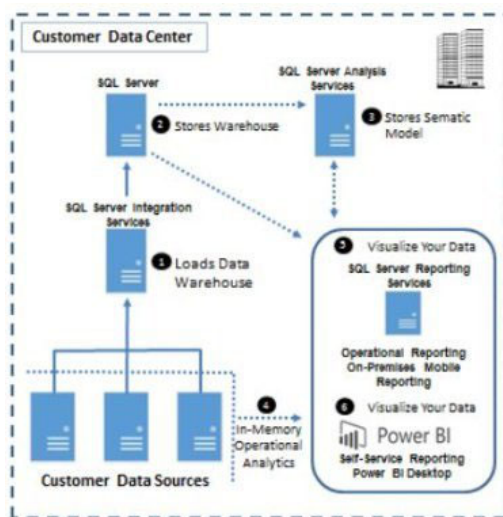
SQL Server Analysis Services

SQL Server Reporting Services

Power View for SharePoint 2013

SharePoint 2013 Dashboards

PerformancePoint Services



Microsoft SQL Server Analysis Services

The basis of any Analysis Services solution is to provide a business intelligence data model on a server that instantiates, processes, queries, and manages objects.

Models are built on the historical operations data that has been collecting in transactional databases and other data stores. When this data is annotated with metadata it lets you measure, manipulate, and compare business data in ad hoc queries or custom reports. After a model is designed, it can then be deployed to an Analysis Services server as a database, where it becomes available to other authorized users who connect to it through Excel and other tools.

SQL Server 2016 Analysis Services

In the architecture above we use SQL Server 2016 as the data warehouse and as the staging area which will extract, load and transform data. MS SQL is ideal for our purpose as it is tightly coupled with the business user tools that most SMB use, such as Excel. Many new DAX functions have been added to the latest version of SQL server, for example, a graphical user interface has been added to SQL Server Management Studio to configure and manage Analysis Services Extended Events.

In addition, SQL Server Data Tools (SSDT) for Visual Studio 2015 now includes project templates for Analysis Services solutions.

Microsoft SQL Server Reporting Services

Another strong reason for using Microsoft SQL Server Reporting Services in our SMB design is that it enables IT to create and manage both static and parameterized reports, which provides a platform for delivering unified information throughout the organization.

Advanced Data Visualization

Furthermore, when using a single-vendor architecture rather than a ‘best of breed’ mash up means that we can benefit from tightly coupled reporting and analysis tools. For example, when the user accesses the reports via a portal, they will be able to interact and perform analysis through the use of parameters and filters embedded in the reports. With the use of parameters, drill-down, and drill-through capabilities, you are able to perform more guided reporting and analysis. This level of integration is not always possible when using third party reporting and analysis tools. Additionally, reports can be linked together to allow for more in-depth analysis via drill-through capabilities.

Further, having integrated reporting and analysis allows for collaboration as the report can be scheduled and distributed using different delivery mechanisms and in different generation formats. For example, the reports may be delivered via portal, file share, email, or sent directly to a printer, and may be generated as PDF, Excel, XML, comma delimited text file, TIFF image, HTML or Microsoft Word formats.

SQL Server 2016, uses an HTML5 rendering engine, which supports modern web "full" standards mode and modern browsers so you can now export reports in the PPTX format from Report Builder, Report Designer (in SSDT), and Report Manager.

SQL Server 2016 also supports Power BI so you can pin Reporting Services report items from the report viewer toolbar to a Power BI dashboard as a new tile or you can create Reporting Services reports using Report Builder and Report Designer in SQL Server Data Tools.

Report Builder

Another advantage of using the SMB architecture above built on a MS platforms is that you can use the integrates tools such as Report Builder to develop highly formatted reports using an Excel-like ribbon. Report Builder doesn't just do formatting it also provides advanced visualization options such as geospatial mapping, sparklines, and gauges.

However, although you don't necessarily need to know how to write SQL queries to use Report Builder, it does require a more advanced skills and practice than using Excel or PowerPivot.

Sequel Server Data tools (SSDT)

An additional benefit that comes via a free download id Sequel Server Data tools, with this tool you can integrate with Report Designer, so you can open, modify, preview, save, and deploy report definitions, shared data sources, shared datasets, and report parts.

When working with SSDT and Reporting Services, you can use the following project templates :

Report Server Project. Which is a Business Intelligence Project template installed by Visual Studio.

Report Server Project Wizard. Using this inbuilt wizard, you can create business reports by creating a connection string to a data source, as well as setting data source credentials, designing a query, adding a table, and specifying report data and groups, and then publish the report to a report server.

Power View for SharePoint

Another component of the Microsoft BI suite of tools is Power View, which is a browser-based application launched from SharePoint Server 2013 that enables you to present and share insights with others in the organization through interactive presentations.

Using this tool you can create a single report with multiple views. All the views in one report are based on the same Data Model.

Power View reports in SharePoint Server are created and stored as RDLX files. However, it is important to realise that Power View sheets in Excel are part of the Excel XLSX workbook therefore you can't open a Power View RDLX file in Excel, or open an Excel XLSX file with Power View sheets in Power View in SharePoint.

SharePoint 2013 Dashboards

A SharePoint server is quite common in Microsoft houses and if you have one already you can use it to produce a dashboard. A Dashboard is a handy presentation and collaborative tool that combines related groups of interactive scorecards (KPI) and report views that are organized together in a SharePoint or Web-hosted site. By using the facility in SharePoint Server 2013 it enables you to create and use dashboards that provide up-to-date information in a centrally managed, easily accessed location.

When we consider why we would want to use a dashboard report, it helps business decision makers understand the current health of their business. A dashboard often combines key performance indicators (KPI) from various business functions on a single page to help provide an at-a-glance view of current performance. Dashboards provide summary information at the strategic or tactical level but also the ability to drill down further when further detail is required.

The following are key characteristics of performance monitoring dashboards.

Provides an up-to-date view of business performance.

Provides a holistic view of the business performance by combining multiple KPI together.

Data can be presented in multiple formats depending on which is easier to consume for example a dashboard might include a tabular report with spark-lines, along with trended graphs or bar charts, and geospatial maps or scorecards.

Dashboards are typically designed to display summary data but they enable drill down to perform root cause analysis for data anomalies.

Dashboards are often deployed across the organization to assist in information dissemination.

PerformancePoint Services

Another component of com Microsoft SharePoint Server 2013 that you may well find beneficial is the module called the PerformancePoint Service; this is a performance management service that is used to create dashboards and scorecards.

PerformancePoint Services provides a method for designing and building both balanced and free-form scorecards. The balanced scorecards are used to focus on finding a balanced set of metrics across multiple perspectives, i.e. balancing the financial indicators with leading indicators that better predict future performance. A major benefit of scorecards is the ability to aggregate and display non-like data into unified and summarized scores.

The PerformancePoint Services component provides the ability to pull multiple view types into a single dashboard. In a PerformancePoint Services dashboard, you can interact with all types of reporting data such as the SSRS reports, cube-based graphs, performance maps, decomposition trees and Visio diagrams. There are also many other content types that can be integrated into reports.

For example, with cube-based graphs and charts, you can perform ad-hoc analysis, slice and dice dimensional data, navigate through hierarchies and pivot and change chart types. Furthermore, with Dashboards you can pull in data from a variety of diverse sources, providing the capability to develop and manage the presentation of reports through one tool.

PerformancePoint Services allows companies to set up hierarchical KPIs with relative weightings that align with perspectives and objectives. Targets can be defined at each intersection and ranges can be set for defining Red/Yellow/Green status thresholds for each KPI. Scorecards can also be pivoted and filtered by a number of dimensions, allowing you to perform root cause analysis on KPIs that are not meeting the targets.

Typical SMB Data Warehouse Infrastructure

For corporate BI, a data warehouse, data mart and/or online-analytical-processing (OLAP) environment is often in place and used as the data source. The existence of OLAP cubes forms the foundation of BI analysis in these environments. Data is typically refreshed regularly on schedule to provide the most current information possible. For these reasons, this will almost always require that an IT department is available to support the infrastructure for BI.

The IT department is in charge of creating and managing the corporate data sources. The IT department should be experienced dealing with BI structures and reporting technologies, and will be heavily involved in report

development and management including having a firm understanding of how to use the reporting tools and query languages such as SQL and MDX.

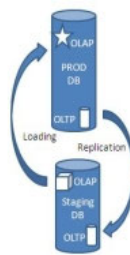
However, what if you do not have Microsoft SQL Server, as many SMB do not, they may well be running MySQL or PostGRES in which case we can either build our own, which is challenging, but if you have to time and knowledge it can be satisfying and you can save considerably on the cost of licensing. However, it is not a perfect solution as we will demonstrate as MYSQL just wasn't designed and optimized to handle large data sets. However as an SMB if we recognize these constraints and keep the data volume to realistic levels we should have no issues.

A DIY Approach: MySQL as a Data Warehouse

The first phase of the project will be to decide upon the physical architecture; ideally if we are building a production system we should adhere to best practices and build a redundant system with a production and a staging server. This design will provide redundancy and separate the processes of ETL and OLTP (transaction processing). Within the design we will have to create two instances of MySQL on each server, one for OLAP and the other primary instance for the OLTP database. If we consider the diagram below we can see how the two servers, the production and staging will replicate their database contents ensuring that they remain synchronized.

As we can see from the MySQL architecture the idea is to isolate production and staging, and we will accomplish this by separation of duties. For example in the production server all OLTP – transactions – happens there and it therefore servers as the master for replication to the staging server for all OLTP data. The production server however does support a second OLAP schema which is used to load data from the staging server.

The Staging MySQL server as we saw acts as a data replication slave for OLTP data but is responsible for the ETL load process where it will map surrogate keys between dimensions (OLTP) and the facts tables, as well as load data into the Fact table and generate aggregate tables for frequent access. In addition the Staging server will be responsible for the final upload of the aggregation tables into the production OLAP database.



The next issue we will have to address is the design of the OLAP data schema, i.e. will it utilize a 3NF relational, Star, Snowflake or combination schema.

We will need to take the MYSQL configuration into consideration and so in order to manage the OLAP tasks we will have to adjust some of the tunable parameters:

Use InnoDB on a Staging tables with upsert + on duplicate update

On the Production DB use MyISAM on read-only tables for faster loads

Use partitions for easy purging

Highly recommended to keep small dataset size (< 500G per server)

Build OLAP cubes (Pentaho Mondrian or any multidimensional reporting solution)

Compression, sequential IO and read-ahead is the key

As MySQL is optimized for OLTP so it is sub-optimal for OLAP but that isn't a problem if we can keep the data sets down to less than 500GB and pre-aggregate the data as much as is feasible. If that is too restrictive we will need to scale outwards horizontally by adding more production/staging server pairs and distributing the workload.

This design is not difficult to build technically using MySQL and Pentaho. Often the most daunting part is the replication but with MySQL that is straightforward and easily implemented. The benefits are that we will get a very cost effective data warehouse solution that will handle small data sets, real time analytics, and standard reporting on historical data. The design will also provide the much needed single source of truth in the business. For small data sets, we can use three optimization tips such as, use page cache, buffer pools and query caching. For advanced reporting and analytics it will be best to use a Pentaho cube for multidimensional modeling on the Staging server as most BI solution use in-memory OLAP processing.

However unlike the MS SQL Data Warehouse design it will not have built in reporting, visualization or analytical tools or handle medium sized data sets, though you can connect and use any open source tools for all your BI requirements.

As we have seen The MySQL data warehouse solution is a good fit for those with sufficient MySQL skills and for projects with small data sets. For SMBs that do not have the time or the resources to build an open source solution there is another alternative that is easy to implement and easy on the budget – the cloud.

Amazon AWS Data Warehouse

A possible solution too many of the constraints with SMB data analytics can perhaps be solved by simply outsourcing the infrastructure and administration burden to the cloud. In this way the SMB offloads much of the financial and technical burden and just about all of the administration of the data warehouse. In return for a monthly usage fee the SMB can obtain a system that is vastly scalable – on-demand – and hugely robust with performance and capacities far out with their wildest dreams or needs.

In this section we briefly consider coupling the Excel-BI self-service model we discussed in the early chapters but this time we will integrate with a cloud service rather than an on-premises MS SQL Warehouse. The service we will consider is the Amazon Web Services (AWS) data warehouse cloud solution called Redshift, which is a fast, fully managed, petabyte-scale cloud based data warehouse that makes it simple and cost-effective to analyze all of your data using your existing business intelligence tools. Unfortunately, like our MySQL design we will still need use our own analysis and reporting tools. However in the solution we propose that isn't an issue as we

can use the Pentaho ETL for connecting and transforming our data and Pentaho for the reporting, visualisation and analysis. Should you wish to use any other reporting, visualisation or ETL tools then they are easily substituted and plugged in as Redshift uses standard ODBC and JDBC connectors.

To give you a rough ball park figure for prices, Amazon has a limited free trial but for production systems you will be paying around \$0.25 per hour for a small warehouse with no commitments and can scale to petabytes for \$1,000 per terabyte per year, and this they suggest is less than a tenth the cost of traditional solutions.

In addition Amazon Redshift also includes Redshift Spectrum, which allows you to directly run SQL queries against exabytes of unstructured data in Amazon S3. No loading or transformation is required, and you can use open data formats, including CSV, TSV, Parquet, Sequence, and RCFile. Furthermore, connecting to Amazon Redshift and having it find and connect to your data sources through AWS Data Pipeline or Pentaho ETL or through Amazon APIs directly is relatively straightforward and so is connecting the reporting and analytical tools as those connections are all done via the user dashboard or through APIs.

The reference Architecture for AWS Redshift Data Warehouse is simplicity itself:



Where the cloud AWS components we will utilise consist of the Amazon S3 object storage database for easy connection to data sources on the web. Then we have the Amazon Redshift Data warehouse, which is loaded via the connection with Amazon S3 for tight integration within the cloud. As a reporting/Analysis tool that directly connect to the Redshift Warehouse we have the integrated Amazon visualisation and reporting tool Amazon QuickSight which we will take a deeper look at now.

Amazon QuickSight

Although you can just as easily use just about any BI, Reporting or Visualisation tools with Amazon Redshift and S3 Amazon do have their own BI analysis tool called Amazon QuickSight.

The advantages of using Amazons own tightly integrate tools rather than your own perhaps ‘best of breed’ open source tools is that it makes plumbing everything together extremely straightforward. Trying to connect several open source tools together can often be a very time consuming and often futile exercise and that is why we in this book where possible try to limit the amount of mix n’ match. By using Amazon products you can cut the time spend plumbing, testing and verifying the infrastructure to a couple of hours – and most of that will be simply finding your way around the configuration panels. Therefore, to begin with at least we suggest that you utilise Amazon QuickSight for the analysis, visualisation and reporting. You can always go back later and connect up your favourite tool once you have everything up and working.

Amazon QuickSight is a fast, cloud-powered business analytics service that makes it easy to build visualizations, perform ad-hoc analysis, and quickly get business insights from your data. Using Amazons tightly coupled cloud-based service you can easily connect to your data, perform advanced analysis, and create an array of impressive visualizations such as scorecards, dashboards and interactive reports that can be accessed from any browser or mobile device.

As we now will have a good grounding in the types of data and the selection of technology available to us let us revisit why we want to do Data Analytics in the first place?

Chapter 13–Advanced Data Analysis & Business Intelligence

Forrester Research Definition: “Business Intelligence is a set of methodologies, processes, architectures, and technologies that transform raw data into meaningful and useful information used to enable more effective strategic, tactical, and operational insights and decision-making.”

Business intelligence (BI) is the discipline of creating value for an organization based on facts that we can derive from data. Of course BI has been around for a long time and executives may be quite ambivalent as to its worth as for them gut feelings and hard-nosed business experience trumps most technology predictions. In some cases, executives will openly embrace BI as it provides them with the backup and support for their hypothesis and so long as the results continue to be supportive all is well.

BI was coined by Gartner (Dresner, 1989) as an umbrella term to describe the set of concepts and methods used to improve business decision-making by using fact-based support systems. As a brief step through BI's long history it was around in the 1980's as Executive Information Systems (EIS), in the 1990's, OLAP, followed by scorecards, dashboards, KPIs (key performance indicators), and real time alerts through business activity monitoring. In 2000, BI manifested itself once again through Data mining of Data Warehouses, and since 2010 it has evolved into Big Data, using advanced analytics, AI algorithms and Machine Learning. BI therefore has a long resume with several aliases and has been at the core of business management for decades.

However, no matter how long BI has been around its purpose has remained the same in so much as Business intelligence is not necessarily about tools and technologies; rather it is strategies of combining data from various sources with methodologies that derives value in the form of information and business insights in a cohesive manner. The problem is though that many of the data sources have now changed to being digital channels which we now need to access. These sources could be web based social media sites, forums or IoT sensor information. Also, these new channels often produce unstructured data such as website click-data, email, text from social media comments, messenger text, and even photographs and video, which present the analysts with issues.

BI is about analyzing an organization's performance using data with the purpose to improve its revenue, profits and competitiveness. BI uses a combination of historical information about past transactions or events and reference data about, for example, customers or products, to enable a wide variety of analyses and decision support techniques.

Thus, BI is something organizations, even SMBs, have always been involved with on some strategic level using a combination of historical data, market experience and knowledge and gut instinct. But SMB organizations today are doing things a bit more scientifically – or rather that's what they think – by collecting ever increasing volumes of data for the vague purpose of analytics. However, collecting data should have a purpose, an end product in mind and not just be an arbitrary pastime. If not we end up with the collection of data being the project instead of a task within a project. What's more we will end up with a situation where it is no longer possible to analyze such quantities of raw data –even if we could find a purpose - without specialized tools and methods. Therefore it is essential to always keep the focus on the purpose and SMBs especially should have a clear business case for performing business intelligence beyond standard reporting and trend analysis.

The reason for this is not only can the complexity and hence the expense escalate rapidly once we start to delve into higher levels of data analytics augmented by visual analytical tools, for example using machine learning and complex algorithms and models that users interact with via visual presentation tools, but so does the understandability, an importantly the capability to interpret the results. Consequently, we must never lose track of the purpose of the BI analytics and always ensure that the goals mirror or are tightly aligned with the business objectives so as to justify their existence as well as to increase the potential of ROI (Return on Investment).

The typical BI or DA output that comes from a general analytical project is best described as being represented through:

- Reports: Standard, preformatted templates for ‘what happened’ style of analysis.
- User-defined analyses: This is a self-service form of limited ad-hoc query but with limited input criteria for example the user can select input criteria from a “pick-lists” to filter (select) the information they wish to analyze, such as sales for a selected product, region, during a selected month or quarter.
- Ad hoc analyses: This form requires that power users create their own queries to extract self-selected pre-staged information and then use the information to perform a user enabled analysis. This is the most difficult to implement as it requires users who are knowledgeable in both business and database technology as well as an intuitive database schema or pre-processed OLAP cubes.
- Scorecards and dashboards: These predefined visual interactive reports provide up to date business performance metrics regarding key performance indicators. The choice of variables to be displayed can be selected and determined as to whether the scorecard/Dashboard is for strategic, tactical or operational purpose. This type of continuous visual presentation especially if used in large TV screen wallboards is highly effective ways to disseminate important KPI to the organization.
- Multidimensional analysis: Provides the flexible tool-based user-defined analysis of business performance and the underlying drivers or root causes of that performance.
- Alerts & Notifications: Predefined analyses of key business performance variables, comparison to a performance standard or range, and communication to designated business people when performance is outside the predefined performance standard or range.
- Advanced analytics: This is best described as being the application of statistical and/or Operational knowledge and research to historical business information. The idea is to apply knowledge or a hypothesis to large scale historical data in such a way as to characterize a relevant aspect of business performance, thereby proving or disproving the theory.
- Predictive analytics: This is one of the easier business cases to sell as it can have a real impact on operational efficiency and therefore a short or valuable ROI. The principle is to monitor the application of long-established statistical and/or operations data in order to predict an event. In industry predictive analysis is heavily used in predicting maintenance on machinery.

Research methods to historical business information to predict, model, or simulate future business and/or economic performance and potentially prescribe a favoured course of action for the future.

From the preceding list of different approaches to BI we can see that it is primarily about the analysis of data but for diverse purposes. The operational role in a BI strategy is traditionally to collect, monitor/process and store historical operational data and then display it for the subject matter experts in a format they can make use of to make important inferences. Historical reporting by its nature makes the pivoting of any combination of dimensions or attributes within a report or table possible. This is done by “slicing and dicing” the dimensions in the tables or report in order to focus only on the relevant data. An important part of Data Analytics is in presenting data in a format that the experts can use, which requires presenting results in varying levels of granularity and aggregation. Today a plethora of On Line Analytic Processing (OLAP) type data structures provide the ability to drill in, around and through data structures such as multidimensional cubes in order to make sense of the data presented.

While OLAP presents a multi-dimensional highly structured interface to the data the object still remains to dig out interesting details of value. In other words, the technology is less important than the ability to synthesize the data into meaningful bits of information. What is more, the driver for change often comes from the executives’ desire to move away from static dashboards and to provide a method to query and analyze the data themselves. Hence, a self-service method of data query has evolved and it comes in a more natural human based language

such as in the way we interact with Siri, Alexa and Google personal assistants in the home with “Show me what’s wrong” or “Highlight the facts for me,” and this was the impetus behind the interactive dashboard and scorecards in today’s executive toolkit.

The limitations are that reporting on the past can only show what has happened, and provide some predictions through trend analysis but it cannot predict the future. For that we must combine historical information with real-time data and then using layered predictive analytics so that we can glean the probability or the likelihood of an event happening and we need that in order to have true foreknowledge. This type of analytics comes through data mining, forecasting and this is where other predictive analytics especially those advanced ML algorithms play an important role.

Consequently, the types of data analytics we are now interested in have expanded to include:

Prescriptive analytics is really valuable, but largely not used. According to Gartner, 13 percent of organizations are using predictive but only 3 percent are using prescriptive analytics. Where big data analytics in general sheds light on a subject, prescriptive analytics provides a focus to answer specific questions. For example, in the Insurance industry, it is possible to manage the customer base by using prescriptive analytics to measure the number of drivers who are past claimants, then to add filters for factors like gender, age and even post code to determine the price of a policy, thereby benefiting careful drivers over the unlucky ones that have failed to avoid accidents. However this is categorizing individuals and also encouraging stereotyping. The same prescriptive model is applied in almost all industry to identify desirable or problematic social groups.

Predictive analytics uses big data to identify past patterns to predict the future. For example, companies may well use predictive analytics in generating sales lead scoring. Many SMB take this approach as predictive analytics performed over the entire sales process such as when analyzing lead source, number of communications, types of communications, social media channels, documents, CRM data, etc. can determine the most valuable low cost customers.

Diagnostic analytics are typically used for as the name suggests discovering why something happened. For example, for a social media marketing campaign, you can use descriptive analytics to assess the number of product reviews, followers, page views, etc. This is because thousands of online mentions can be distilled into a single dashboard that visualizes what marketing techniques work and what didn’t.

Descriptive analytics or data mining are often considered to be at the bottom of the big data value chain, but they can still be valuable for uncovering patterns that offer insight. A simple example of descriptive analytics would be assessing credit risk; using past financial performance to predict a customer’s likely financial performance. Descriptive analytics can be useful in the sales cycle, for example, to categorize customers by their likely product preferences and sales history.

Today’s Business Intelligence tools allow business users to interrogate huge amounts of data in a matter of seconds. That alone is not a result in itself or just down to technology as fast response times are not simply the result of modern hardware and software often the way the data is framed or presented has a greater overall effect on performance. This of course is more the result of operational knowledge and detailed business process planning and they in turn are the product of subject matter expertise.

Of course this may sound like BI is out with the budget or wherewithal of SMB but that is not true. Indeed it is the business and operational knowledge that is vital in knowing what questions to ask of the data and then

interpreting the results. In most cases subject matter experts will often be the only ones capable of interpreting incidents of correlation and causation by the application of their technical expertise, experience and common sense. Therefore, SMBs should not be put off simply because they do not have a team of data scientists, what they do have is experts in the relevant fields of business and operations and it will be these people who eventually will glean the value from the results. Hence, data experts and technology are not prerequisites though they are both very helpful to have.

But of course some SMBs need the aid of technology to provide the stimulus to overcome inertia and finally endeavor on a BI project more than others. The factors that drive organizations into actual business intelligence efforts are typically split into external and internal factors. External factors are forces and circumstances outside the organization's control, e.g. competitors, customers, environment and government. Internal factors are circumstances and processes that are under the organization's control such as procurement, logistics, sales, employees etc.

External factors driving organizations towards BI

Competitive pressure: Some companies are under intense competitive pressure and are operating in a crowded price-sensitive market. This applies especially to products and services that have high volume and low barriers for entry. In the mobile telecom market, mobile apps and accessory have just those attributes in that they are high volume, low price and the cost of entry into the market is minimal. On the other hand Mobile Telecom operators have extremely high barriers for entry, very few competitors and deal in high volumes of subscribers. Therefore the focus for the Mobil Operator is to maximize subscriber numbers and minimize churn, which is the loss of a subscriber to a competitor. Therefore the mobile operators marketing and BI is aimed at seducing customers from competitors as in most countries it is fairly easy to change operator.

Market volatility: In markets, as the mobile telecom example, with quickly changing customer demands and/or quickly changing services or goods (e.g. computers and accessories) it is extremely important to be agile and ahead of the situation, always have the right services or products in the right quantities at the right prices.

Legal requirements: Some companies, such as in the financial or health care industries are under restrictions by law to be able to deliver specific reports to external entities to demonstrate compliance with regulations. Especially well-known is the Sarbanes-Oxley regulations that governs large companies in USA and GDPR, which applies to the companies that process EU citizens personal data. These regulations and the requirement for compliance is a major driver for executives to demand having robust data management and a well-defined report delivery system.

Internal factors driving organizations towards BI

Volumes of data: The sheer volumes of data that an organization collects today can be crippling and although you might think that it would make them more likely to establish formalized systems for extracting this data and utilizing it doesn't appear to. Instead there are plenty of cases where the sheer volume of data makes it difficult to keep a detailed understanding of the dynamics of the organization's processes and why it is being collected in the first place. Consequently as the data volume accumulates advanced tools are needed to analyze it. Respecting the volumes of data that will be collected is essential in BI as you will have at some stage to aggregate, dive into and analyze that data in order to maintain the purpose of the project.

Single version of the truth: In SMBs too often, as we have no doubt witnessed with interdepartmental squabbles over spreadsheets, reports that supposedly display the same information show different results. This is the highly damaging, as trust in the data evaporates and everyone retreats to an insular defensive position. Consequently,

the need for a single source of truth – a data warehouse – becomes essential as all analytics and reporting can be distilled from the same data within the same source.

Information overload: Ironically BI systems are often undesirable for executives and managers simply because there are just too many reports all from different perspectives. What BI tools can do is let organizations reduce the number of reports or at least the detail by:

Combining multiple information types in single views (dashboards)

Show data on a higher (summary) level while giving operational users access to detailed data through drill-down. Thus, users such as executives and managers who typically work at the strategic or tactical level will only see summarized data and only deal with detailed data when it is relevant.

Culture: As we discussed earlier some companies have a culture that makes them more receptive to using data pro-actively. This doesn't mean that they are slow moving and like to make detailed measurements of every process or document every step in a procedure it just means that they can see the benefits that can be derived from the data. Other organizations perhaps due to market conditions have to be agile and fast moving and that necessitates a great deal of reactive thinking in which case they will act first on 'gut instinct' – experience – and consider the results later. However as long as the BI data analytical process is transparent to the operations, i.e. it is not an impediment to fast moving operations, then either way data analytics is important. Organization culture may then be an enabler and not a barrier to successful implementation depending on how the operational teams view its immediate impact.

Shrinking response time: The faster an organization can derive insights and take the appropriate action, the more agile and hence competitive it will be in a fast moving and volatile market. Managers are in a constant need to get to up-to-the-minute information to help with decision-making and they need this information as fast as possible.

Some of the notable features and functions of an Online Analytical Processing system are in the way in which business users can slice and dice their way through data using sophisticated tools that allow for the navigation of dimensions such as time, or dig deep into their hierarchies. Usually the display is some numeric measure such as "sales dollars."

OLAP systems also support robust Query and Reporting functionality as they have capabilities of being able to ask questions of the data that may not have been asked before. That is, reports constructed to answer novel questions or where the presentation is not yet defined. Once generated, the report (or just the report instructions) can be saved for future viewing. In traditional reporting and analytical systems the Querying usually involved users entering SQL or an English-translation of SQL commands and an immediate execution environment where results can be viewed. In modern OLAP systems the query can take the form of natural language and this allows much more natural queries such as "how much was the sales of iPhones in January?" The adoption of natural language processing in BI has gone a long way towards self-service and the democratization of information throughout the business.

OLAP also caters for the application of many Advanced Analytics techniques—sometimes referred to as data mining, forecasting or predictive analytics, this takes advantage of existing statistical analysis techniques to predict or provide certainty measures on facts. Other key features on an OLAP system are:

- Collaborative BI—refers to the concept of taking structured data, analysis and/or reports and combining unstructured content (such as comments, discussions, attachments) so that facts can be augmented by additional content.
- Corporate Performance Management (Portals, Scorecards, Dashboards)—this general category usually provides a container for several pieces to plug into so that the aggregate tells a story. For example, a balanced scorecard

that displays portlets for financial metrics combined with say organizational learning and growth metrics.

- Proactive BI (BAM)—also known as real time BI, allows for the real time distribution of metrics through email, messaging systems and/or interactive displays. Taken together, they create BI. That is not to say all of these must exist in a single system in order to be consider a BI environment, but some or all of these must be present.

Although an increasingly popular and accepted technology for improving business performance, BI - like any other system - must be carefully considered before deciding to implement it. Benefits must be clear and understood and they must justify the investments. This is an important point as many businesses and not just SMBs are unable to articulate the benefits from their data analytics or BI projects. Some businesses will claim that there have undoubtedly been benefits but they just cannot quantify them but that can sound a bit like a case of buyers regret or worse that they do not wish to admit an investment may have been wasted.

However even with the most un-ambitious Business Intelligence projects there will be some quantifiable gains such as through:

Reduced labor costs

The most tangible benefit of BI is the time and effort saved with manually producing the standard reports for the organization. It is rarely the largest benefit though but it will reduce the angst suffered by managers once a month. However, because it is so tangible it is often part of the equation when a decision must be made about implementing BI, and if it turns out that these savings alone can justify the BI system, then it is the easiest way to justify it.

BI systems reduce labor costs for generating reports by:

automating data collection and aggregation

automating report generation

providing report design tools that make programming of new reports much simpler

reducing training needed for developing and maintaining reports

Reduce information bottlenecks

Another measurable benefit that can be had from a BI project is that the ensuing BI system will facilitate end-users to extract their own reports when they need them rather than depending on IT or the financial department to prepare them. The BI system will even allow authorized users to design new reports to match their requirements and run them when they wish and this is another benefit of self-service.

BI systems reduce information bottlenecks by:

providing individualized, role-based dashboards that collect the most important data for daily operations

letting the user open and run reports autonomously

providing documentation of KPIs and other information

allowing users to analyze and validate the data without involving IT specialists

allowing users to create new views of data as needed

Making data available and actionable

Many SMB organizations either fail to make information available to their staff or they make far too much information available which has a similar effect. Information after all is only useful when it is made available at the correct time and at the correct level of detail (the grain) for the recipient. Information will only become actionable when the BI system:

provides information through unified views of data where KPIs are assembled and calculated using a central repository of definitions - a data model - to prevent conflicting definitions and incomparable report data

provide up-to-the-minute information in real-time reports that show the state of the business in this very moment - not a historical view of how it looked days or weeks ago

permit users to search and design reports autonomously on a self serve basis instead of being dependent on specialists in the IT department

show data in a context, e.g. by benchmarking KPI values against comparable values (e.g. averages, budgets/target and last period) to let the user interpret whether the KPI value is acceptable or needs corrective action

applies rules to highlight KPI thresholds as "good" or "bad"

provides integrated documentation to help the user understand the meaning and definition of the KPIs

provides links back to the operational systems that make it easy for the user to carry out corrective actions (closed loop)

provides for data collaboration, e.g. let the user forward and share selected data with other users and assign targets and responsible persons to KPIs

only showing data relevant to the specific user in a role-based environment to avoid "Information overload"

showing data on a high, aggregated level where overall trends can be easily spotted and then let the user drill-down to detail data in a top-down manner

using intuitive visualizations that enhance on the nature of the data such as graphs/charts and gauges

Forwarding relevant information based on the occurrence of predefined events, i.e. only sending certain reports when specific business events occur, such as too high stock levels, customer churn etc.

shortening the analyze-decision loop to avoid losing the train of thought

Assisting in Decision-making

Originally Bi and Data Analytics were called Decision Systems and were used to aid executives in making the hard decisions that need to be made every day. Good decisions can provide tremendous benefits but alas bad decisions provide no benefits –or they can make a bad situation even worse.

BI systems help make better decisions by:

providing decision makers with rich, exact and up-to-date information

letting users dive into data for further investigation

In this context the term decision maker needs to be seen in a broad perspective; it is not only management that makes decisions. In fact, the decisions that affect an organization the most are those made by people all over the organization, from the sales person who decide to give a customer a discount to the procurement assistant who decides to buy certain products for inventory. Indeed, an interesting fact is that most valuable suggestions for changes in operational efficiency come from the workforce and not just data scientists.

Making quicker decisions

For managers it is safe to say that a sound decision can only be made the moment you have all the relevant information at your hands. In other words, the faster the relevant information gets into your hands the faster you can make a sound decision based on facts. Fast decisions are important for two reasons:

It makes the organization more agile and hence more responsive to business threats and equally opportunities

It shortens the time between thought and action. Most people will lose their train of thought if they need to wait a long time for further information about the problem they are dealing with.

Consequently, BI systems enable fast decisions by:

Combining multiple data sources in common reports, thus saving the user from manually combining data in spreadsheets etc.

Providing analytical and ad-hoc reporting capabilities that allow users to quickly retrieve new or different combinations of data as needed instead of having to request new reports in the IT or financial departments.

Providing reduced system response times by using pre-aggregated data or other techniques for fast data aggregation

Align the organization towards its business objectives

The most successful organizations are those that succeed to make every person in the organization work towards a common goal – the reason for its existence as stated in the company's business objectives and the company's mission or vision statement. However it is rare to find anyone other than the CEO in a company actually knows what either of these is. But for data analytics to be successful it must be aligned with the business objectives.

The way that BI systems can help organizations align all parts of the organization towards common business objectives by:

Centralizing KPI definitions. BI reports don't calculate KPIs using autonomous queries and scripts. They retrieve KPI values and definitions through a central repository and thus prevent conflicting KPI definitions and values

guiding information presentation using advanced visualizations, benchmarks and KPI thresholds thus ensuring a common interpretation of the KPIs

Providing a single source of information. All reports collect their data from one source - the BI system.

"Pushing" selected information throughout the organization. By enabling organizations to push KPIs and other information to the end-users the BI system helps focus employees' attention on the most critical success factors.

assigning targets to KPI values for each organizational unit to be used for measuring the ability to achieve the goals set forth and thus pushing the organization towards the defined goals

Providing New insights

Traditional reporting systems aim to give users data according to a fixed and predefined structure. This rigid approach gives the organization answers to exactly the questions it is able to specify in advance. Modern business intelligence systems on the other hand provide ad hoc query capabilities that allow users to poke randomly around in data to get answers to any question that comes to their mind. This allows users to strengthen their understanding of the underlying patterns of the business and thus to gain new insights into the dynamics that lead to success or failure.

In another application, some BI systems provide special mathematical algorithms for finding hitherto unknown patterns in data - so-called data mining. Such algorithms comprise Cluster Analysis, Decision Trees, Neural Networks and Rule Induction. (We will cover these technical models later in detail)

Definition of a BI infrastructure for an SMB

The case for LEAN BI

Business Intelligence is an extremely useful and if done correctly a profitable discipline for most types of SMB organizations but the costs and complexity of building BI systems is often preventive for smaller organizations. However as we have already seen BI is about leveraging information and the underpinning technology such as data warehouses and data marts and all the other technologies are simply BI enablers. The discipline of BI encompasses various methodologies for handling and processing information and providing analyses with the aim to improve business results and not all of them are prohibitively expensive.

The main obstacles that SMB face when considering implementing BI are:

Cost of implementation of enterprise class solutions

Skill requirements, both technical and business skills

Infrastructure complexity

Cost of tools

Lack of a business justification (cost/benefit analysis) as benefits aren't clear

However, if we look upon these challenges from the perspective of a smaller company and consider their constraints we can see that the typical small organizations have to somehow overcome the issues of having:

All their primary data in one or more usually proprietary systems, typically a CRM and also an ERP system, which they have very limited control over or access to the database

Limited volumes of data

Limited knowledge about the inner workings of their ERP system and SQL databases either in-house or through third party integrators

Limited access to the code for their proprietary systems meaning all interaction is through proprietary APIs, which can be expensive

Require external assistance when constructing new reports or customizing existing ones due to the issues stated above

Therefore, the two ways that we can approach this dilemma is through the use of a single uniform homogeneous solution that provides all the base points and functionality or through a mash up of open source solutions. Typically the former will be more expensive but ultimately easier to implement and use. The latter, the open source mash-up will certainly be cheaper but is only viable if there exists highly motivated and skilled technical employees over the whole gamut of technology. What that means, is you need extensive systems, network, database and application development skills all rolled into possibly only one or two employees as they will need to design, implement, integrate and support these systems through its lifecycle.

Therefore, realistically the aim would be to standardize the technology and if possible reduce the complexity of the ETL process of connecting and uploading data into the warehouse as this alone may reduce around 75% of the total costs.

Therefore the ideal BI infrastructure conceptual design can be defined as having these main attributes:

Use as few infrastructure components as possible

Consolidation of end-users tools into as few products as possible

Make development cycles as short as possible. Particularly the first cycle.

Use standard technologies with vendor support

Sometimes the above requirements conflict so striking the right balance between the above requirements is one of the most critical planning tasks in BI system development.

In a minimal and thus optimal LEAN scheme the BI system must:

Be standards based and cloud hosted

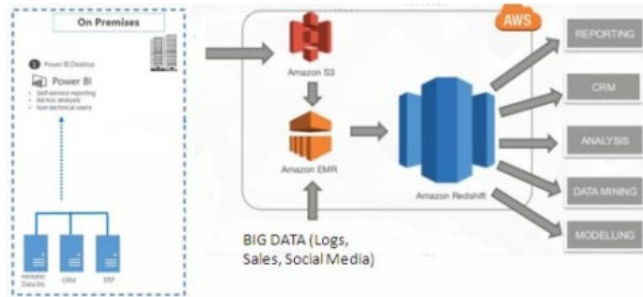
Have no on-premises middleware so ideally it must handle operational data directly from the source systems

Be based on one end-user tool which they buy into

Be able to be implemented easily and within a day

An SMB BI infrastructure provides a flexible framework that allows a BI system to be built to match the BI ambitions of an SMB. The goals are to keep the design, implementation and commissioning with the vendor or system integrator and make the system easy to manage by offloading most of the challenging maintenance and housekeeping into the cloud.

Introducing a Lean BI Infrastructure using on-premises and Cloud tools



The advantage of using a LEAN BI solution such as the architecture above is that it is very quick to implement and can be used with either the MS SQL design above or the simple DIY warehouse design built upon open source MySQL. Either way we can connect to the AWS cloud and Amazon S3 database storage using Amazon's Data Pipeline, or an open source ETL solution such as Pentaho.

One of the issues with AWS is that there is so much choice of tools it is very confusing as to which is the best fit for the purpose in hand. In the architecture above we use AWS S3 as the storage engine, Redshift as our cloud based warehouse and AWS EMR as the Hadoop (Big Data) ecosystem. We will briefly look at what each of these services can provide for the SMB.

AWS S3

Amazon Simple Storage Service is object file storage for the Internet. As it is designed to make web-scale computing easier Amazon S3 has a simple web services interface that you can use to store and retrieve any amount of data, at any time, from anywhere on the web. It provides a cheap, highly scalable, reliable, fast, data storage infrastructure.

AWS EMR

Amazon EMR provides a managed Hadoop ecosystem framework that makes it easy, fast, and cost-effective to process vast amounts of data across dynamically scalable Amazon EC2 instances. EMR enables you to run other popular distributed Hadoop tools such as Apache Spark, HBase, Presto, and Flink. In addition, Amazon EMR can interact with data in other AWS data stores such as Amazon S3 and Amazon DynamoDB as well as connect to our Redshift cloud based data warehouse.

Furthermore, Amazon EMR provides SMBs with the capability to securely and reliably handle a broad set of big data use cases, including log analysis, web indexing, data transformations (ETL), machine learning, financial analysis, scientific simulation, and bioinformatics.

In the architecture the functions of reporting, analysis, data mining and modeling have been left to the user's choice of tool. This is because any tool that can connect using ODBC or JDBC, which is just about everything on the open source or proprietary market can easily connect and integrate with Redshift or AWS S3.

This Lean Warehouse and BI design is extremely cost effective and high performance, it is an extremely good fit if your data is generated and resides on the Internet. That perhaps is a clue as to its one drawback – loading the data generated and stored on the premises. Redshift and S3 both have APIs and connectors that will make integrating with on-premises servers relatively easy. For example you can use Amazon's standard Data Pipeline for most standard data sources, if not Amazon partners with many ETL service providers, so connecting the plumbing will not be the problem. The issue is transporting all that data up through an SMB internet pipe to the

cloud. Therefore, you should think carefully about where you process your data especially if you are doing serious data mining.

Chapter 14 -Data Mining for BI in the SMB

Every business today it seems is in the process of a digital transformation, a casting off of old analogue processes and procedures and the embracing of new digital technologies. Data is a by-product of this digital phenomenon and so we are deluged by the stuff some of it mundane and useless but some may have potential value. The common metaphor of “Data is the new oil” or “Data is gold” are certainly hyperbole and overused but they may serve some purpose in so much as they highlight the potential end product of data analytics as being something of value – information, which comes in the form of answers.

As humans we are insatiably curious by nature so searching for the more profound questions in life has always been high on our academic agenda. Data mining provides a very auspicious method for probing those curiosities as combined with keen intellect and technology such as algorithms we can set about harvesting patterns, correlations and sometimes even answers to profound questions from seemingly the waste products of operational procedures – raw data.

In order to really benefit from the analysis of data we need to have clear methodologies and practices, and certainly as SMB’s realistic goals. Once, only a generation ago we would have had to temper our ambitions as building the infrastructure required for serious data mining within an SMB organisation would have required deep pockets and a benevolent benefactor willing to indulge in a non-core operational and somewhat ethereal undertaking. However, the advent of Cloud computing changed all that with the introduction of Amazon’s AWS cloud service in the early to mid- 2000s and since then everyone can afford to indulge their curiosity and research their most profound and challenging business problems using data mining.

Significant progress has also been made over the last decade in the rise of Artificial Intelligence and Machine Learning, which complement data mining. These technologies, although they are strictly different disciplines as members of the Data Science community, still play a large part in data mining as they share not just a common heritage but mostly synonymous goals. Many may dispute that last sentence but from an SMB perspective there is no significant differentiation between data analytics, business intelligence, data mining or data science they all share the same roots and although the methodologies may have advanced through technology the ultimate goals remain the same. Certainly, in large enterprises or web sized technology organisations there may well be a separation of roles and responsibilities regards a Data Scientist and a Data Analyst but at the SMB level where no distinction of duties exists the question of difference is moot.

However where there is a significant difference is in the methodology and the scale and type of technology required to perform each task, whether that be data analytics, BI or data mining. Therefore in this section we will consider what type of infrastructure we require, why it is needed, where we can obtain it and finally we will propose a suitable SMB style data mining infrastructure.

Introduction to Data Mining Algorithms

This section will provide a high-level overview of some of the most common data mining algorithms in use today. Traditionally there were the classical statistical techniques based upon classification and regression but lately newer technologies have become popular so it is fair to say there are two main groups of algorithms:

Classical Techniques: Statistics, Nearest Neighbour and Clustering

Next Generation Techniques: Decision Trees, Bayesian Networks and Association Rules

Despite the large number of specific data mining algorithms and patterns developed over the years, there are only a handful of fundamentally different types of tasks these algorithms address.

As a result the six broad classes of data mining algorithms listed above cover most use cases and one of the algorithms from this group of six is almost always used in real world deployments of data mining systems.

An interesting aspect of Data Analytics and data mining is that just about every data-driven business decision-making problem is likely to be unique in so much as it will comprise its own combination of goals, constraints, and desired outcomes. Fortunately they will also be very similar in most of their business goals and attributes so there are sets of common processes that underlie the analytical tasks.

Therefore it is a required skill for data scientists' to recognise and categorise the business problem, then to decompose it into subtasks. It might turn out that some of these subtasks are unique to the particular business problem, but more often than not other sub-tasks are common data mining tasks that have been solved and documented. These common tasks and their solutions are now incorporated into a body of knowledge or framework for decomposing common data mining tasks. Hence, having knowledge of these frameworks and their ability to decompose a data-analytics problem into pieces such that each piece matches a known task is invaluable. After all having the ability to recognize familiar problems and their solutions for which tools, libraries and procedures are available avoids wasting time and resources reinventing the wheel.

Statistical Techniques

The use of Statistical techniques go back hundreds of years but are still relevant as they are data centric and are especially useful in discovering patterns and building predictive models. Indeed there is little practical difference between a statistical technique and a data mining technique, the label has changed more to accommodate the scale of the data analysed and the variety of the source data.

Knowing statistics will help the average business person make better decisions by allowing them to figure out risk and uncertainty when all the facts either aren't known or can't be collected indeed it might be the one thing that defines the gut feeling manager. The better the understanding of statistics, even if it is not recognised as such, the better the decision that can be made. For example, it could be the ancient Egyptians and Greeks knew about probability and statistics long before the field was officially recognized.

Today some of the techniques used in data mining such as CHAID and CART really grew out of the statistical profession more than anywhere else, and the concepts of probability, independence and causality and overfitting are the foundation on which both data mining and statistics are built.

Today people have to deal with terabytes of diverse types of data and they have to make sense of it and glean the important patterns. Statistics can help greatly in this process by helping to answer several important questions about raw data:

What patterns are in the database?

What is the probability that an event will occur?

Which patterns are significant?

What is a summary of the data that describes what is contained in the database?

The last point is interesting because statistics is concerned with summarizing data, and this has to do with counting. Therefore, statistics can present a high level view of a database that provides some useful information without requiring every record to be understood in detail. Summarization is used in the reporting of important information from which people may be able to make useful decisions such as in strategic or tactical reports.

By presenting summaries of the data in graphical format we can visualise all of the data in the database for a particular predictor or data column. However the important part is that we can also deduce the average, the maximum and minimum. These values are called summary statistics. Some of the most frequently used summary statistics include:

Max - the maximum value for a given predictor.

Min - the minimum value for a given predictor.

Mean - the average value for a given predictor.

Median - the value for a given predictor that divides the database as nearly as possible into two databases of equal numbers of records.

Mode - the most common value for the predictor.

Variance - the measure of how distant the values are from the average value.

Many data distributions are well described by just two numbers, the mean and the variance. The variance uses the square of the distance rather than the actual distance from the mean and the average is taken by dividing the squared sum by one less than the total number of records. In terms of prediction a user could make some guess at the value of a predictor without knowing anything else just by knowing the mean and also gain some basic sense of how variable the guess might be based on the variance. This is because the variance measures the average distance of each predictor value from the mean value over all the records in the database. Therefore, if the variance is high it implies that the values do not have a distinct pattern however if the variance is low most of the data values are fairly close to the mean making predictions more likely.

Prediction is a commonly used tool in statistics and is synonymous with regression of some form. There are a variety of different types of regression in statistics but the simplest form of regression is simple linear regression that just contains one predictor and a prediction. The relationship between the two can be mapped on a two dimensional space and the records plotted for the prediction values along the Y axis and the predictor values along the X axis.

The simple linear regression model then could be viewed as the line that minimized the error rate between the actual prediction value and the point on the line (the prediction from the model). Of the many possible lines that could be drawn through the data the one that minimizes the distance between the line and the data points is the one that is chosen for the predictive model.

The actual equation would look something like: $\text{Prediction} = a + b * \text{Predictor}$. Which is just the equation for a line $Y = a + bx$. As an example, for a bank the predicted average consumer bank balance might equal $\$1,000 + 0.01 * \text{customer's annual income}$. The trick, as always with predictive modeling, is to find the model that best minimizes the error. The most common way to calculate the error is the square of the difference between the predicted value and the actual value. The values of variables 'a, and b' in the regression equation that minimize this error can be calculated directly from the data relatively quickly.

However, regression can become more complicated than the simple linear regression we've introduced so far. For example, when trying to predict a customer response that is just 'yes' or 'no' (e.g. they bought the product or they didn't) the standard form of a line doesn't work. Typically in these situations a transformation of the prediction values is made in order to provide a better predictive model. This type of regression is called logistic regression and because so many business problems are response problems, logistic regression is one of the most widely used statistical techniques for creating predictive models.

Nearest Neighbour

Clustering and the Nearest Neighbour prediction technique are among the oldest techniques used in data mining. Most people have an intuition that they understand what clustering is - namely that like records are grouped or clustered together. Nearest neighbour is a prediction technique that is quite similar to clustering - its essence is

that in order to predict what a prediction value is in one record look for records with similar predictor values in the historical database and use the prediction value from the record that is “nearest” to the unclassified record.

The nearest neighbour prediction algorithm simply stated is:

Objects that are “near” to each other will have similar prediction values as well. Thus if you know the prediction value of one of the objects you can predict it for its nearest neighbours.

One of the classical places that nearest neighbour have been used for prediction has been in text retrieval. The nearest neighbour technique is used to find other documents that share important characteristics with those documents that have been marked as interesting.

Unfortunately, just relying on the sole nearest neighbour to the unclassified record can be unreliable, as that may be an outlier that is unrepresentative, so one of the improvements that are usually made to the basic nearest neighbour algorithm is to take a vote from the “K” nearest neighbors.

In cases like these a vote of the 9 or 15 nearest neighbors would provide a better prediction accuracy for the system than would just the single nearest neighbour. The voting mechanism is accomplished by simply taking the majority of predictions from the K nearest neighbors if the prediction column is a binary or categorical or taking the average value of the prediction column from the K nearest neighbours. So basically you are taking the average prediction from the K nearest neighbours in order to improve the reliability of the prediction.

Another important aspect of any system that is used to make predictions is that the user be provided with, not only the prediction, but also some sense of the confidence in that prediction (e.g. the prediction is default with the chance of being correct 60% of the time). The nearest neighbour algorithm provides this confidence information in a number of ways:

The distance to the nearest neighbour provides a level of confidence. For example, if the neighbour is very close or an exact match then there is much higher confidence in the prediction than if the nearest record is a great distance from the unclassified record.

The degree of homogeneity amongst the predictions within the K nearest neighbors can also be used. If all the nearest neighbors make the same prediction then there is much higher confidence in the prediction than if half the records made one prediction and the other half made another prediction.

Clustering is the method by which like or similar records are grouped together. Usually this is done to give the end user a high level view of what is going on in the database. Clustering is sometimes used to mean segmentation where companies have grouped the population by demographic information into segments that they believe are useful for direct marketing and sales.

Sometimes clustering is performed not so much to keep records together as to make it easier to see when one record sticks out from the rest. For instance:

The nearest neighbour algorithm is basically a refinement of clustering in the sense that they both use distance in some feature space to create either structure in the data or predictions. The nearest neighbour algorithm is a refinement since part of the algorithm usually is a way of automatically determining the weighting of the importance of the predictors and how the distance will be measured within the feature space. Clustering is one special case of this where the importance of each predictor is considered to be equivalent.

An important constraint on clustering is that a reasonable number of clusters are formed for example when there are too few clusters (too much generalization) or too many clusters and original records, which is also unacceptable. Many clustering algorithms either let the user choose the number of clusters that they would like to see created as an input the algorithm will then strive to create that number using the data points available.

It is possible to guarantee that homogeneous clusters are created by breaking apart any cluster that is inhomogeneous into smaller clusters that are homogeneous. However this usually means creating too many clusters which defeats the original purpose of the clustering.

For clustering the n-dimensional space is usually defined by assigning one predictor to each dimension. For example, cluster with time, product, sales, colour, price, would be said to be in a 5 dimensional space. For the nearest neighbour algorithm predictors are also mapped to dimensions but then those dimensions are literally stretched or compressed based on how important the particular predictor is in making the prediction. The stretching of a dimension effectively makes that dimension (and hence predictor) more important than the others in calculating the distance.

Hierarchical and Non-Hierarchical Clustering

There are two main types of clustering techniques, those that create a hierarchy of clusters and those that do not. This hierarchy of clusters is created through the algorithm that builds the clusters. There are two main types of hierarchical clustering algorithms:

Agglomerative - With the Agglomerative clustering techniques the process starts with each record in a cluster, i.e. there are as many clusters as there are records and each cluster contains just one record. This of course defeats the purpose so the clusters that are nearest each other are merged together to form the next largest cluster. This merging is continued until a hierarchy of clusters is built with just a single cluster containing all the records at the top of the hierarchy.

Divisive - Divisive clustering techniques take the opposite approach from agglomerative techniques. These techniques start with all the records in one cluster and then try to split that cluster into smaller pieces and then in turn to try to split those smaller pieces.

Of the two the agglomerative techniques are the most commonly used for clustering and have more algorithms developed for them. The non-hierarchical techniques in general are faster to create from the historical database but require that the user make some decision about the number of clusters desired or the minimum “nearness” required for two records to be within the same cluster.

New Approaches to Data Mining

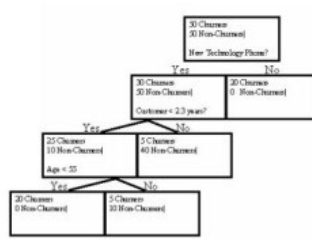
The data mining techniques in this section represent the vast majority of the techniques that are being spoken about and have been developed over the last two decades of research. These techniques can be used for either discovering new information within large databases or for building predictive models.

Decision Trees

What is a Decision Tree?

Decision trees are data mining technology that has been around in a form very similar to the technology of today for almost twenty years now and early versions of the algorithms date back in the 1960s.

A decision tree is a predictive model that, as its name implies, can be viewed as a tree. Specifically each branch of the tree is a classification question and the leaves of the tree are partitions of the dataset with their classification. For instance if we were going to classify customers who churn (don't renew their phone contracts) in the Cellular Telephone Industry a decision tree might look something like that found in the following diagram.



If you look closely, there is something interesting things about the decision tree:

It divides up the data on each branch point without losing any of the data (the number of total records in a given parent node is equal to the sum of the records contained in its two children).

The number of churners and non-churners is conserved as you move up or down the tree

It is pretty easy to understand how the model is being built (in contrast to the models from neural networks or from standard statistics).

It would also be pretty easy to use this model if you actually had to target those customers that are likely to churn with a targeted marketing offer.

You may also build some intuitions about your customer base. E.g. “customers who have been with you for a couple of years and have up to date cellular phones are pretty loyal”.

From a business perspective decision trees can be viewed as creating a segmentation of the original dataset (each segment would be one of the leaves of the tree). In this case the segmentation is done for a particular reason - namely for the prediction of some important piece of information. The records that fall within each segment fall there because they have similarity with respect to the information being predicted - not just that they are similar - without similarity being well defined. These predictive segments that are derived from the decision tree also come with a description of the characteristics that define the predictive segment. Thus although the decision trees and the algorithms that create them may be complex, the results can be presented in an easy to understand way that can be quite useful to the business user.

Applying decision trees to business problems

Because of their tree structure and ability to easily generate rules decision trees are a favoured technique when building understandable models. Because of their high level of automation and the ease of translating decision tree models into SQL for deployment in relational databases the technology has also proven to be easy to integrate with existing IT processes.

Where can decision trees be used?

Because decision trees can be used in a wide variety of business problems for both exploration and for prediction within data mining and score very highly for their effectiveness they have been used for problems ranging from credit card attrition prediction to time series prediction of the exchange rate of different international currencies. However, there are also some scenarios and business problems where decision trees will not do as well as other techniques. For example in very simple problems where the prediction is just a multiple of the predictor these can be solved much quicker using linear regression. However, commonly in business predictions the interactions to be detected are much more complex than linear regression can handle and this is where decision trees excel.

Using decision trees for Exploration

The decision tree technology can be used for exploration of the dataset and business problem. This is often done by looking at the predictors and values that are chosen for each split of the tree. Often there are times when these predictors provide insights or propose questions that need further investigation.

Using decision trees for Data Preprocessing

Another way that the decision tree technology has been used is for pre-processing data for other prediction algorithms. Because the algorithm can be run relatively quickly decision trees can be used on the first pass of a

data mining run to create a subset of possibly useful predictors, which can then be fed into neural networks, nearest neighbour and normal statistical routines.

CART - Growing a forest and picking the best tree

CART stands for Classification and Regression Trees and is a data exploration and prediction algorithm. In building the CART tree each predictor is picked as to how well they decrease the disorder of the data. For instance one measure that is used to determine whether a given split point for a give predictor is better than another is the entropy metric.

CART automatically validates the Tree

One of the great advantages of CART is that the algorithm has the validation of the model and the discovery of the optimally general model built deeply into the algorithm. CART accomplishes this by building a very complex tree and then pruning it back to the optimally general tree based on the results of cross validation or test set validation. The tree is pruned back based on the performance of the various pruned version of the tree on the test set data. The most complex tree rarely fares the best on the held aside data as it has been overfitted to the training data. By using cross validation the tree that is most likely to do well on new, unseen data can be chosen.

CHAID

Another equally popular decision tree technology to CART is CHAID or Chi-Square Automatic Interaction Detector. CHAID is similar to CART in that it builds a decision tree but it differs in the way that it chooses its splits. Instead of the entropy for choosing optimal splits the technique relies on the chi square test used in contingency tables to determine which categorical predictor is furthest from independence with the prediction values.

Because CHAID relies on the contingency tables to form its test of significance for each predictor all predictors must either be categorical or formed via binning (e.g. fitting age ranges into 10 bins from 0-9, 10-19, 20-29 etc.). CART and CHAID have been shown to be comparable in real world direct marketing response models.

Association Rule Induction

Association Rule induction is perhaps the form of data mining that most closely resembles the imaginary process of data mining, namely “mining” for gold through a vast hoard of raw data. Rule induction on a data base can be a massive undertaking where all possible patterns are discovered then an accuracy and significance are added to them that indicates how strong the pattern is and how likely it is to occur again. In general these rules (patterns) are relatively simple such as for a supermarket basket database of items scanned at the check-out there might be some interesting correlations such as:

If bagels are purchased then cream cheese is purchased 90% of the time and this pattern occurs in 3% of all shopping baskets.

If live plants are purchased from a hardware store then plant fertilizer is purchased 60% of the time and these two items are bought together in 6% of the shopping baskets.

The rules that are pulled from the database are extracted and ordered to be presented to the user based on the percentage of times that they are correct and how often they apply.

The bane of rule induction systems is it retrieves too much data and this overabundance of patterns can also be problematic as there may be conflicting predictions made by equally interesting rules. Therefore there is an active field of research looking at automating the process of culling and combing the recommendations to reduce the output.

Applying Rule induction to Business

Rule induction systems are highly automated and are probably the best of the data mining techniques for exposing all possible predictive patterns in a database. They can be modified for use in prediction problems but the algorithms for combining evidence from a variety of rules comes more from rules of thumbs and practical experience.

What is an association rule?

In rule induction systems the rule itself is of a simple form of “if this and this and this then this”. In order for the rules to be useful there are two pieces of information that must be supplied as well as the actual rule:

Accuracy - How often is the rule correct?

Coverage - How often does the rule apply?

Just because the pattern in the data base is expressed as rule does not mean that it is true all the time. Thus just like in other data mining algorithms it is important to recognize and make explicit the uncertainty in the rule. This is what the accuracy of the rule means. The coverage of the rule has to do with how much of the database the rule “covers” or applies to.

The rules themselves consist of two halves. The left hand side is called the antecedent and the right hand side is called the consequent. The antecedent can consist of just one condition or multiple conditions which must all be true in order for the consequent to be true at the given accuracy. Generally the consequent is just a single condition (prediction of purchasing just one grocery store item) rather than multiple conditions.

Since there is both a left side and a right side to a rule (antecedent and consequent) they can be used in several ways for example we can:

Target the antecedent. In this case all rules that have a certain value for the antecedent are gathered and displayed to the user.

Target the consequent. In this case all rules that have a certain value for the consequent can be used to understand what is associated with the consequent and perhaps what affects the consequent.

Target based on accuracy. Sometimes the most important thing for a user is the accuracy of the rules that are being generated. Highly accurate rules of 80% or 90% imply strong relationships that can be exploited even if they have low coverage of the database and are only likely to appear a limited number of times.

Target based on coverage. Sometimes user want to know what the most ubiquitous rules are or those rules that are most readily applicable. By looking at rules ranked by coverage they can quickly get a high level view of what is happening within their database most of the time.

Target based on “interestingness”. Rules are interesting when they have high coverage and high accuracy and deviate from the norm. There have been many ways that rules have been ranked by some measure of interestingness so that the tradeoff between coverage and accuracy can be made.

Caveat: Rules do not imply causality

It is important to recognize that even though the patterns produced from rule induction systems are delivered as if-then rules they do not necessarily mean that the left hand side of the rule (the “if” part) causes the right hand side of the rule (the “then” part) to happen. This is particularly important to remember for rule induction systems because the results are presented as if-this-then-that as many causal relationships are presented.

Types of databases used for rule induction

Typically rule induction is used on databases with either fields of high cardinality (many different values) or many columns of binary fields. A super market basket data may contain individual product names and quantities for tens of thousands of different items with different packaging that create hundreds of thousands of SKU identifiers (Stock Keeping Units).

If we consider the typical Star Schema for many data warehouses that store the supermarket transactions as separate entries in the fact table. This layout of the data is not typically the best for most data mining algorithms which would prefer to have the data structured as one row per shopping basket and each column to represent the presence or absence of a given item. This isn't feasible since the typical grocery store contains 60,000+ SKUs or different items. This structure of the records can also create a very high dimensional space (60,000 binary dimensions) which would be unwieldy for many classical data mining algorithms like neural networks and decision trees.

Rules in knowledge discovery

Rule induction systems may not be so great at prediction (because rule induction systems produce many rules for a given antecedent or consequent there can be conflicting predictions with different accuracies) but where they excel is for knowledge discovery in unsupervised learning systems. These systems display a nice combination of both micro and macro views of data where:

Macro Level - Patterns that cover many situations are provided to the user that can be used very often and with great confidence and can also be used to summarize the database.

Micro Level - Strong rules that cover only a very few situations can still be retrieved by the system and proposed to the end user. These may be useful when the patterns discovered are highly valuable or represent a small but growing subpopulation, which may indicate a market shift or the emergence of a new competitor.

The premise of a rule classification system is that rules are created that show the relationship between events captured in your database. These rules can be simple with just one element in the antecedent or they might be more complicated with many column value pairs in the antecedent all joined together by a conjunction (item1 and item2 and item3 ... must all occur for the antecedent to be true).

The rules are used to find interesting patterns in the database but they are also used at times for prediction. There are two main things that are important to understanding a rule:

Accuracy - Accuracy refers to the probability that if the antecedent is true that the precedent will be true. High accuracy means that this is a rule that is highly dependable.

Coverage - Coverage refers to the number of records in the database that the rule applies to. High coverage means that the rule can be used very often and also that it is less likely to be a spurious artifact of the sampling technique or idiosyncrasies of the database.

The business importance of accuracy and coverage

From a business perspective accurate rules are important because they imply that there is useful predictive information in the database that can be exploited - namely that there is something far from independent between the antecedent and the consequent. The lower the accuracy the closer the rule comes to just random guessing. If the accuracy is significantly below that of what would be expected from random guessing then the negation of the antecedent may well in fact be useful (for instance people who buy denture adhesive are much less likely to buy fresh corn on the cob than normal).

From a business perspective coverage implies how often you can use a useful rule. For instance you may have a rule that is 100% accurate but is only applicable in 1 out of every 100,000 shopping baskets. You can rearrange your shelf space to take advantage of this fact but it will not make you much money since the event is not very likely to happen. The table below displays the tradeoff between coverage and accuracy.

Accuracy Low

Coverage High Rule is rarely correct but can be used often.

Coverage Low Rule is rarely correct and can be only rarely used.

Accuracy High

Rule is often correct and can be used often.

Rule is often correct but can be only rarely used.

Rule coverage versus accuracy.

Which Technique and When?

Clearly one of the hardest things to do when deciding to implement a data mining system is to determine which technique to use when. When neural networks are appropriate and when are decision trees appropriate? When is data mining appropriate at all as opposed to just working with relational databases and reporting? When would just using OLAP and a multidimensional database be appropriate?

Some of the criteria that are important in determining the technique to be used are determined by trial and error. There are definite differences in the types of problems that are most conducive to each technique but the reality of real world data and the dynamic way in which markets, customers and hence the data that represents them is formed means that the data is constantly changing. These dynamics mean that it no longer makes sense to build the "perfect" model on the historical data since whatever was known in the past cannot adequately predict the future because the future is so unlike what has gone before.

Balancing exploration and exploitation

There is always the tradeoff between exploration (learning more and gathering more facts) and exploitation (taking immediate advantage of everything that is currently known). This theme of exploration versus exploitation is echoed also at the level of collecting data in a targeted marketing system: from a limited population of prospects/customers to choose from how many to you sacrifice to exploration (trying out new promotions or messages at random) versus optimizing what you already know

Chapter 15 - BI and Microsoft Azure Cloud Technologies

We saw in the previous chapter how we resolve the issue with data silos and reporting and analysis in a heterogeneous environment. There is also perhaps an even bigger issue with silos of relational data in diverse schemas we need to consider.

Information has many characteristics some forms are static and slow to grow, whilst others are dynamic, changing and growing in size rapidly. The increase in the quantity of information available skyrocketed with the advent of the internet, ubiquitous personal computing and internet enabled devices. Others though require qualitative data which requires to be sourced via customer feedback, or comments on social media.

With the advent of the WWW came a new source of data, which has led to a huge increase in the dissemination of information worldwide. This era of the Internet and the phenomena of social networking, where people can create, exchange and publish multimedia files on the WWW has created a deluge of information, and it is all information that needs to be stored somewhere.

These new formats of information not only consumes large amount of storage but they also take time to retrieve specific pieces of the information. In order to make these searches more efficient a new framework for storing information is required and this is the non-relational or unstructured database.

An unstructured database or as they are often called a NoSQL database, which is an acronym for 'Not Only SQL' is a database designed to store diverse data objects that do not fit naturally and conveniently into common SQL database schemas. The data types it will typically hold may include the textual contents of email messages, videos, photographs, social media chats, journals, system logs, multimedia objects, Twitter tweets, Facebook posts, Forum comments, etc. The names used to refer to these database types may be misleading since some objects can be highly structured and some do hold relational data in normalized form. What is more, the new type of data is not something our self-service model can readily accommodate as it is almost always in unstructured data. Therefore we need to look at redesigning our self-service SMB data analysis model in order for it to accommodate all types of data and from all sorts of sources. For example a design constraint with the retail on-premises BI model is that users Data may require to build reports where the data comes from customer surveys, support call feedback, social media comments, email, Twitter tweets or a myriad of other social media sources. This type of unified communication will provide our earlier BI models with significant challenges and again most likely managers will have to revert to much onerous manual work collating and processing the multitude of data from such diverse sources.

Furthermore the vast majority of this data may well be unstructured data and that will require different analysis techniques which are far removed from our earlier Pivot table analysis.

To address these new and significant challenges we have to look to the cloud for both the infrastructure and the intelligent services we require, but again the premise is not to lose the self-sufficiency and compatibility that we have already achieved. There are good reasons for wishing to maintain our on-premises servers and warehouse as a good maxim is:

Data sourced on-premises stays on premises, Data sourced from the cloud stays in the cloud.

Furthermore, the web is also a huge source of unstructured data that we can utilize for data analytics and there are no shortages of APIs freely available for us to use in analyzing text, reports or even photographs and video. The effects of Big Data and the availability of affordable cloud services are having a profound impact on SMB

opportunities in almost any industry, allowing companies of all sizes to more effectively serve customers to perform risk-analyses and create new revenue streams.

The IoT will also provide opportunities through the ability to connect devices to each other and to the cloud and this will be essential for businesses of all kinds and their consumers. This is because the IoT combined with the cloud, analytics, mobile and social applications have three major advantages for small and midsize businesses:

1. These technologies allow an SMB to go from an innovative idea, a pilot, or a proof of concept to an actual product or service quickly by removing big, upfront capital investments in technology and staff.
2. The technologies remove the barriers to entry so allow businesses to scale up or down quickly, providing flexibility when responding to changing customer demands, which mitigates the risks associated with having to speculate and over provision hardware, networks and capacity.
3. The technologies are global so they remove the geographic constraints and open new markets that previously were not accessible to an SMB.

Consequently, bearing that in mind we need to look towards a cloud based solution for managing Internet sourced data. The Azure Machine Learning, Azure Stream Analytics, and the Cortana Analytics Suite are enabling technologies for business intelligence that we can look to for that technology and support.

Azure SQL Server

The first step that we need to take when building our infrastructure in the cloud is to build our data warehouse. In this case we are using Azure SQL Server but you may just as well go for Amazon Redshift. Whichever one you select is all a matter of choice. However as explained earlier as we are focusing on a transition between excel workbooks and the cloud for advanced data analytics, we will be using Azure SQL Server to host our data warehouse.

Azure SQL Data Warehouse is a cloud service available through Azure. In essence it is a preconfigured, and engineered, massively parallel processing (MPP) cloud-based relational database. It can also provide scale-out and is therefore capable of processing massive volumes of data. The relevant technical capabilities of the SQL Data Warehouse are that it:

Combines the SQL Server relational database with Azure cloud scale-out capabilities.

Decouples storage from compute.

Enables increasing, decreasing, pausing, or resuming compute.

Integrates across the Azure platform.

Utilizes SQL Server Transact-SQL (T-SQL) and tools.

Complies with various legal and business security requirements such as SOC and ISO.

As we have seen SQL Data Warehouse is a massively parallel processing (MPP) distributed database system, but what does that mean?

SQL Data Warehouse works by spreading all the data being stored in the Data Warehouse across many shared-nothing storage and processing units. This style of distributed storage is commonly used in Big Data management

such as Hadoop. However with SQL Data Warehouse the data queries are distributed across a layer of locally redundant Compute devices. Compute nodes, directly and dynamically connect to the storage layer and they are responsible for executing SQL queries. SQL Data Warehouse takes a "divide and conquer" approach to handling the distributed data loads and running complex queries. It does this through the Control node, which when it receives a query it optimizes it for distribution, and then passes it to the Compute nodes to do their work in parallel.

Therefore the control node is at the front-end and it interacts with the application through SQL queries. It optimises and manages the queries by dividing them into separate smaller queries which it distributes in parallel across the array of compute nodes it manages.

Compute nodes are simply the SQL databases managers that process the query. They return the individual results back to the Control node, which then aggregates the results before passing it back to the application.

When you add data the SQL Data Warehouse distributes the rows of data across all the Compute nodes however the data is kept in Azure Blob storage, which decouples it from the Compute nodes. When the Compute nodes need to read or write to the data it accesses it directly.

By decoupling the storage from the compute, SQL Data Warehouse can:

Grow or shrink storage size independent of compute.

Grow or shrink compute power without moving data.

Pause compute capacity while leaving data intact, only paying for storage.

Resume compute capacity during operational hours.

As Azure Data Warehouse is built upon SQL Server technology it is very similar in operation and performance. Therefore if you already have experience of T-SQL then you will find the transition to Azure SQL Data Warehouse to be trivial. For example, Data Warehouse uses T-SQL syntax for many operations and it also supports a broad set of traditional SQL constructs, such as stored procedures, user-defined functions, table partitioning, indexes, and collations. However Data Warehouse does have some additional features such as clustered 'columnstore' indexes, PolyBase integration, and data auditing (complete with threat assessment). By using column-based storage, SQL Data Warehouse gets on average 5x compression gains over traditional row-oriented storage, and up to 10x or more query performance gains. In addition you will find that analytic queries that need to scan a large number of rows work better with columnstore indexes.

In addition it is very easy to provision and load data into the SQL Data Warehouse.

A Microsoft tutorial is located here:

<https://docs.microsoft.com/en-us/azure/sql-data-warehouse/sql-data-warehouse-get-started-tutorial>

The best part of using Azure SQL Data Warehouse is that it is so easily integrated with other Azure services such as the Data Factory, Data Mining or the Machine Learning studio.

Azure Machine Learning

With Microsoft Azure Machine Learning Studio, you can build, test, and deploy predictive analytics solutions that operate on your data. The Machine Learning service and development environment is cloud-based, providing compute resource and memory flexibility.

In Machine Learning Studio, you simply drag-and-drop datasets and analysis modules onto a canvas, connect them together to create an experiment, and then run the experiment. You can edit the experiment and re-run it, enabling you to iterate on your model design. Programming is not required, but if you need to extend your experiment with custom code you can use both R and Python. More languages will be supported in the future. You can also publish your experiment, or part of the experiment, as a web service that can be consumed by applications and BI tools.

An experiment typically includes data that you have either uploaded to your workspace or loaded from a cloud data source such as Azure storage or URLs. And, the experiment includes an analytical module that processes or analyzes data.

In Azure Machine Learning, you work with data in the form of datasets, which are tables of data with accompanying metadata that describes the schema and analytical usage of each column. The modules provide many tools that are important and useful in data science, including a variety of leading-edge machine learning algorithms. Other modules provide data entry functions, tools for transforming data for statistical analysis, and support for machine learning workflows such as training, scoring, and validation processes.

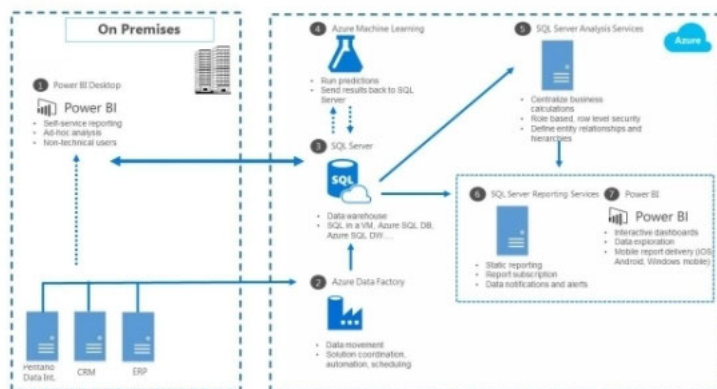
Data Mining in Analysis Services

As we saw earlier the SQL Server Analysis Services provides an integrated platform for solutions that incorporate data mining. You can use either relational or cube data to create business intelligence solutions with predictive analytics.

As a result we can use Data mining techniques in Analysis Services to discover patterns in data, helping us to make knowledgeable decisions about complex problems. Also, by applying the data mining algorithms in Analysis Services to our data, we can also forecast trends, identify patterns, create rules and recommendations, analyze the sequence of events in complex data sets, and gain new insights.

Consequently with all these tools in one toolkit we will look to MS to provide a straightforward and highly integrated platform for SMB data mining.

The infrastructure blueprint for SMB Data Mining using Azure Cloud and BI



Azure Stream Analytics

The major components that we have added to the Data Warehouse architecture are the cloud based Azure Stream Analytics, which provides complex event processing over streaming data in the cloud. The processing performed by Azure is low-latency, highly available and it has built-in recovery and check-pointing capabilities, as well as the

expected cloud scalability. Therefore we can use Azure to handle streaming data from the internet and social media such as to combine streams of data with historic records or reference data to derive business insights.

The key business scenarios that drive Azure Stream are:

The ability to handle and perform complex event processing on high-volume and high-velocity data

The capacity to collect all the high volume and velocity event data from globally distributed assets or equipment, such as IoT device sensors

To handle and process such diverse data as telemetry data for near real-time monitoring and diagnostics in industrial M2M scenarios

Capture and archive real-time events for future processing and send notifications

The real advantage of Azure Stream Analytics is that the building of a stream-processing system is easy. Indeed, all that is required is that within the Azure portal, you write a streaming task by using a language similar to SQL to specify transformations and analyze the data. What's more you do not need to know or have to write complex code. Hence, just by using the SQL-like Stream Analytics query language, you can define time-based windowed operations, correlate multiple streams to detect patterns, or even compare current conditions to historic values and models.

The Azure Stream Analytics service can scale from a few kilobytes to a gigabyte or more of events processed per second. Connection support includes connecting Stream Analytics directly to Azure Event Hubs, and to the Azure Blob service for historical data. With Azure Event Hubs, you can then integrate Stream Analytics with other data sources and processing engines. The results of the Stream Analytics job can be outputted to a number of targets, including but not limited to Azure Blob Storage, Azure Table Storage, and Azure SQL Database.

Cortana Analytics Suite

The Cortana Analytics Suite is an advanced analytics suite that enables you to transform your data into intelligent action through advanced analytics.

Advanced Analytics

Advanced analytics involves tools and techniques that are used to forecast future outcomes and behaviors and in this case we are using Data mining and predictive analytics, which are some of the analytical categories that are part of advanced analytics.

In order to build upon the infrastructure for our MS Azure Data Mining architecture we suggest the following additional tools:

The following tools support advanced analytics.

Data Mining Client for Excel

Data Mining in Analysis Services

Data Mining Client for Excel



As we discussed earlier with the self-service BI model Excel already enables you to create, test, explore, and manage data mining models using either your spreadsheet data or through some external data sources that may be accessible through your SQL Server Analysis Services instance.

Using simply Excel you can perform common data mining tasks, from data cleansing to model building and prediction queries. In addition the options for Data Modeling on the toolbar let you derive patterns from source data such as to group rows of data based on attributes, or explore associations based upon relation. Use the inherent Excel wizards on the Accuracy and Validation toolbar you can also use validation tools to test the accuracy of your models, and for assessing the viability of the data set for creating models.

Data Analysis Techniques

As we saw earlier spreadsheets and even basic databases are fine for SMB analytical needs as often the value is directly associated to the skill of the analyst. Hence, we all often see high value reports and analytical competence come from accountants and sales employees that are equipped only with office style applications and tools. However, if we wish to analyse the far less quantifiable world of social media and public opinion then we require using more sophisticated and efficient tools.

The reason for this is simply that text analysis is the model to use but it is also extremely difficult to perform. So why do we want to go through the pain of text analysis if it is so much trouble?

The answer is that is that previously we could only use traditional algorithms to reveal historical correlations amongst quantitative figures but by analysing all the unstructured data that is abundant on the web it enables us to unlock the secrets hidden amongst what was once vast incomprehensible qualitative data.

The huge advantage is that most organizations are simply scratching the surface when analysing structured data and it is only when they delve into the vastly more complex world of unstructured data that they can genuinely harvest really valuable insights.

Text analytics is about collecting insights an understanding from all sorts of diverse sources such as email, social media, Tweets, chat, website reviews, as well as comments left on paper as well as online media sites.

Analysis of social media and general communications allows a business to understand how its products or brand is appreciated amongst the community and allow it to take appropriate action to improve marketing, sales, business and operational performance.

The strange thing about unstructured data analysis is that it is all about transforming it to structured data, so that we can more easily evaluate its worth and relate it to trends or to support our hypothesis. Consequently, text analysis is commonly found in customer service, marketing or technical support where there is a high level of interaction with the public. Text analysis and its closely related cousin sentiment analysis provide a business with insights into the behaviours of customers, as well as the root causes behind customer complaints. Indeed with hi-tech cloud based technology allowing streaming analysis companies can make on-the-fly decisions, provide real-time recommendations or even detect fraudulent transactions.

Consequently the need for text analysis has shot through the roof and current trends show it to have doubled in uptake since 2010. Furthermore the future prospects are bright with even more unstructured data hitting the market. For example; as the proliferation of internet based personal assistants, such as Siri, Cortana or Alexa have not just driven text analysis but have also driven through the vast amount of voice communications data natural language recognition.

Of course we have been able to do business intelligence and data analysis since the 60's but what has changed today in 2017 is that we can do real-time and in-stream data analysis.

The power of such technology is that now we can interact with the customer in real time and determine their sentiment, their preferences or estimate their brand empathy or loyalty. Hence, the opportunities to up-sell or X-sell are vast as are the opportunities to have consumers participate in recommending our products to their social circle or provide suggestions to improve the aesthetic style, technical design or to engage in future product development.

Now that we can appreciate the value of data and importantly the new avenues of data analytics that are appropriate to even SMB, we can turn our attention towards how to turn this infrastructure and knowledge into practice.

Data Analysis Examples and Use Cases

Of course many of these use cases will not be directly comparable with your SMB business but most can be easily customised to fit most SMB business use cases.

Utilizing the Cloud for Slim Agile Data Analytics

Data analytics and big data have come under unprecedented hype over the last few years, and most executives are asking how is existing data leveraged in business? They have become convinced that it is possible to generate data within the operations and then analyse it in the cloud. They may or not be aware of the fact that analysis requires using reference data from reputable data sources? For example, if we consider an ecommerce site collects vast amounts of transaction logs produced by customers buying or perusing the site catalogue. The company will wish to analyze these web logs to gain insights in to customer preferences, demographics, shopping-cart abandonment ratios, navigation behaviour etc. to identify up-sell and cross-sell opportunities, develop new compelling features to drive business growth, and provide a better experience to their customers.

To analyze these retail web logs, requires that the business has reference historical data for comparison hence the company will need to use reference data such as customer information, shopping-cart abandonment ratio, sales figures, and perhaps even marketing campaign information, typically all the data that can be used as a comparison to real time performance KPIs. The most cost-efficient way to do this is to host the historical data in the cloud on an azure cloud. Then the company would simply ingest log data from the various ecommerce servers from the premises and compare it to reference data from the Azure Warehouse or S3 data store. The cloud technology can then process the data by using Hadoop in the cloud (Azure HDInsight) and publish the result data into a cloud data warehouse such as Azure SQL Data Warehouse or an on-premises data store such as SQL Server. This architecture and analytical workflow is easy to set up and to run once or twice a week.

All that is needed is a platform that allows the company to create a workflow that can ingest data from both on-premises and cloud data stores, and transform or process data by using existing compute services such as Hadoop, and publish the results to an on-premises or cloud data store for BI applications to consume.

Sounds easy, but how do we do this in practice? Well the architecture we outlined previously is the model to follow as the Azure Data Factory is the platform for this kind of scenarios. Azure Data Factory is a cloud-based data integration service that allows you to create data-driven workflows in the cloud for orchestrating and automating data movement and data transformation. What this means in common terms is that by using Azure Data Factory, you can create and schedule data-driven tasks (workflows or pipelines) that can collect data from several different data stores, and then handle the process/transform process so that the data can be transported and transferred into Amazon cloud compute services such as Azure HDInsight Hadoop, Spark, Azure Data Lake

Analytics, and Azure Machine Learning. Furthermore, these services can publish output data to data stores such as Azure SQL Data Warehouse for business intelligence (BI) applications to consume.

It is easy to consider Azure Data Factory as just another Extract-Load-Transform platform but currently, in Azure Data Factory, the data that is consumed and produced by workflows is time-sliced data (hourly, daily, weekly, etc.) as a result, a pipeline may read input data, process data, and produce output data once a day. You can also run a workflow just one time.

So what are some typical use cases where we can take advantage of the architecture that we have proposed using Azure Cloud services in conjunction with Excel BI for desktop, SQL Server and other self-service BI tools.

One of the very interesting things that you can do with Azure Cloud services is Text analysis to reveal topics being discussed, the sentiment, keywords and the language used. Microsoft has detailed tutorials how to on-board your application for using the Text Analysis API <https://docs.microsoft.com/en-us/azure/cognitive-services/text-analytics/quick-start> however, we will briefly discuss the key points here – one thing to note however is that the Text Analysis API only supports JSON and not XML.

A prerequisite step is to sign up for the text analytics service.

To do this you will need to login to your Azure subscription to sign up for the service and then create your account to use Text Analytics API it is under the Cognitive Services category.

After you sign up for Text Analytics, you will be given an API Key. You need to copy the primary key, as you need it when accessing the API services. Once you have accomplished these steps you are ready to use the Text Analytics API.

To use the API set the JSAN headers as shown below:

Ocp-Apim-Subscription-Key: <your API key> Content-Type: application/json Accept: application/json

Then set the input rows for JSON however the options and format are the same for sentiment, key-phrases and language. An example is set below using an optional parameter for language. The ID used must be unique and returned by the system.

There are several constraints however when using the API as each document must be a max of 10KB and the largest submitted is 1MB.

```
{ "documents": [ { "language": "en", "id": "1", "text": "First document" }, ... { "language": "en", "id": "100", "text":
```

Then POST the requests to the API as follows:

POST <https://westus.api.cognitive.microsoft.com/text/analytics/v2.0/sentiment> POST <https://westus.api.cognitive.m>

The POST above will return a JSON formatted response with the IDs and detected properties. The output for sentiment is returned for each document and is shown below with a score between 0 and 1:

```
// Sentiment response { "documents": [ { "id": "1", "score": "0.934" }, ... { "id": "100", "score": "0.002" }, ] } // Key
```

You can use PowerBI to then visualize the results.

There is also a function for detecting the current topics or key phrases but that is due to be deprecated in August 2017, but the details are on the link above.

Microsoft Cognitive Services also provide many other language and knowledge extraction APIs that you can quickly access in applications of in web based bots. Some of the more important ones are listed below:

Cognitive Services provides five APIs for language understanding:

TheLanguage Understanding Intelligent Service (LUIS)is able to process natural language.

TheBing Spell Check API is able to recognize the difference between names, brand names, and slang.

TheLinguistic Analysis APIuses advanced linguistic analysis algorithms to process text, and perform operations such as breaking down the structure of the text, or performing part-of-speech tagging and parsing.

TheWeb Language Model (WebLM) APIprovides natural language processing, and is used for tasks such as next-word prediction or word frequency.

Knowledge extraction

In addition to the language calls Cognitive Services provides five knowledge APIs. These services are able to identify named entities or phrases in unstructured text, add personalized recommendations, provide auto-complete suggestions, and search academic papers and other research like a personalized FAQ service.

TheEntity Linking Intelligence Serviceannotates unstructured text with the relevant entities mentioned in the text. This service understands the context of the supplied text and will identify each entity in your text.

TheRecommendations APIprovides "frequently bought together" recommendations to a product, as well as personalized recommendations based on a user's history.

TheKnowledge Exploration Serviceprovides natural language interpretation of user queries and returns annotated interpretations to enable search and auto-completion functions in your application or bot.

TheAcademic Knowledge APIreturns academic research papers, authors, journals, conferences, topics, and universities from theMicrosoft Academic Graph. You can perform a search for a topic, a professor, a university, or a conference, and the API will provide relevant publications. The grammar is flexible and supports natural queries like "Papers by Michael Jordan about machine learning after 2010".

TheQnA Makeris a REST API and web-based service that is free and easy to use and it trains AI to respond to users' questions in a natural, conversational way. QnA Maker distills semi-structured data like question and answer pairs into distinct, helpful answers.

Cognitive Services also provide four APIs that you might like to use to process images or videos:

The Computer Vision API extracts information about images such as cats or people and processes text (using OCR) in images. However, it can also be used to determine if the image contains explicit content.

The Emotion API analyzes human faces and recognizes their emotion across eight possible categories of human emotions.

The Face API detects human faces, compares them to similar faces, and can even organize people into groups according to visual similarity.

The Video API analyzes and processes video to stabilize video output, detects motion, tracks faces, and can generate a motion thumbnail summary of the video.

Also by alasdair gilchrist

Data Science

[Spreadsheets To Cubes \(Advanced Data Analytics for Small Medium Business\)](#)

Thinking Machines

[Robotics: from Mechanical to Sentient Machines](#)

[A.I: The Path towards Logical and Rational Agents](#)

[Machine Learning: Adaptive Behaviour Through Experience](#)

Standalone

[Red Herring](#)

[Seven](#)

[The Katoey of Gethsemane Soi](#)

[Virgin Auction](#)

[You Can't Always Get What You Want](#)

[The Certified Ethical Hacker Exam - version 8 \(The concise study_guide\)](#)

[Emotional Intelligence & Interpersonal Skills](#)

[Assertiveness and Effective Discipline](#)

[Concise Guide to Internet Marketing for the Entrepreneur](#)

[Crowdsourcing for Entrepreneurs](#)

[Management Accounting for New Managers](#)

[The Concise Guide to the Internet of Things for Executives](#)

[Concise Guide to CompTIA Security +](#)

[An Executive Guide Biometrics](#)

[A Practical Guide Wireshark Forensics](#)

[The Concise Guide to SSL/TLS for DevOps](#)

[A concise guide to PHP MySQL and Apache](#)

[Concise Guide to DWDM](#)

[Concise Guide to OTN optical transport networks](#)

[A Concise Guide to Object Orientated Programming](#)

[REST API Design Control and Management](#)

[Earning a living on the Internet](#)

[SRS - How to build a Pen Test and Hacking Platform](#)

[Concise and Simple Guide to IP Subnets](#)

[The Layman's Guide GDPR Compliance for Small Medium Business](#)

[GDPR for DevOp\(Sec\) - The laws, Controls and solutions](#)

[FinTech Rising: Navigating the maze of US & EU regulations](#)

[PSD2 - Open Banking for DevOps\(Sec\)](#)

[Tackling Fraud](#)

[A Last Minute Hands-on Guide to GDPR Readiness](#)

[Supply Chain 4.0: From Stocking Shelves to Running the World Fuelled by Industry 4.0](#)

[An Introduction to SDN Intent Based Networking](#)

[A Concise Guide to Microservices for Executive \(Now for DevOps too!\)](#)

[An Executive Guide to Identity Access Management - 2nd Edition](#)

[An Executive Guide CCPA: The Why, When, Where, What , and Who Guide to the California Consumer Privacy Act -2018](#)

[Digital Success: A Holistic Approach to Digital Transformation for Enterprises and Manufacturers](#)

[Google Cloud Platform for Data Engineering: From Beginner to Data Engineer using Google Cloud Platform](#)

[Google Cloud Platform - Networking](#)

[Google Cloud Platform an Architect's Guide](#)

OceanofPDF.com